

FPGA 应用实训

（自编资料）

2020 年 7 月

目录

目录	I
1. RGB 转灰度实验	- 1 -
1.1 算法原理	- 1 -
1.1.1 RGB, YUV, 灰度	- 1 -
1.1.2 BMP 图片格式	- 1 -
1.2 FPGA 实现架构	- 5 -
1.3 FPGA 代码实现	- 6 -
1.4 FPGA 硬件调试	- 10 -
1.5 参考文献	- 11 -
2. 基于 FPGA 的 EEPROM 数据读写	- 12 -
2.1 算法原理	- 12 -
2.2 FPGA 实现架构	- 15 -
2.3 FPGA 实现代码	- 16 -
2.3.1 e2promtop	- 16 -
2.3.2 e2prom_rw	- 18 -
2.3.3 i2c_dri	- 20 -
2.2.4 led_alarm	- 22 -
2.4 FPGA 硬件调试	- 23 -
2.5 参考文献	- 24 -
3. 基于 FPGA 的 M 序列发生器	- 25 -
3.1 算法原理	- 25 -
3.2 MATLAB 验证代码	- 25 -
3.3 FPGA 实现架构	- 27 -
3.4 FPGA 实现代码	- 27 -
3.5 FPGA 硬件调试	- 29 -
3.6 参考文献	- 29 -
4. 基于 FPGA 的 FFT 实现	- 30 -
4.1 算法原理	- 30 -
4.2 FPGA 实现架构	- 30 -
4.3 FPGA 实现代码	- 30 -
4.4 调试仿真	- 34 -
4.5 参考文献	- 35 -
5. 基于 FPGA 的阵列图像采集控制系统设计	- 36 -
5.1 算法原理	- 36 -
5.1.1 基于 UDP 的相机采集控制协议	- 36 -
5.1.2 UDP 链路建立流程	- 38 -
5.1.3 相机配置	- 39 -
5.1.4 相机数据上报	- 39 -

5.2	FPGA 实现架构	- 39 -
5.2.1	相机采集单元架构概述	- 39 -
5.2.2	FPGA 逻辑设计	- 40 -
5.3	FPGA 实现代码	- 44 -
5.4	FPGA 硬件调试	- 51 -
5.4.1	调试环境及流程	- 51 -
5.4.2	实验结果及分析	- 54 -
5.5	参考文献	- 55 -
6.	基于 FPGA 的 FLASH 在线 JTAG 编码与识别	- 56 -
6.1	算法原理	- 56 -
6.2	PYTHON、TCL 验证代码	- 56 -
6.3	FPGA 实现架构	- 58 -
6.4	FPGA 实现代码	- 59 -
6.5	FPGA 硬件调试	- 75 -
6.6	参考文献	- 77 -
7.	基于 FPGA 的 IIR 滤波器设计	- 78 -
7.1	算法原理	- 78 -
7.2	MATLAB 验证代码	- 78 -
7.3	FPGA 实现架构	- 82 -
7.4	FPGA 实现代码	- 82 -
7.5	FPGA 硬件调试	- 88 -
7.6	参考文献	- 89 -
8.	基于 FPGA 的 LZW 压缩算法优化实现	- 90 -
8.1	算法原理	- 90 -
8.2	C++验证代码	- 91 -
8.2.1	Compress.h	- 91 -
8.2.2	Compress.cpp:	- 92 -
8.2.3	main.cpp:	- 95 -
8.3	FPGA 实现架构	- 96 -
8.4	FPGA 实现代码	- 97 -
8.6	参考文献	- 108 -
9.	基于 FPGA 的永磁同步电机伺服控制	- 110 -
9.1	算法原理	- 110 -
9.2	MATLAB 验证代码	- 113 -
9.3	C++验证代码	- 116 -
9.4	FPGA 实现架构	- 119 -
9.5	FPGA 实现代码	- 119 -
9.6	FPGA 硬件调试	- 143 -
9.7	参考文献	- 144 -
10.	基于 FPGA 的 FAST 角点检测算法 HLS 实现	- 145 -

10.1	算法原理.....	- 145 -
10.2	C++验证代码	- 146 -
10.3	FPGA 实现架构.....	- 149 -
10.4	FPGA 实现代码.....	- 150 -
10.5	X86 仿真结果.....	- 151 -
10.6	参考文献.....	- 152 -

1. RGB 转灰度实验

1.1 算法原理

1.1.1 RGB, YUV, 灰度

对一种颜色进行编码的方法统称为“颜色空间”或“色域”。用最简单的话说，世界上任何一种颜色在某一种“颜色空间”中都可定义成一组固定的数字量。

RGB（红、绿、蓝）是众多“颜色空间”的一种。采用这种编码方法，每种颜色都可用 R、G、B 三原色按照不同的亮度组合来表示。记录及显示彩色图像时，RGB 是最常见的一种方案。像素是图像的基本单元，在 RGB 格式下，一个像素需要 3 块表示，分别代表 R、G、B。对于 R、G、B，其亮度不同，给人的感受就不同，如果以 8bit 表示亮度，就有 256 种不同亮度。若想要直观感受亮度对颜色的影响，可使用电脑工具“画图 -> 编辑颜色”，通过调整 R、G、B 的值进行尝试。

YUV 是另一种颜色编码方法，在 YUV 空间中，每一个颜色有一个亮度信号 Y，和两个色度信号 U 和 V。亮度信号和色度信号分开存储，这样就可以在不影响颜色的情况下改变亮度。YUV 和 RGB 可以相互转换，有多种转换公式，常用的 RGB 数字信号转 YUV 信号公式如下：

$$\begin{aligned} Y &= (66R + 129G + 25B) \gg 8 + 16 \\ U &= (-38R - 74G + 112B) \gg 8 + 128 \\ V &= (112R - 94G - 18B) \gg 8 + 128 \end{aligned}$$

如公式所示，YUV 使用彩色图像的 RGB 信号生成一个亮度（灰度）信号和两个色度信号。把这三个信号组合起来就可以生成一个彩色图像。对于一张 RGB 格式的彩色图片，依次读取每个像素点的 R、G、B 值，计算亮度值（YUV 空间），令 R、G、B 值均等于这个亮度值，遍历所有像素点，将得到一张原图片的灰度图。

灰度图像是用不同饱和度的黑色来表示每个像素点，比如用 8 位 0-255 数字表示“灰色”程度，每个像素点只需要一个灰度值。RGB 值和灰度的转换，实际上是人眼对于彩色的感觉到亮度感觉的转换，这是一个心理学问题，有一个公式：

$$Grey = 0.299R + 0.587G + 0.114B$$

根据这个公式，依次读取每个像素点的 R、G、B 值，计算灰度值，将灰度值赋值给新图像的相应位置，所有像素点遍历一遍后完成转换。

1.1.2 BMP 图片格式

BMP（全称 Bitmap）是 Windows 操作系统中的标准图像文件格式，可以分成两类：设备相关位图（DDB）和设备无关位图（DIB），使用非常广。它采用位映射存储格式，除了图像深度可选以外，不采用其他任何压缩，因此，BMP 文件所占用的空间很大。BMP 文件的图像深度可选 1bit、4bit、8bit、24bit 及 32bit。BMP 文件存储数据时，图像的扫描方式是按从左到右、从下到上的顺序。由于 BMP 文件格式是 Windows 环境中交换与图有关的数据的一种标准，因此在 Windows 环境中运行的图形图像软件都支持 BMP 图像格式。下面详细介绍 BMP 文件的格式。

1.整体信息

BMP 格式的文件从头到尾依次是如下信息：

- （1）bmp 文件头（bmp file header）：共 14 字节；
- （2）位图信息头（bitmap information）：共 40 字节；
- （3）调色板（color palette）：为单色（1bit）、16 色（4bit）、256 色（8bit）图像文件所特有，可选；

(4) 位图数据:

最常见的就是 24 位图，所谓的 24 位图，就是说一个像素的颜色信息用 24 位来表示，也就是说，对于三原色 RGB，每一个颜色都用以一个字节（8bit）来表示。除了 24 位图，还有 1 位（单色），2 位（4 色，CGA），4 位（16 色，VGA），8 位（256 色），16 位（增强色），24 位（真彩色）和 32 位等。下图（a.bmp）是一个 640x480 像素的 BMP 格式图片，左键选中，右键单击属性，选择“详细信息”可以看到其位深位 24 位。



图 1 a.bmp

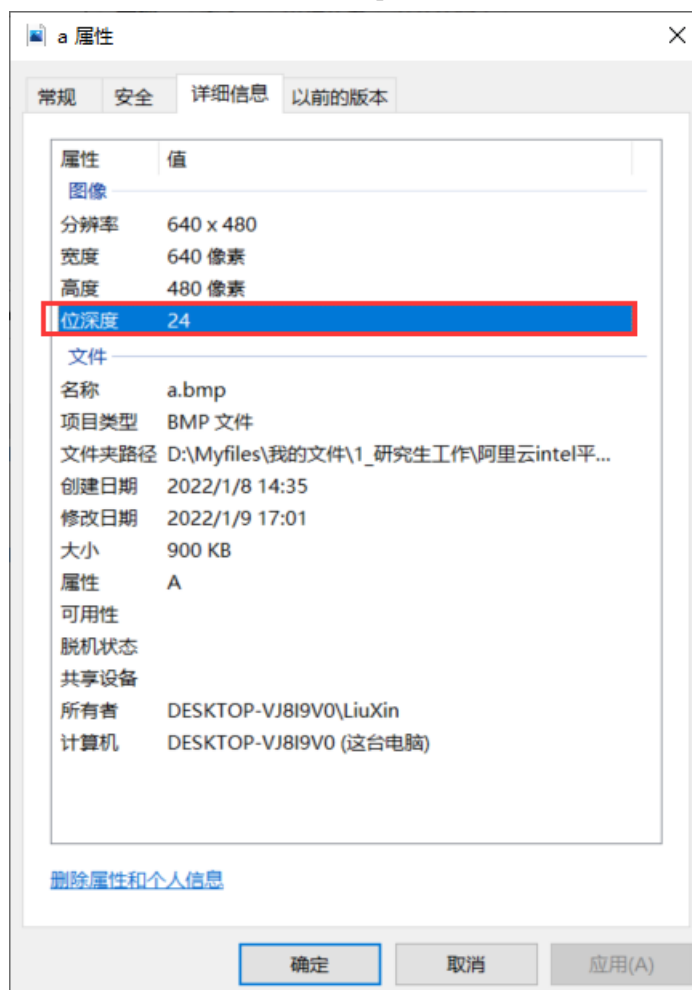


图 2 a.bmp 图片位深

2.bmp 文件头 (bmp file header)

bmp 文件头包含如下信息 (共 14 字节):

- (1) bfType: 2 字节, 文件类型;
- (2) bfSize: 4 字节, 文件大小;
- (3) bfReserved1: 2 字节, 保留, 必须设置为 0;
- (4) bfReserved2: 2 字节, 保留, 必须设置为 0;
- (5) bfOffBits: 4 字节, 从头到位图数据的偏移;

使用 UltraEdit 工具打开 a.bmp 文件, 可以看到其 bmp 文件头如下图高亮部分。下面逐个解释。

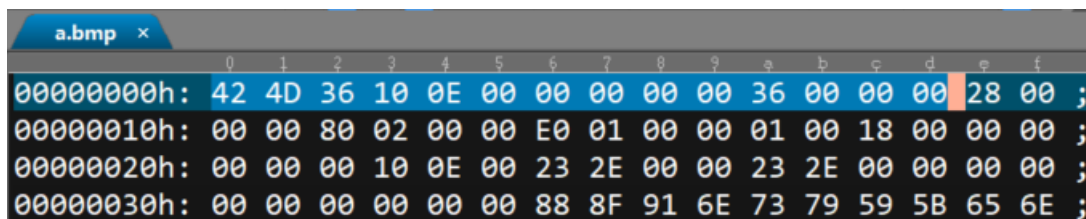


图 3 a.bmp 的 bmp 文件头数据

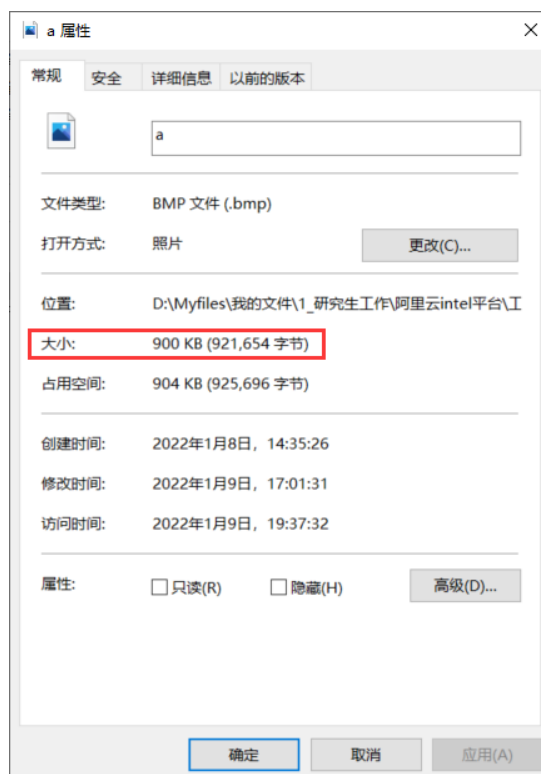


图 4 a.bmp 的文件大小

- (1) 0-1: bfType, 2 字节, 表示文件类型, BMP 格式的文件这两个字节是 0x4D42, 十进制就是 19778, 字符显示就是 “BM”;
- (2) 2-5: bfSize, 4 字节, 表示文件的大小, 这里的是 0x000E1036, 十进制是 921654, 也就是 900KB, 对比图 4 文件信息, 验证正确;
- (3) 6-7: bfReserved1, 2 字节, 保留位, 必须设置为 0;
- (4) 8-9: bfReserved2, 2 字节, 保留位, 必须设置为 0;
- (5) a-d: bfOffBits, 4 字节, 表示从文件头到位图数据的偏移(单位字节), 这里是 0x00000036, 十进制是 54。

3. 位图信息头 (bitmap information)

位图信息头包含如下内容 (40 字节):

- (1) biSize: 4 字节, 信息头的大小, 即 40;
 - (2) biWidth: 4 字节, 以像素为单位说明图像的宽度;
 - (3) biHeight: 4 字节, 以像素为单位说明图像的高度, 如果为正, 说明位图倒立 (即数据表示从图像的左下角到右上角), 如果为负说明正向;
 - (4) biPlanes: 2 字节, 为目标设备说明颜色平面数, 总被设置为 1;
 - (5) biBitCount: 2 字节, 说明比特数/像素数, 值有 1、2、4、8、16、24、32;
 - (6) biCompression: 4 字节, 说明图像的压缩类型, 最常用的就是 0 (BIRGB), 表示不压缩;
 - (7) biSizeImages: 4 字节, 说明位图数据的大小, 当用 BIRGB 格式时, 可以设置为 0;
 - (8) biXPelsPerMeter: 4 字节, 表示水平分辨率, 单位是像素/米, 有符号整数;
 - (9) biYPelsPerMeter: 4 字节, 表示垂直分辨率, 单位是像素/米, 有符号整数;
 - (10) biClrUsed: 4 字节, 说明位图使用的调色板中的颜色索引数, 为 0 说明使用所有;
 - (11) biClrImportant: 4 字节, 说明对图像显示有重要影响的颜色索引数, 为 0 说明都重要;
- 使用 UltraEdit 工具打开 a.bmp 文件, 可以看到其位图信息头如下图高亮部分。
一共 40 字节, 解释如下:

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
00000000h:	42	4D	36	10	0E	00	00	00	00	00	36	00	00	00	28	00
00000010h:	00	00	80	02	00	00	E0	01	00	00	01	00	18	00	00	00
00000020h:	00	00	00	10	0E	00	23	2E	00	00	23	2E	00	00	00	00
00000030h:	00	00	00	00	00	00	88	8F	91	6E	73	79	59	5B	65	6E

图 5 a.bmp 的位图信息头数据

- (1) 0e-11: 4 字节的 biSize, 这里是 0x28, 即十进制的 40, 验证正确;
- (2) 12-15: 4 字节的 biWidth, 这里是 0x00000280, 即十进制的 640, 用像素表示图像的宽度, 查看文件信息验证正确;
- (3) 16-19: 4 字节的 biHeight, 这里是 0x000001E0, 即十进制的 480, 用像素表示图像的高度, 查看文件信息验证正确; 同时, 这是一个正数, 表示图像是倒立的, 即图像数据是从左下角到右上角排列的;
- (4) 1a-1b: 2 字节的 biPlanes, 值为 0x0001;
- (5) 1c-1d: 2 字节的 biBitCount, 值是 0x0018, 即 24, 表示每个像素用 24 位表示;
- (6) 1e-21: 4 字节的 biCompression, 值是 0, 即 BI_RGB 格式, 不压缩;
- (7) 22-25: 4 字节的 biSizeImage, 图像的大小, 值是 0x000E1000, 十进制为 921600, 由上面的 bfSize (文件大小) 和 bfOffBits (文件头到数据的偏移) 分别是 921654 和 54 可以得到, biSizeImage=bfSize-bfOffBits, 即图像大小=文件大小-偏移量;
- (8) 26-29: 4 字节的 biXPelsPerMeter, 水平分辨率, 值是 0x00002E23, 十进制 11811;
- (9) 2a-2d: 4 字节的 biYPelsPerMeter, 垂直分辨率, 值是 0x00002E23, 十进制 11811;
- (10) 2e-31: 4 字节的 biClrUsed, 使用的颜色索引数, 值是 0;
- (11) 32-35: 4 字节的 biClrImportant, 重要的颜色索引数, 值是 0;

4. 调色板 (Color Palette)

24 位色图没有调色板。对于单色 (1bit)、16 色 (4bit)、256 色 (8bit) 图, 调色板是可选的。假如一个 BMP 文件有调色板, 那么接下来的数据就是调色板。调色板就是一个颜色的索引, 以 4 个字节为单位, 分别代表 B、G、R 以及保留位。我们知道真彩至少需要 8 位

RGB 表示，那么一个像素至少需要 24 位，可以表示 $256 \times 256 \times 256$ 种颜色。当图片为 1, 8, 24 位时，分别可以表示 2, 256, 256×256 种颜色。以 8 位为例，我们需要确定我们的 256 种颜色对应真彩的哪 256 种颜色，每一种颜色就记录在调色板中。

5. 位图数据

接下来是位图数据。由于是 24 位色图，所以每个像素用 3 个字节表示，取出每个字节，显示到相应的设备上就可以了。

注意，若 biHeight 为正数，说明图像倒立，从左下角开始到右上角，以行为主序排列。24 位色图，按照 BGR 的顺序排列，这点尤其需要注意，不要想当然以为按 RGB 顺序排列。

位图数据排列还有一个规则，就是对齐。Windows 默认的扫描的最小单位是 4 字节，如果数据对齐满足这个值的话对于数据的获取速度等都是有很大的增益的。因此，BMP 图像顺应了这个要求，要求每行的数据的长度必须是 4 的倍数，如果不够需要进行比特填充（以 0 填充），这样可以达到按行的快速存取。这样，位图数据的大小就不一定是宽*高*每像素字节数，因为每行还可能有 0 填充。填充后的每行数据如下：

$$Rowsize = 4 \times \frac{BPP \times Width}{32}$$

其中，BPP 是每像素的比特数（Bits Per Pixel），即 biBitCount，Width 是宽度，单位是像素即 bfWidth。对 24 位色图 a.bmp，按照数据可以得到：

biBitCount=0x0018=24;

bfWidth=0x00000280=640;

biHeight=0x000001E0=480;

biSizeImage=0x000E1000=921600;

按照没填充计算： $640 \times 480 \times 3 = 921600$ bytes，与真实值相同，说明没有填充。

1.2 FPGA 实现架构

本实验的系统框图如下，RGB2YUV（通用测试平台）实现 AFU 和 HOST 主机的数据交换，RGB2YUV（用户待测算法）是用户自定义部分，本实验在此部分实现了 RGB 图片转 YUV 图片算法。注意，RGB2YUV（通用测试平台）的读写控制部分可以实现对各种数据的读写，不局限于图片数据，此处仅以图片数据为例。

Intel 平台使用了 PICE 的配置接口来发送数据（BAR 空间），简称为直接访问方式（MMIO）。将 PICE 的 BAR 空间地址映射到 FPGA 内部寄存器，没有使用到 DMA。你可以理解为带宽较小的接口，原本是用来配置 FPGA 内部寄存器的。在主机上的驱动程序反映为主机的一个虚拟地址对应 FPGA 内部的寄存器，这个寄存器可以是 32 比特或者 64 比特的（可以有很多虚拟地址，每个地址都对应不同的寄存器）。但这里只有用到 64 比特中的低 24 比特来传输数据，所以每次只能传输一个像素点。同时驱动程序中的读写命令和判断 FPGA 是否正确获取数据也是通过这个接口来实现的，只不过用到了不同的地址来区分。

在存储方面，Intel 平台使用了 bram 来存储图像数据，可存储图像大小为 640×480 分辨率的 RGB 图像，其中读和写的存储单元是分离的。

在提供给用户的数据接口方面，Intel 平台采用了 VGA 时序来发送图片，每次发送一个像素点。其中图像分辨率为 **640x480（必须）**，并且会不间断的输出图片数据给用户待测算法模块。虽然使用了 VGA 时序，但是我们更改了 VGA 时序的时钟频率，提供给用户侧的时钟为 100Mhz。

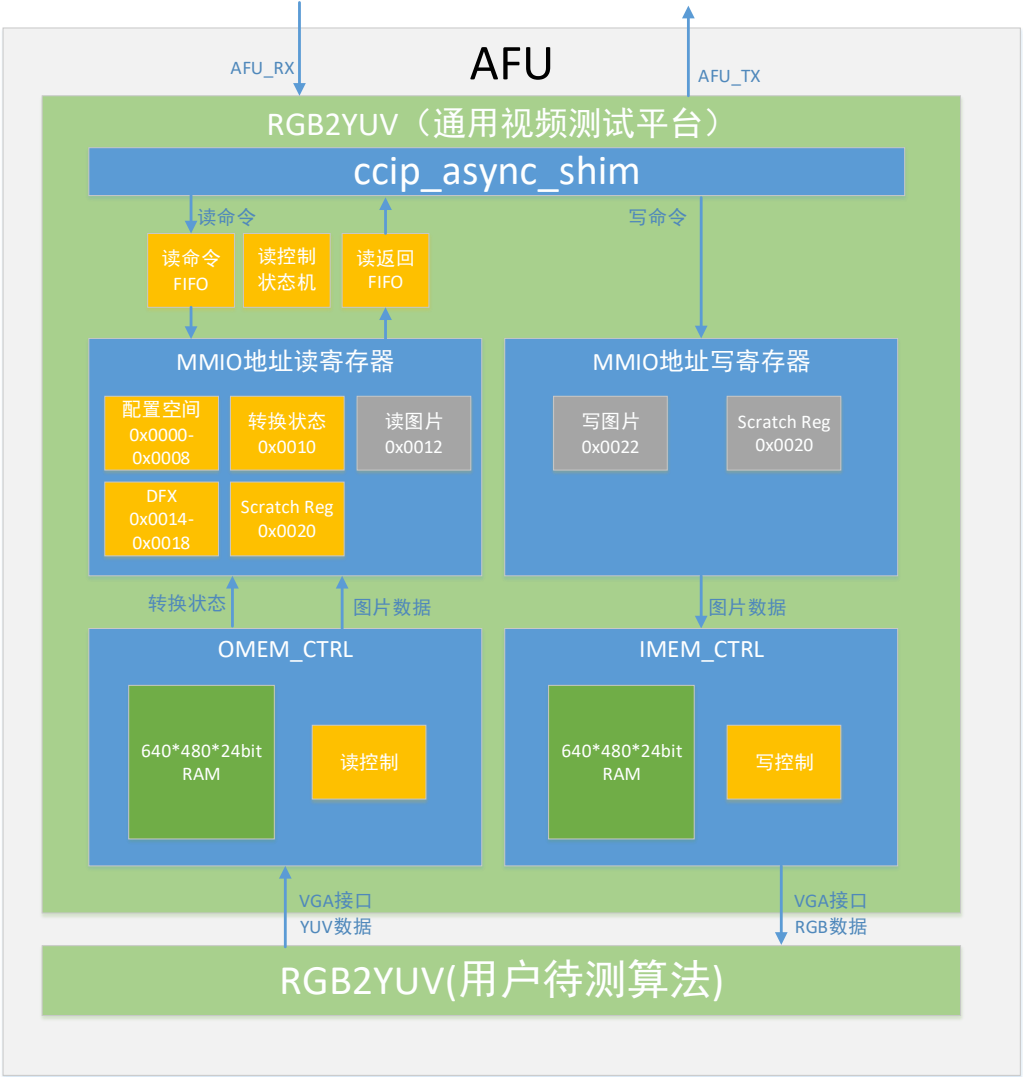


图 26 实验平台系统框图

1.3 FPGA 代码实现

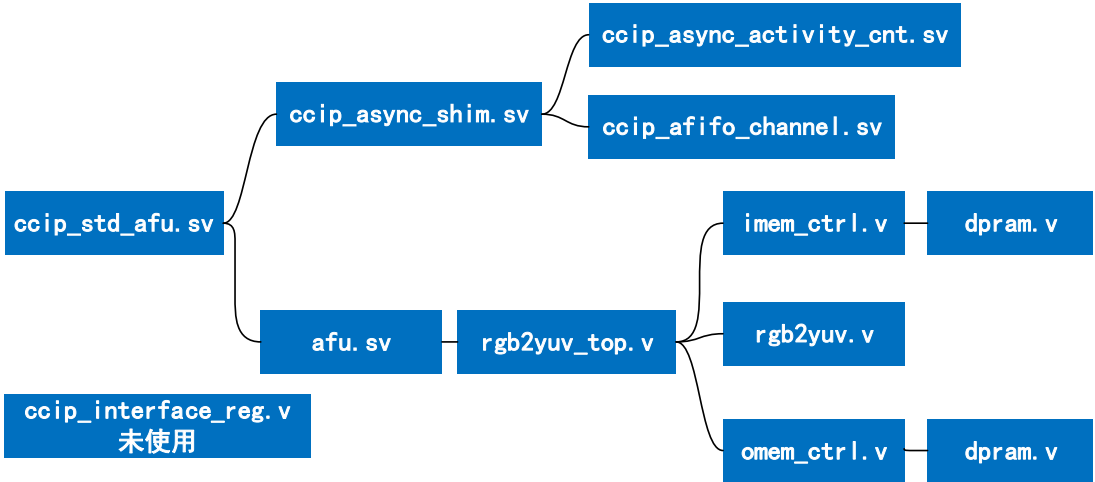


图 6 Verilog 模块关系

intel 平台下的 FPGA 模块关系如图 6 所示，其中，只有 rgb2yuv_top.v 和 rgb2yuv.v 可以修改。rgb2yuv_top.v 调用 rgb2yuv.v 模块、imem_ctrl.v 模块、omem_ctrl.v 模块，rgb2yuv.v 实现 RGB 转 YUV 的功能。其原理如下：

输入三路 8bit 的 R、G、B 数据，按照下述公式：

$$Y = (66R + 129G + 25B) \gg 8 + 16$$

$$U = (-38R - 74G + 112B) \gg 8 + 128$$

$$V = (112R - 94G - 18B) \gg 8 + 128$$

计算出 YUV 空间中的亮度信号 Y 以及色度信号 U、V，并将 Y、U、V 进行输出，Y、U、V 各 8bit。

Verilog 代码

```
module rgb2yuv(
    input                clk        ,
    input                rst_n      ,

    input    [ 7:0]      vga_r      ,
    input    [ 7:0]      vga_g      ,
    input    [ 7:0]      vga_b      ,
    input                vga_hs      ,
    input                vga_vs      ,

    output    [ 7:0]      yuv_y      ,
    output    [ 7:0]      yuv_cb     ,
    output    [ 7:0]      yuv_cr     ,
    output                yuv_hs     ,
    output                yuv_vs     ,
);

    reg    [ 15:0] y0_mult0_tmp    ;
    reg    [ 15:0] y0_mult1_tmp    ;
    reg    [ 15:0] y0_mult2_tmp    ;

    reg    [ 15:0] cb0_mult0_tmp   ;
    reg    [ 15:0] cb0_mult1_tmp   ;
    reg    [ 15:0] cb0_mult2_tmp   ;

    reg    [ 15:0] cr0_mult0_tmp   ;
    reg    [ 15:0] cr0_mult1_tmp   ;
    reg    [ 15:0] cr0_mult2_tmp   ;

    reg    [ 15:0] yuv_y_tmp        ;
    reg    [ 15:0] yuv_cr_tmp       ;
    reg    [ 15:0] yuv_cb_tmp       ;
```

```

reg      [ 23:0] yuv_rslt      ;

reg      vga_hs_1dly          ;
reg      vga_hs_2dly          ;
reg      vga_vs_1dly          ;
reg      vga_vs_2dly          ;

always @ (posedge clk or negedge rst_n)
begin : RGB2YUV_Y_MULT
    if (rst_n == 1'b0) begin
        y0_mult0_tmp <= {16{1'b0}};
        y0_mult1_tmp <= {16{1'b0}};
        y0_mult2_tmp <= {16{1'b0}};
    end else begin
        y0_mult0_tmp <= 8'd66  * vga_r;
        y0_mult1_tmp <= 8'd129 * vga_g;
        y0_mult2_tmp <= 8'd25  * vga_b;
    end
end

always @ (posedge clk or negedge rst_n)
begin : RGB2YUV_Y_RSLT
    if (rst_n == 1'b0) begin
        yuv_y_tmp <= {16{1'b0}};
    end else begin
        yuv_y_tmp <= {8'd16, 8'b0} + y0_mult0_tmp + y0_mult1_tmp + y0_mult2_tmp;
    end
end

always @ (posedge clk or negedge rst_n)
begin : RGB2YUV_CB_MULT
    if (rst_n == 1'b0) begin
        cb0_mult0_tmp <= {16{1'b0}};
        cb0_mult1_tmp <= {16{1'b0}};
        cb0_mult2_tmp <= {16{1'b0}};
    end else begin
        cb0_mult0_tmp <= 8'd38  * vga_r;
        cb0_mult1_tmp <= 8'd74  * vga_g;
        cb0_mult2_tmp <= 8'd112 * vga_b;
    end
end

always @ (posedge clk or negedge rst_n)
begin : RGB2YUV_CB_RSLT

```



```

        if (rst_n == 1'b0) begin
            yuv_cb_tmp <= {16{1'b0}};
        end else begin
            yuv_cb_tmp[15: 0] <= {8'd128, 8'b0} - cb0_mult0_tmp - cb0_mult1_tmp +
cb0_mult2_tmp;
        end
    end

    always @ (posedge clk or negedge rst_n)
    begin : RGB2YUV_CR_MULT
        if (rst_n == 1'b0) begin
            cr0_mult0_tmp <= {16{1'b0}};
            cr0_mult1_tmp <= {16{1'b0}};
            cr0_mult2_tmp <= {16{1'b0}};
        end else begin
            cr0_mult0_tmp <= 8'd112 * vga_r;
            cr0_mult1_tmp <= 8'd94  * vga_g;
            cr0_mult2_tmp <= 8'd18  * vga_b;
        end
    end

    always @ (posedge clk or negedge rst_n)
    begin : RGB2YUV_CR_RSLT
        if (rst_n == 1'b0) begin
            yuv_cr_tmp <= {16{1'b0}};
        end else begin
            yuv_cr_tmp  <= {8'd128, 8'b0} + cr0_mult0_tmp - cr0_mult1_tmp -
cr0_mult2_tmp;
        end
    end

    // Sync signal delay
    always @ (posedge clk or negedge rst_n)
    begin : HS_DLY
        if (rst_n == 1'b0) begin
            vga_hs_1dly <= 1'b0;
            vga_hs_2dly <= 1'b0;
        end else begin
            vga_hs_1dly <= vga_hs;
            vga_hs_2dly <= vga_hs_1dly;
        end
    end
end

```

```
always @ (posedge clk or negedge rst_n)
begin : VS_DLY
    if (rst_n == 1'b0) begin
        vga_vs_1dly <= 1'b0;
        vga_vs_2dly <= 1'b0;
    end else begin
        vga_vs_1dly <= vga_vs;
        vga_vs_2dly <= vga_vs_1dly;
    end
end

assign yuv_hs    = vga_hs_2dly;
assign yuv_vs    = vga_vs_2dly;
assign yuv_y     = yuv_y_tmp[15:8];
assign yuv_cr    = yuv_cr_tmp[15:8];
assign yuv_cb    = yuv_cb_tmp[15:8];

endmodule
```

1.4 FPGA 硬件调试

输入彩色图片 a.bmp，经过处理后得到其灰度图 b.bmp 图片。



图 8 a.bmp



图 9 b.bmp

1.5 参考文献

<https://opae.github.io/2.0.0/index.html#>

<https://blog.csdn.net/frozenshore/article/details/49975931>

https://blog.csdn.net/qq_34840129/article/details/93137238

<https://blog.csdn.net/giantpoplar/article/details/47295979>

2. 基于 FPGA 的 EEPROM 数据读写

2.1 算法原理

在 FPGA 平台上利用 Verilog HDL 编程实现 I2C 协议并对 AT24C EEPROM 进行读写。

本实验所用的 AT24C64 存储容量为 64Kbit，内部分成 256 页，每页 32 字节，共有 8192 个字节，且其读写操作都是以字节为基本单位。AT24C64 采用两线串行接口的双向数据传输协议——I2C 协议实现读写操作。

在 I2C 器件开始通信（传输数据）之前，串行时钟线 SCL 和串行数据线 SDA 线由于上拉的原因处于高电平状态，此时 I2C 总线处于空闲状态。如果主机（此处指 FPGA）想开始传输数据，只需在 SCL 为高电平时将 SDA 线拉低，产生一个起始信号，从机检测到起始信号后，准备接收数据，当数据传输完成，主机只需产生一个停止信号，告诉从机数据传输结束，停止信号的产生是在 SCL 为高电平时，SDA 从低电平跳变到高电平，从机检测到停止信号后，停止接收数据。I2C 整体时序如下图。起始信号之前为空闲状态，起始信号之后到停止信号之前的这一段为数据传输状态，主机可以向从机写数据，也可以读取从机输出的数据，数据的传输由双向数据线（SDA）完成。停止信号产生后，总线再次处于空闲状态。

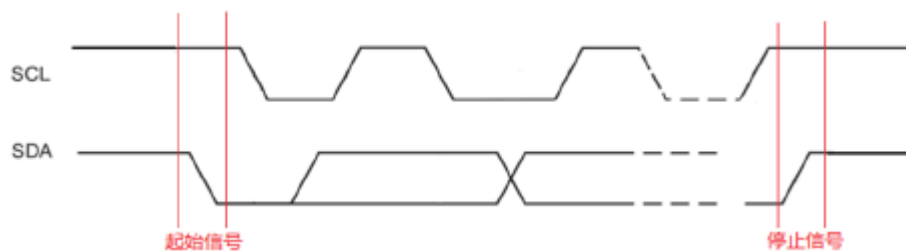


图 1 I2C 整体时序图

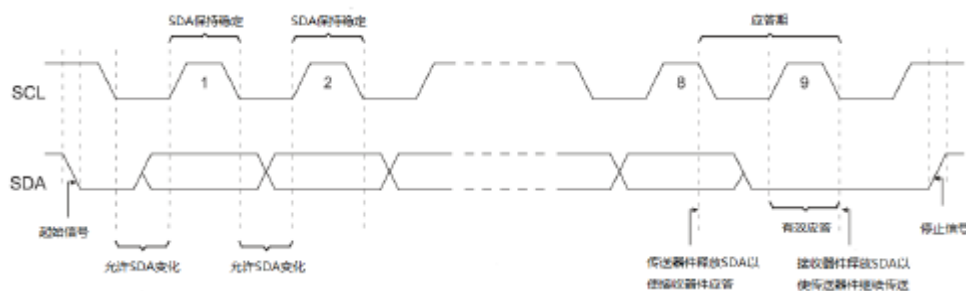


图 2 I2C 具体时序图

由图 2 可知，在起始信号之后，主机开始发送传输的数据；在串行时钟线 SCL 为低电平状态时，SDA 允许改变传输的数据位（1 为高电平，0 为低电平），在 SCL 为高电平状态时，SDA 要求保持稳定，相当于一个时钟周期传输 1bit 数据，经过 8 个时钟周期后，传输了 8bit 数据，即一个字节。第 8 个时钟周期末，主机释放 SDA 以使从机应答，在第 9 个时钟周期，从机将 SDA 拉低以应答；如果第 9 个时钟周期，SCL 为高电平时，SDA 未被检测到为低电平，视为非应答，表明此次数据传输失败。第 9 个时钟周期末，从机释放 SDA 以使主机继续传输数据，如果主机发送停止信号，此次传输结束。我们要注意的数据以 8bit 即一个字节为单位串行发出，其最先发送的是字节的最高位。

每个 I2C 器件都有一个器件地址，有些 I2C 器件的器件地址是固定的，而有些 I2C 器件的器件地址由一个固定部分和一个可编程的部分构成，这是因为很可能在一个系统中有几

个同样的器件，器件地址的可编程能最大数量的使这些器件连接到 I2C 总线上，例如 EEPROM 器件，为了增加系统的 EEPROM 容量，可能需要多个 EEPROM。器件可编程地址位的数量由它可使用的管脚决定，比如 EEPROM 器件一般会留下 3 个管脚用于可编程地址位。但有些 I2C 器件在出厂时器件地址就设置好了，用户不可以更改（如实时时钟 PCF8563 的器件地址为固定的 7'h51）。所以当主机想给某个器件发送数据时，只需向总线上发送接收器件的器件地址即可。

对于 AT24C64 而言，其器件地址为 1010 加 3 位的可编程地址，3 位可编程地址由器件上的 3 个管脚 A2、A1、A0 的硬件连接决定。当硬件电路上分别将这三个管脚连接到 GND 或 VCC 时，就可以设置不同的可编程地址。对于我们的开发板，这 3 个管脚连接到地。

进行数据传输时，主机首先向总线上发出开始信号，对应开始位 S，然后按照从高到低的位序发送器件地址，一般为 7bit，第 8bit 位为读写控制位 R/W，该位为 0 时表示主机对从机进行写操作，当该位为 1 时表示主机对从机进行读操作，然后接收从机响应。对于 AT24C64 来说，其传输器件地址格式如下图所示：



图 3 AT24C64 器件地址格式

发送完第一个字节（7 位器件地址和一位读写控制位）并收到从机正确的应答后就开始发送字地址（Word Address）。一般而言，每个兼容 I2C 协议的器件，内部总会有可供读写的寄存器或存储器，对于我们本次实验用到的 EEPROM 存储器，内部就是一系列顺序编址的存储单元。所以当我们对一个器件中的存储单元（包括寄存器）进行读写时，首先要指定存储单元的地址即字地址，然后再向该地址写入内容。该地址为一个或两个字节长度，具体长度由器件内部的存储单元的数量决定，当存储单元数量不超过一个字节所能表示的最大数量（ $2^8=256$ ）时，用一个字节表示，超过一个字节所能表示的最大数量时，就需要用两个字节来表示，例如同是 EEPROM 存储器，AT24C02 的存储单元容量为 2Kbit=256Byte（一般 bit 缩写为 b，Byte 缩写为 B），用一个字节地址即可寻址所有的存储单元，而 AT24C64 的存储单元容量为 64Kb=8KB，需要 13 位（ $2^{13}=8KB$ ）的地址位，而 I2C 又是以字节为单位进行传输的，所以需要两个，如下图所示：

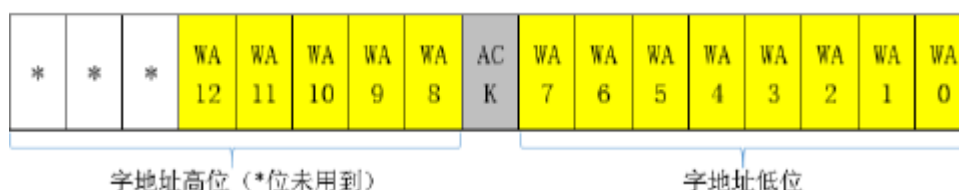


图 4 双字节地址分布

主机发送完字地址，从机正确应答后就把内部的存储单元地址指针指向该单元。如果读写控制位 R/W 位为 0，从机就处于接收数据的状态，此时主机开始写数据。写数据分为单次写（字节写）和连续写（页写），时序如下所示：

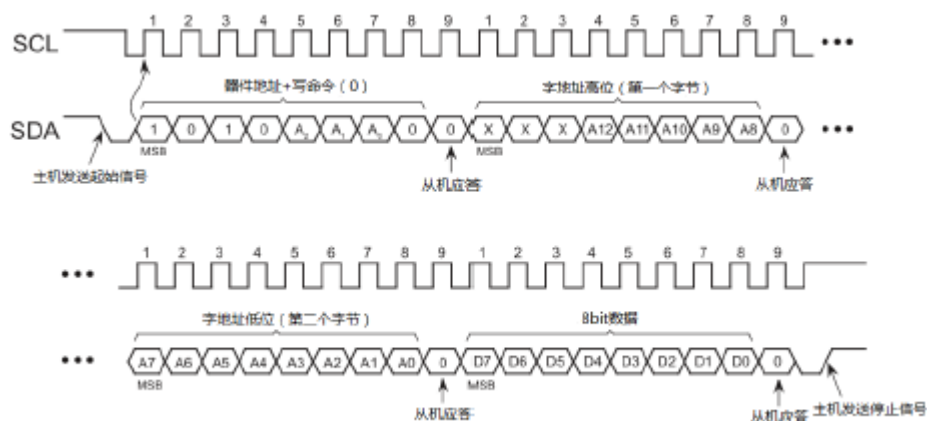


图 5 单次写（字节写）时序

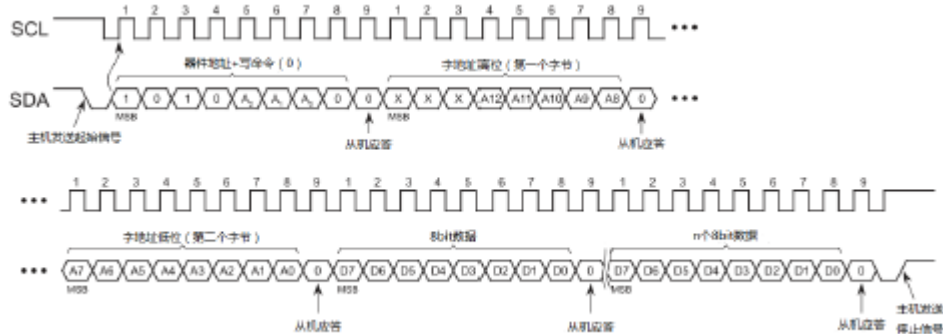


图 6 连续写（页写）时序

如果读写控制位 R/W 位为“1”即读命令，主机就处于接收数据的状态，从机从该地址单元输出数据。读数据有三种方式：当前地址读、随机读和连续读。当前地址读是指在一次读或写操作后发起读操作。由于 I2C 器件在读写操作后，其内部的地址指针自动加一，因此当前地址读可以读取下一个字地址的数据。也就是说上次读或写操作的单元地址为 02 时，当前地址读的内容就是地址 03 处的单元数据。

由于当前地址读极不方便读取任意的地址单元的数据，所以就有了随机读，由于要从机内的存储单元地址指针指向我们想要读取的存储单元地址处，所以先送了一次 Dummy Write 也就是虚写操作，之所以称为虚写，是因为我们并不是真的要写数据，而是通过这种虚写操作使地址指针指向虚写操作中字地址的位置，等从机应答后，就可以以当前地址读的方式读数据了。

至于连续读，对应的是当前地址读和随机读都是一次读取一个字节而言的，它是将当前地址读或随机读的主机非应答改成应答，表示继续读取数据。三种读操作时序如下所示：

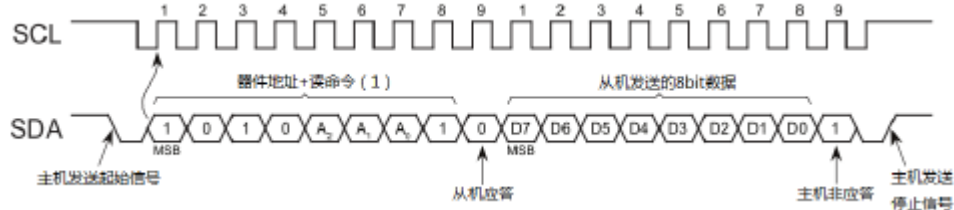


图 7 当前地址读时序

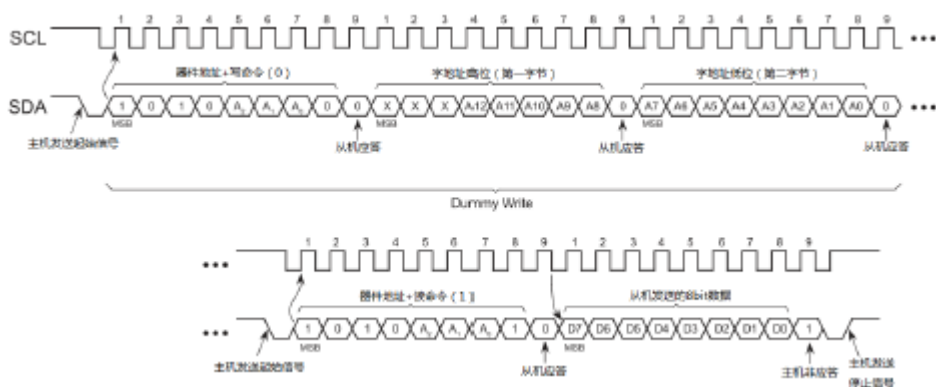


图 8 随机地址读时序

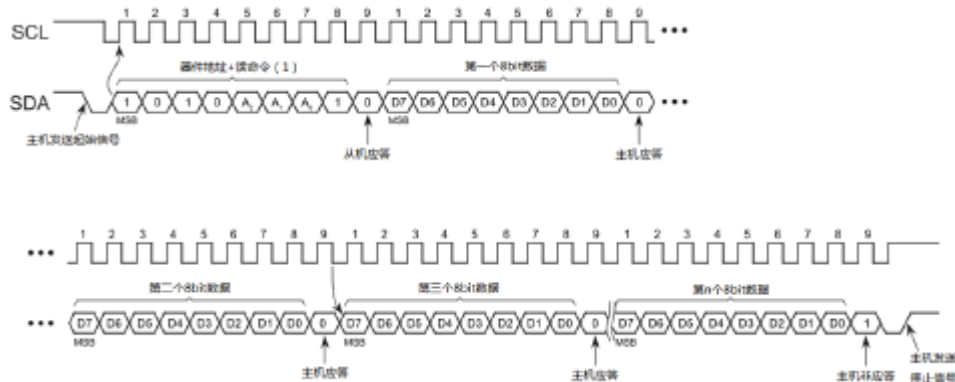


图 9 连续读时序

本实验主要采用单次写和随机读的方式进行 EEPROM 读写测试。

2.2 FPGA 实现架构

本节的实验任务是先向 EEPROM (AT24C64) 的存储器地址 0 至 255 分别写入数据 0~255; 写完之后再读取存储器地址 0~255 中的数据, 若读取的值全部正确则 LED 灯常亮, 否则 LED 灯闪烁。

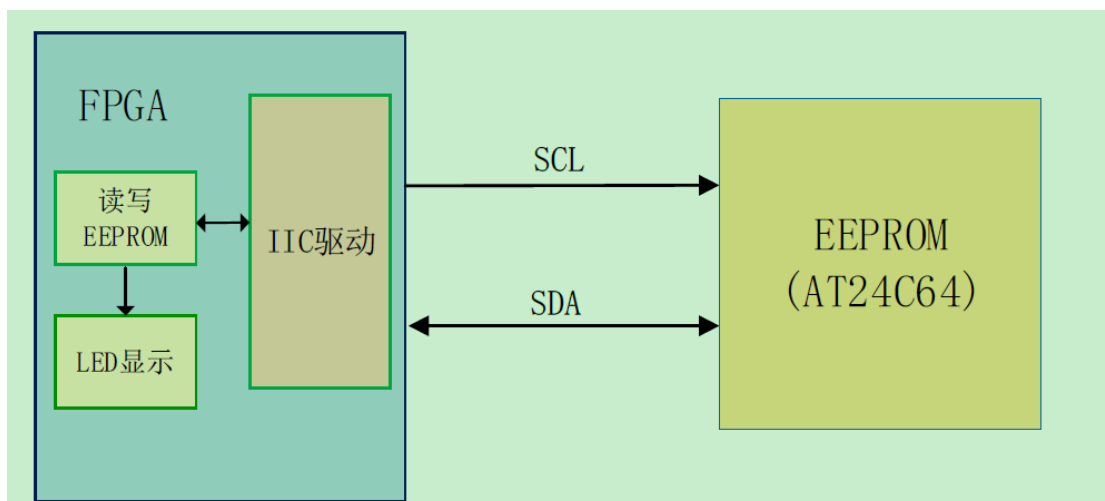


图 10 EEPROM 读写系统框图

FPGA 部分包括四个模块, 顶层模块 (e2prom_top)、读写模块 (e2prom_rw)、I2C 驱动模块 (i2c_dri) 和 LED 灯显示模块 (led_alarm)。在顶层模块中完成对其余模块的例化。

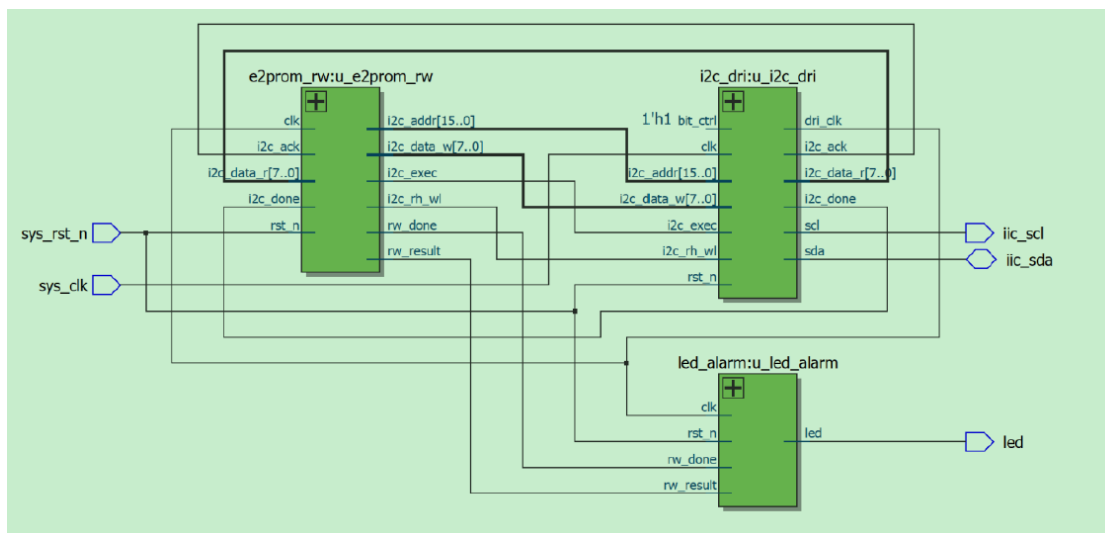


图 11 顶层模块原理图

2.3 FPGA 实现代码

2.3.1 e2promtop

```

module e2prom_top(
    input    sys_clk,
    input    sys_rst_n,
    //eeprom interface
    output    iic_scl,
    inout     iic_sda,
    //user interface
    output    led
);

//parameter define
parameter SLAVE_ADDR = 7'b1010000; //器件地址(SLAVE_ADDR)
parameter BIT_CTRL   = 1'b1; //字地址位控制参数(16b/8b)
parameter CLK_FREQ   = 26'd50_000_000; //i2c_dri 模块的驱动时钟频率
                                   (CLK_FREQ)
parameter I2C_FREQ   = 18'd250_000; //I2C 的 SCL 时钟频率
parameter L_TIME     = 17'd125_000; //led 闪烁时间参数
//wire define
wire      dri_clk; //I2C 操作时钟
wire      i2c_exec; //I2C 触发控制
wire [15:0] i2c_addr; //I2C 操作地址
wire [7:0] i2c_data_w; //I2C 写入的数据
wire      i2c_done; //I2C 操作结束标志
wire      i2c_ack; //I2C 应答标志
wire      i2c_rh_wl ; //I2C 读写控制
wire [7:0] i2c_data_r; //I2C 读出的数据
wire      rw_done; //E2PROM 读写测试完成

```



```

wire          rw_result ;    //E2PROM 读写测试结果 0:失败 1:成功
//*****
//**maincode
//*****
//e2prom 读写测试模块
e2prom_rw u_e2prom_rw(
.clk          (dri_clk),      //时钟信号
.rst_n        (sys_rst_n),    //复位信号
//i2c interfac
.i2c_exec      (i2c_exec),    //I2C 触发执行信号
.i2c_rh_wl     (i2c_rh_wl ),  //I2C 读写控制信号
.i2c_addr      (i2c_addr),    //I2C 器件内地址
.i2c_data_w    (i2c_data_w),  //I2C 要写的数据
.i2c_data_r    (i2c_data_r),  //I2C 读出的数据
.i2c_done      (i2c_done),    //I2C 一次操作完成
.i2c_ack       (i2c_ack),     //I2C 应答标志 0:应答 1:未应答
//user interfac
.rw_done       (rw_done),     //E2PROM 读写测试完成
.rw_result     (rw_result )   //E2PROM 读写测试结果 0:失败 1:成功
)

//i2c 驱动模块
i2c_dri #(
    .SLAVE_ADDR  (SLAVE_ADDR), //EEPROM 从机地址
    .CLK_FREQ    (CLK_FREQ),    //模块输入的时钟频率
    .I2C_FREQ    (I2C_FREQ)     //IIC_SCL 的时钟频率
) u_i2c_dri(
.clk          (sys_clk),
.rst_n        (sys_rst_n ),
//i2c interface
.i2c_exec      (i2c_exec),    //I2C 触发执行信号
.bit_ctrl      (BIT_CTRL),    //器件地址位控制(16b/8b)
.i2c_rh_wl     (i2c_rh_wl ),  //I2C 读写控制信号
.i2c_addr      (i2c_addr),    //I2C 器件内地址
.i2c_data_w    (i2c_data_w),  //I2C 要写的数据
.i2c_data_r    (i2c_data_r),  //I2C 读出的数据
.i2c_done      (i2c_done),    //I2C 一次操作完成
.i2c_ack       (i2c_ack),     //I2C 应答标志
.scl           (iic_scl),     //I2C 的 SCL 时钟信号
.sda           (iic_sda),     //I2C 的 SDA 信号
//user interface
.dri_clk       (dri_clk)      //I2C 操作时钟
);

```

```

    //led 指示模块
    led_alarm #(.L_TIME(L_TIME)) //控制 led 闪烁时间
)u_led_alarm(
    .clk          (dri_clk),
    .rst_n        (sys_rst_n ),

    .led          (led),
    .rw_done      (rw_done),
    .rw_result    (rw_result )
);

endmodule

```

2.3.2 e2prom_rw

```

module e2prom_rw(
    input          clk,          //时钟信号
    input          rst_n,        //复位信号

    //i2c interface
    output reg     i2c_rh_wl,    //I2C 读写控制信号
    output reg     i2c_exec,     //I2C 触发执行信号
    output reg     [15:0] i2c_addr, //I2C 器件内地址
    output reg     [7:0] i2c_data_w , //I2C 要写的的数据
    input          [7:0] i2c_data_r , //I2C 读出的数据
    input          i2c_done,     //I2C 一次操作完成
    input          i2c_ack,      //I2C 应答标志

    //user interface
    output reg     rw_done,      //E2PROM 读写测试完成
    output reg     rw_result    //E2PROM 读写测试结果 0:失败 1:成功
);

//parameter define
//EEPROM 写数据需要添加间隔时间,读数据则不需要
parameter rWR_WAIT_TIME =14'd5000;//写入间隔时间
parameter MAX_BYTE=16'd256;//读写测试的字节个数

//reg define
reg [1:0] flow_cnt;//状态流控制
reg [13:0] wait_cnt;//延时计数器

//*****
//**main code
//*****

```

```

//EEPROM 读写测试,先写后读, 并比较读出的值与写入的值是否一致
always @(posedge clk or negedge rst_n) begin
    if(!rst_n) begin
        flow_cnt <=2'b0;
        i2c_rh_wl <=1'b0;
        i2c_exec <=1'b0;
        i2c_addr <=16'b0;
        i2c_data_w <=8'b0;
        wait_cnt <=14'b0;
        rw_done <=1'b0;
        rw_result <=1'b0;
    end
    else begin
        i2c_exec <=1'b0;
        rw_done <=1'b0;
        case(flow_cnt)
            2'd0: begin
                wait_cnt <=wait_cnt +1'b1;//延时计数
                if(wait_cnt ==WR_WAIT_TIME -1'b1)
begin//EEPROM 写操作延时完成
                    wait_cnt <=1'b0;
                    if(i2c_addr ==MAX_BYTE) begin//256 个字
节写入完成
                        i2c_addr <=1'b0;
                        i2c_rh_wl <=1'b1;
                        flow_cnt <=2'd2;
                    end
                    else begin
                        flow_cnt <=flow_cnt +1'b1;
                        i2c_exec <=1'b1;
                    end
                end
            end
            2'd1: begin
                if(i2c_done ==1'b1) begin//EEPROM 单次写入完成
flow_cnt <=2'd0;
                    i2c_addr <=i2c_addr +1'b1;//地址 0~255
分别写入.
                    i2c_data_w <=i2c_data_w +1'b1;//.数据
0~255
                end
            end
            2'd2: begin

```

```

        flow_cnt <=flow_cnt +1'b1;
        i2c_exec <=1'b1;
    end
    2'd3: begin
        if(i2c_done ==1'b1) begin//EEPROM 单次读出完成
            //读出的值错误或者 I2C 未应答,读写测试失败
            if((i2c_addr[7:0]!=i2c_data_r)|| (i2c_a
ck ==1'b1)) begin
                rw_done <=1'b1;
                rw_result <=1'b0;
            end
            else if(i2c_addr ==MAX_BYTE -1'b1)
begin//读写测试成功
                rw_done <=1'b1;
                rw_result <=1'b1;
            end
            else begin
                flow_cnt <=2'd2;
                i2c_addr <=i2c_addr +1'b1;
            end
        end
    end
end
    end
    default: ;
endcase
end
endmodule

```

2.3.3 i2c_dri

在程序中我们采用三段式状态机。由于代码较长，我们在这里将其中第二段的源代码粘贴如下：

```

//组合逻辑判断状态转移条件
always @(*) begin
    next_state =st_idle;
    case(cur_state)
        t_idle: begin //空闲状态
            if(i2c_exec)
                next_state =st_sladdr;
            else
                next_state =st_idle;
        end
        st_sladdr: begin
            if(st_done) begin
                if(bit_ctrl)//判断是 16 位还是 8 位字地址
                    next_state =st_addr16;
            end
        end
    endcase
end

```

```

        else
            next_state =st_addr8 ;
        end
    else
        next_state =st_sladdr;
    end
st_addr16: begin    //写 16 位字地址
    if(st_done)
        next_state =st_addr8;
    else
        next_state =st_addr16;
    end
st_addr8: begin    //8 位字地址
    if(st_done) begin
        if(wr_flag==1'b0)    //读写判断
            next_state =st_data_wr;
        else
            next_state =st_addr_rd;
        end
    else begin
        next_state =st_addr8;
    end
end
st_data_wr: begin    //写数据(8 bit)
    if(st_done)
        next_state =st_stop;
    else
        next_state =st_data_wr;
    end
st_addr_rd: begin    //写地址以进行读数据
    if(st_done) begin
        next_state =st_data_rd;
    end
    else begin
        next_state =st_addr_rd;
    end
end
st_data_rd: begin    //读取数据(8 bit)
    if(st_done)
        next_state = st_stop;
    else
        next_state = st_data_rd;
    end
st_stop: begin    //结束 I2C 操作

```

```

        if(st_done)
            next_state = st_idle;
        else
            next_state = st_stop ;
        end
        default: next_state = st_idle;
    endcase
end

```

2.2.4 led_alarm

```

module led_alarm
    #(parameter L_TIME = 25'd25_000_000
    )
    (
        input clk, //时钟信号
        input rst_n, //复位信号

        input rw_done, //错误标志
        input rw_result, //E2PROM 读写测试完成
        output reg led //E2PROM 读写测试结果 0:失败 1:成功
    );

    //reg define
    reg rw_done_flag; //读写测试完成标志
    reg [24:0] led_cnt; //led 计数

    /*******
    /**main code
    /*******

    //读写测试完成标志
    always @(posedge clk or negedge rst_n) begin
        if(!rst_n)
            rw_done_flag <= 1'b0;
        else if(rw_done)
            rw_done_flag <= 1'b1;
    end

    //错误标志为 1 时 PL_LED0 闪烁, 否则 PL_LED0 常亮
    always @(posedge clk or negedge rst_n) begin
        if(!rst_n) begin
            ed_cnt <= 25'd0;
            ed <= 1'b0;
        end
        else begin

```

```

        if(rw_done_flag) begin
            if(rw_result)//读写测试正确
                led <=1'b1;//led 灯常亮
            else begin//读写测试错误
                led_cnt <=led_cnt +25'd1;
                if(led_cnt ==L_TIME -1'b1) begin
                    led_cnt <=25'd0;
                    led <=~led;//led 灯闪烁
                end
            end
        end
    end
else
    led <=1'b0;//读写测试完成之前,led 灯熄灭
end
end
endmodule

```

2.4 FPGA 硬件调试

程序中定义了两个参数，WR_WAIT_TIME（写入间隔时间）和 MAX_BYTE（读写测试的字节个数）。AT24C64 官方手册对写入数据后，数据写入完成的时间最大不超过 10ms，所以为了保证数据能够正确写入，单次写入数据操作完成后，最好延时 10ms 的时间。本次实验为了节省数据写入的时间，WR_WAIT_TIME 的值设置为 5000，即 5ms（输入时钟的周期为 1us，1us*5000=5ms），实测延时 5ms 也可以正确写入。这里不建议大家将写入的间隔设置的过于短，否则会导致数据写入失败。另外，EEPROM 只有对写操作有时间间隔要求，对读操作没有间隔要求，因此读写测试模块仅对写操作增加时间间隔。

程序先对 I2C 驱动模块发起写操作，即拉高 i2c_exec，拉低 i2c_rh_wl（低电平表示写），然后分别向 EEPROM 的地址 0 至地址 255 写入数据 0 至 255，并且在每次写操作之间增加 5ms 的延时。数据全部写完后，发起读操作，即拉高 i2c_exec，拉高 i2c_rh_wl（高电平表示读），然后分别从 EEPROM 的地址 0 至地址 255 读出数据，并判断读出的值与写入的值是否一致，如果数据一致并且每次操作 IIC 都有应答信号产生（i2c_ack），EEPROM 的读写测试才正确，否则读写测试失败。

读写测试完成后，输出 rw_done 信号和 rw_result 信号，rw_done 为 EEPROM 读写测试完成信号，rw_result 为读写测试的结果，0 表示读写失败，1 表示读写正确。

EEPROM 写操作的 SignalTap 波形图如下图所示：

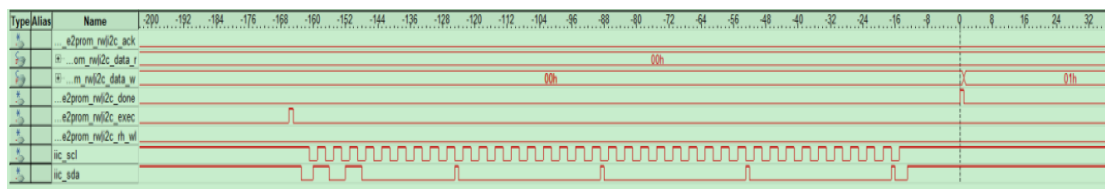


图 12 EEPROM 写操作 ILA 波形图

从该波形图中我们看到读写控制信号 i2c_rh_wl 为低电平，表示处于写操作状态。当 I2C 触发执行信号 i2c_exec 为高电平时开始执行 I2C 写操作。在 IIC 操作结束后，拉高 i2c_done 信号。另外，在 IIC 写操作期间，i2c_ack（IIC 应答标志）一直处于低电平，说明

EEPROM 响应了主机并应答

EEPROM 读操作的 SignalTap 波形图如下图所示:

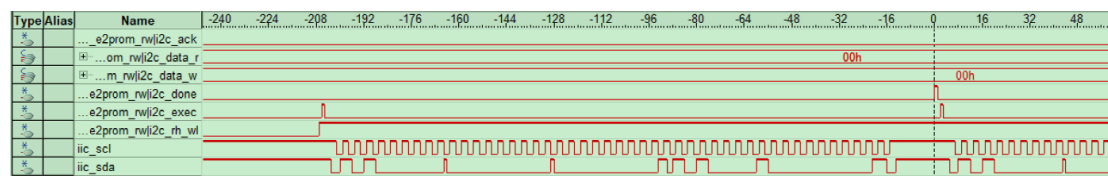


图 13 EEPROM 读操作 ILA 波形图

从该波形图中我们看到读写控制信号 `i2c_rh_wl` 为高电平，表示处于读操作状态。当 I2C 触发执行信号 `i2c_exec` 为高电平时开始执行 I2C 读操作。在 IIC 操作结束后，拉高 `i2c_done` 信号。另外，在 IIC 读操作期间，`i2c_ack` (IIC 应答标志) 一直处于低电平，说明 EEPROM 响应了主机并应答。

`led` 显示模块利用 LED 灯的显示状态来标识读写过程是否出错。程序中寄存输入的 `rw_done` 信号，采集到 `rw_done` 信号之后，拉高 `rw_done_flag` 信号。

根据 `rw_done_flag` 和 `rw_result` 信号控制 LED 灯的状态。在 EEPROM 读写测试完成之前，LED 灯处于熄灭状态；如果 EEPROM 读写测试成功，LED 灯处于常亮状态；如果 EEPROM 读写测试失败，LED 灯会不停的闪烁。

2.5 参考文献

正点原子开拓者之 FPGA 开发指南_V2.1

3. 基于 FPGA 的 M 序列发生器

3.1 算法原理

m 序列是最长线性移位寄存器序列的简称。顾名思义，m 序列是由多级移位寄存器或其延迟元件通过线性反馈产生的最长的码序列。在二进制移位寄存器中，若 n 为移位寄存器的级数， n 级移位寄存器共有 2^n 个状态，除去全 0 状态外还剩下 2^n-1 中状态，因此它能产生的最大长度的码序列为 2^n-1 位，也就是说，一个 n 级线性反馈移位寄存器产生的最长周期等于 2^n-1 。

下图是由 n 级移位寄存器构成的 m 序列发生器。寄存器的状态决定于时钟控制下输入的信息（“0”或“1”），例如第 i 级移位寄存器状态决定于前一时钟脉冲后的第 $i-1$ 级移位寄存器的状态。图中 $C_0, C_1, \dots, C_n$ 均为反馈线，其中 $C_0=C_n=1$ ，表示反馈连接。因为 m 序列是由循环序列发生器产生的，因此 C_0 和 C_n 肯定为 1，即参与反馈。而反馈系数 $C_1, C_2, \dots, C_{n-1}$ ，若为 1，参与反馈；若为 0，则表示断开反馈线，即开路，无反馈连线。

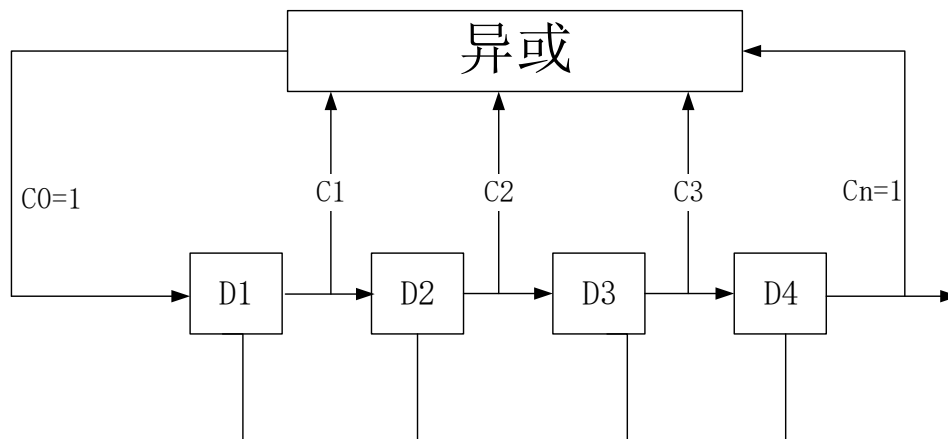


图 1 M 序列发生器

一个线性反馈移动寄存器能否产生 m 序列，决定于它的反馈系数 $C_i (i=0,1,2,\dots,n)$ ，下表中列出了部分 m 序列的反馈系数，按照下表中的系数来构造移位寄存器，就能产生相应的 m 序列。

级数 n	周期 P	反馈系数 C_i (采用八进制)
3	7	13
4	15	23
5	31	45, 67, 75
6	63	103, 147, 155
7	127	203, 211, 217, 235, 277, 313, 325, 345, 367
8	255	435, 453, 537, 543, 545, 551, 703, 747

根据上表中的八进制的反馈系数，可以确定 m 序列发生器的结构。以 7 级 m 序列反馈系数 $C_i=(211)_8$ 为例，首先将八进制的系数转化为二进制的系数即 $C_i=(010001001)_2$ ，由此我们可以得到各级反馈系数分别为： $C_0=1, C_1=0, C_2=0, C_3=0, C_4=1, C_5=0, C_6=0, C_7=1$ ，由此就很容易地构造出相应的 m 序列发生器。根据反馈系数，其他级数的 m 序列的构造原理与上述方法相同。

3.2 Matlab 验证代码

```
function [seq]=mseq(coef)
```

```

%*****
% 此函数用来生成 m 序列
% coef 为反馈系数向量
% Author: FastestSnail
% Date: 2017-10-03
%*****
m=length(coef);
len=2^m-1; % 得到最终生成的 m 序列的长度
backQ=0; % 对应寄存器运算后的值，放在第一个寄存器
seq=zeros(1,len); % 给生成的 m 序列预分配
registers = [1 zeros(1, m-2) 1]; % 给寄存器分配初始结果
for i=1:len
    seq(i)=registers(m);
    backQ = mod(sum(coef.*registers), 2); %特定寄存器的值进行异或运算，即相
加后模 2
    registers(2:length(registers)) = registers(1:length(registers)-1); % 移
位
    registers(1)=backQ; % 把异或的值放在第一个寄存器的位置
end
end
调用方法：
mseq([0 0 1 1])
结果：

```

```

>> mseq([0 0 1 1])

ans =

Columns 1 through 11
    1     0     0     1     1     0     1     0     1     1     1

Columns 12 through 15
    1     0     0     0

```

fx >> | <http://blog.csdn.net/cjbct>

图 2 运行结果

3.3 FPGA 实现架构

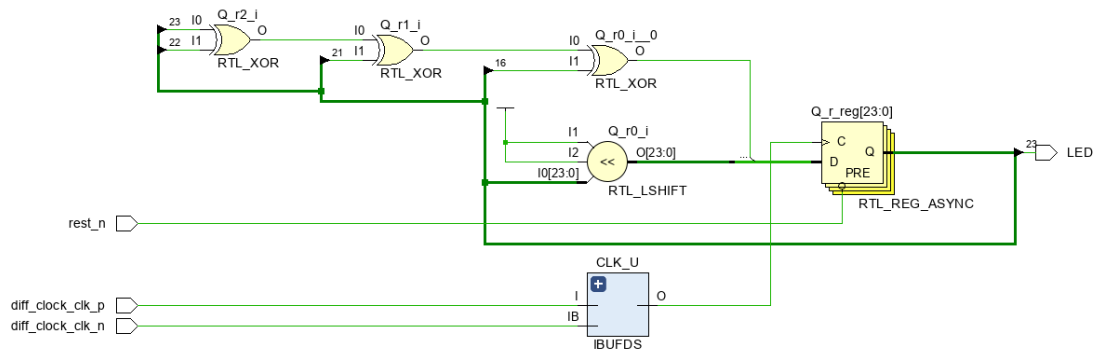


图 3 FPGA 实现架构

3.4 FPGA 实现代码

```

module m_serial#(
    parameter    len = 24
)
(
    diff_clock_clk_p,
    diff_clock_clk_n,
    rest_n,
    LED
);

input diff_clock_clk_p    ;
input diff_clock_clk_n    ;
input rest_n              ;
output LED                ;

wire Q;

IBUFGDS CLK_U
(
    .I(diff_clock_clk_p),
    .IB(diff_clock_clk_n),
    .O(clk) );

reg[(len-1):0] Q_r;

assign Q = Q_r[(len-1)];
assign LED = Q;
always @(posedge clk or negedge rest_n) begin

```

```

if (rest_n == 0) begin // negedge reset
    Q_r <= ~(0);
end

else begin
    Q_r <= Q_r<<1; // shift reg
    case(len)
        2 : Q_r[0] <= Q_r[1]^Q_r[0];
        3 : Q_r[0] <= Q_r[2]^Q_r[1];
        4 : Q_r[0] <= Q_r[3]^Q_r[2];
        5 : Q_r[0] <= Q_r[4]^Q_r[2];
        6 : Q_r[0] <= Q_r[5]^Q_r[4];
        7 : Q_r[0] <= Q_r[6]^Q_r[3];
        8 : Q_r[0] <= Q_r[7]^Q_r[5]^Q_r[4]^Q_r[3];
        9 : Q_r[0] <= Q_r[8]^Q_r[4];
        10: Q_r[0] <= Q_r[9]^Q_r[6];
        11: Q_r[0] <= Q_r[10]^Q_r[8];
        12: Q_r[0] <= Q_r[11]^Q_r[10]^Q_r[7]^Q_r[5];
        13: Q_r[0] <= Q_r[12]^Q_r[11]^Q_r[9]^Q_r[8];
        14: Q_r[0] <= Q_r[13]^Q_r[12]^Q_r[7]^Q_r[3];
        15: Q_r[0] <= Q_r[14]^Q_r[13];
        16: Q_r[0] <= Q_r[15]^Q_r[14]^Q_r[12]^Q_r[3];
        17: Q_r[0] <= Q_r[16]^Q_r[13];
        18: Q_r[0] <= Q_r[17]^Q_r[10];
        19: Q_r[0] <= Q_r[18]^Q_r[17]^Q_r[16]^Q_r[13];
        20: Q_r[0] <= Q_r[19]^Q_r[16];
        21: Q_r[0] <= Q_r[20]^Q_r[18];
        22: Q_r[0] <= Q_r[21]^Q_r[20];
        23: Q_r[0] <= Q_r[22]^Q_r[17];
        24: Q_r[0] <= Q_r[23]^Q_r[22]^Q_r[21]^Q_r[16];
        default: Q_r[0] <= Q_r[0];
    endcase
end
end
endmodule

```

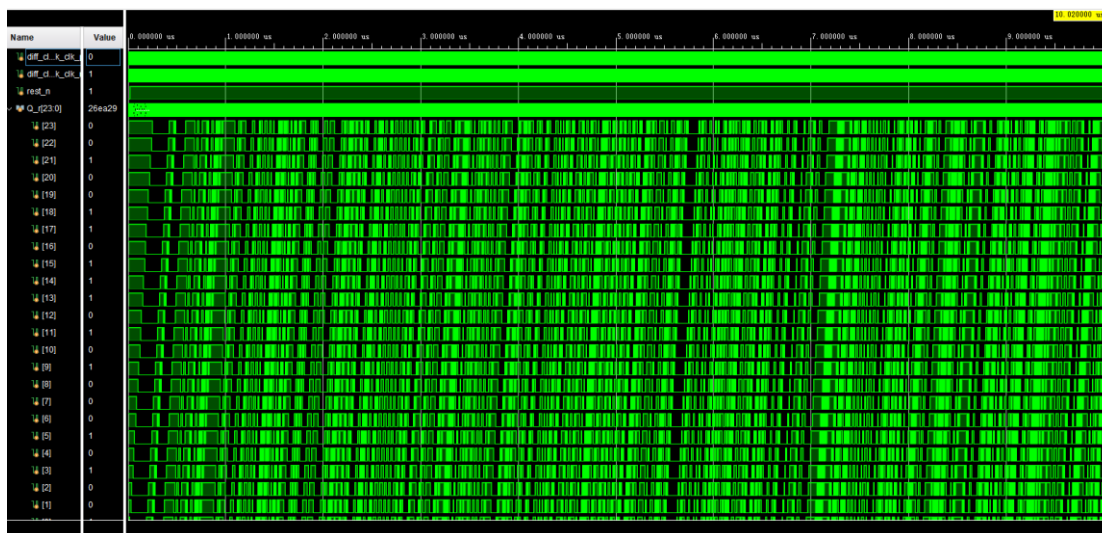


图 4 仿真截图

3.5 FPGA 硬件调试



图 5 硬件连接图

基于行为的设计方法：由相应个数的 D 触发器首位连接而成，移位寄存器所有位的值右移一位，最右边的寄存器移出的值作为输出，最左边的寄存器输入的值由反馈值填充，此过程称为进动一拍。进动 m 拍就产生 m 个输出。

3.6 参考文献

https://blog.csdn.net/weixin_45015947/article/details/89891757

4. 基于 FPGA 的 FFT 实现

4.1 算法原理

快速傅里叶变换 (fast Fourier transform),即利用计算机计算离散傅里叶变换 (DFT)的高效、快速计算方法的统称,简称 FFT。快速傅里叶变换是 1965 年由 J.W.库利和 T.W.图基提出的。采用这种算法能使计算机计算离散傅里叶变换所需要的乘法次数大为减少,特别是被变换的抽样点数 N 越多,FFT 算法计算量的节省就越显著。

FFT (Fast Fourier Transformation) 是离散傅氏变换 (DFT) 的快速算法。即为快速傅氏变换。它是根据离散傅氏变换的奇、偶、虚、实等特性,对离散傅立叶变换的算法进行改进获得的。

$$f(x) = a_0 + \sum_{n=1}^{\infty} \left(a_n \cos \frac{n\pi x}{L} + b_n \sin \frac{n\pi x}{L} \right)$$

FFT 的基本思想是把原始的 N 点序列,依次分解成一系列的短序列。充分利用 DFT 计算中指数因子 所具有的对称性质和周期性质,进而求出这些短序列相应的 DFT 并进行适当组合,达到删除重复计算,减少乘法运算和简化结构的目的。此后,在这思想基础上又开发了高基和分裂基等快速算法,随着数字技术的高速发展,1976 年出现建立在数论和多项式理论基础上的维诺格勒傅里叶变换算法(WFTA)和素因子傅里叶变换算法。它们的共同特点是,当 N 是素数时,可以将 DFT 算转化为求循环卷积,从而更进一步减少乘法次数,提高速度。

4.2 FPGA 实现架构

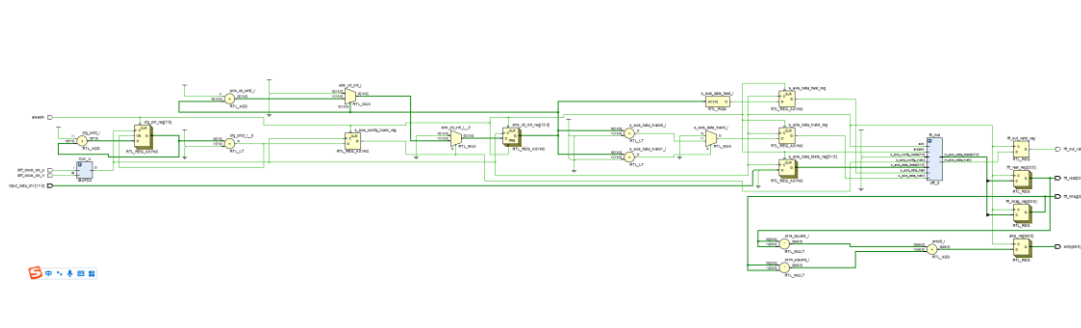


图 1 FPGA 实现架构

4.3 FPGA 实现代码

```
module Xilinx_FFT_Guide_liuqi(
    input aclk,
    input aresetn,

    input [11:0]    input_data_ch1,
    output [23:0]   fft_real,
    output [23:0]   fft_imag,
    output reg [46:0] amp,
    output fft_out_valid
);
```

```

);

reg [11:0] input_data_ch1_reg;
wire [7:0] s_axis_config_tdata;
reg s_axis_config_tvalid;

wire s_axis_data_tready;
reg [31:0] s_axis_data_tdata;
reg s_axis_data_tvalid;
reg s_axis_data_tlast;

wire [47:0] m_axis_data_tdata;
wire [15:0] m_axis_data_tuser;
wire m_axis_data_tvalid;
wire m_axis_data_tlast;
reg [7:0]    cfg_cnt;
reg [13:0]  sink_ctl_cnt;
reg [23:0]  fft_real,fft_imag;
reg fft_out_valid;
wire    event_frame_started;
wire    event_tlast_unexpected;
wire    event_tlast_missing;
wire    event_status_channel_halt;
wire    event_data_in_channel_halt;
wire    event_data_out_channel_halt;

xfft_0 usr_FFT(

    .aclk(aclk),
    .aresetn(aresetn),

    .s_axis_config_tdata(s_axis_config_tdata),
    .s_axis_config_tvalid(s_axis_config_tvalid),
    .s_axis_config_tready(),

    .s_axis_data_tdata(s_axis_data_tdata),
    .s_axis_data_tvalid(s_axis_data_tvalid),
    .s_axis_data_tready(),
    .s_axis_data_tlast(s_axis_data_tlast),

    .m_axis_data_tdata(m_axis_data_tdata),
    .m_axis_data_tuser(m_axis_data_tuser),
    .m_axis_data_tvalid(m_axis_data_tvalid),
    .m_axis_data_tready(1'b1),
    .m_axis_data_tlast(m_axis_data_tlast),

```

```
.event_frame_started(event_frame_started),
.event_tlast_unexpected(event_tlast_unexpected),
.event_tlast_missing(event_tlast_missing),
.event_status_channel_halt(event_status_channel_halt),
.event_data_in_channel_halt(event_data_in_channel_halt),
.event_data_out_channel_halt(event_data_out_channel_halt)
);

//fft core config
always@(posedge aclk or negedge aresetn)
begin
    if(!aresetn)
        cfg_cnt <= 8'd0;
    else
        begin
            if(cfg_cnt < 8'd200)
                cfg_cnt <= cfg_cnt + 1'b1;
            else
                cfg_cnt <= cfg_cnt;
        end
end

always@(posedge aclk or negedge aresetn)
begin
    if(!aresetn)
        s_axis_config_tvalid <= 1'b0;
    else
        begin
            if(cfg_cnt < 8'd200)
                s_axis_config_tvalid <= 1'b1;
            else
                s_axis_config_tvalid <= 1'b0;
        end
end

assign s_axis_config_tdata = 8'd1;

//fft sink_in ctl/
always@(posedge aclk or negedge aresetn)
begin
    if(!aresetn)
        s_axis_data_tdata <= 32'b0;
    else
        s_axis_data_tdata <= {20'd0,input_data_ch1};
end
```



```

end

always@(posedge aclk or negedge aresetn)
begin
    if(!aresetn)
        sink_ctl_cnt <= 14'd8194;
    else if(s_axis_config_tvalid)
        sink_ctl_cnt <= 14'd0;
    else if(sink_ctl_cnt == 14'd8192)
        sink_ctl_cnt <= 14'd1;
    else
        sink_ctl_cnt <= sink_ctl_cnt + 1'b1;
end

//s_axis_data_tvalid
always@(posedge aclk or negedge aresetn)
begin
    if(!aresetn)
        s_axis_data_tvalid <= 1'b0;
    else if(sink_ctl_cnt < 14'd1)
        s_axis_data_tvalid <= 1'b0;
    else if(sink_ctl_cnt < 14'd2049)
        s_axis_data_tvalid <= 1'b1;
    else
        s_axis_data_tvalid <= 1'b0;
end

//s_axis_data_tlast
always@(posedge aclk or negedge aresetn)
begin
    if(!aresetn)
        s_axis_data_tlast <= 1'b0;
    else
        begin
            if(sink_ctl_cnt == 14'd2048)
                s_axis_data_tlast <= 1'b1;
            else
                s_axis_data_tlast <= 1'b0;
        end
    end
end

/fft sink_in ctl/
always@(posedge aclk)
begin
    fft_real <= m_axis_data_tdata[23:0];

```

```

end

always@(posedge aclk)
begin
    fft_imag <= m_axis_data_tdata[47:24];
end

always@(posedge aclk)
begin
    fft_out_valid <= m_axis_data_tvalid;
end

/***** 计算频谱的幅值信号 *****/

wire signed [45:0] xkre_square, xkim_square;
assign xkre_square = fft_real * fft_real;
assign xkim_square = fft_imag * fft_imag;

always @(posedge aclk)
    amp <= xkre_square + xkim_square;

endmodule

```

4.4 调试仿真

对 2MHz 信号的 FFT 分析。根据 Latency 中的估计，计算时间需要 145 μ s，因此仿真需要运行到大约 150 μ s 以上。结果如下：

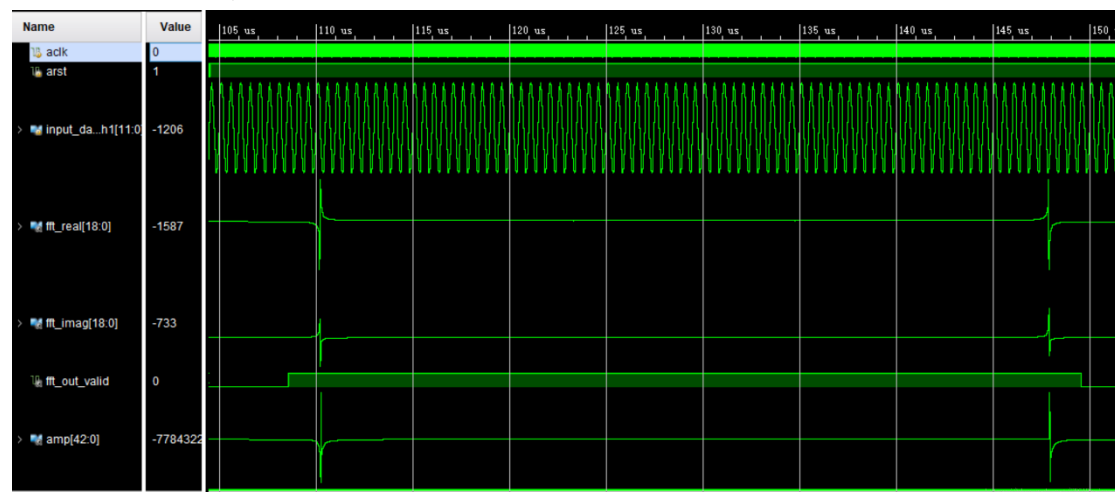


图 2 仿真截图

整个频谱的持续时间恰好是 out_valid 信号保持高电平的时间，很明显可以看到 2MHz 信号在频谱中对应的一个频率点。

4.5 参考文献

<https://blog.csdn.net/FPGADesigner/article/details/80694673>

5. 基于 FPGA 的阵列图像采集控制系统设计

5.1 算法原理

针对 1×3 相机阵列图像采集控制与处理技术开展研究工作。系统架构如下图，总体上由上位机、相机采集单元阵列以及相机阵列 3 部分组成。上位机基于 QT 平台，采用 C++ 进行实现，负责下发相机控制指令，同时负责接收、显示数据汇集单元上报的图像数据和反馈信息；相机采集单元实时检测上层控制信号，实现对相机阵列的采集控制及数据上报。

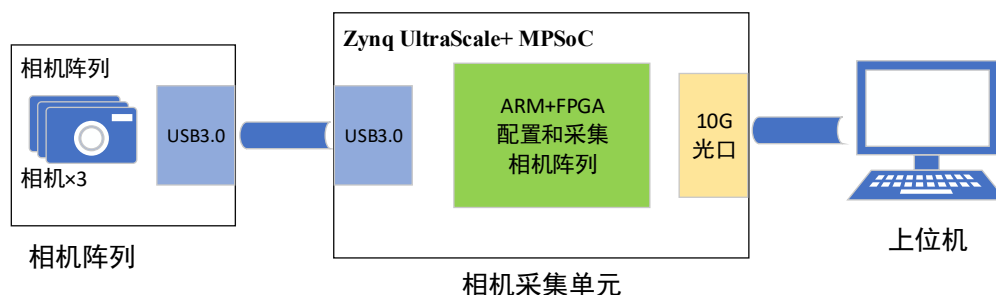


图 1 基于 FPGA 的超高速阵列式相机电控系统架构

5.1.1 基于 UDP 的相机采集控制协议

为对相机阵列进行远程定位，实现针对单个相机的精确控制与配置，同时实现采集单元向上位机的高效率图像数据上报，本文基于 UDP 链路制定相机采集控制协议。此协议详细定义了上位机与下游相机采集单元之间的通信流程和细节，包括 UDP 数据包格式、链路建立流程、配置参数下发机制和相机数据上报机制四个部分。

为明确相机控制细节，首先定义 UDP 数据包格式。UDP 数据包分为 UDP 下发数据包和 UDP 上报数据包。UDP 下发数据包由上位机发送，由相机采集单元接收，用于链路建立确认和相机参数配置。UDP 上报数据包由相机采集单元发送，由上位机接收，用于上传相机图像数据。

(1) UDP 下发数据包

表 5-1 详细定义了 UDP 下发数据包格式。下发数据包主要有数据包类型 (type)、相机编号 (Cam_num)、像素水平移动距离 (OffsetX)、像素垂直移动距离 (OffsetY)、曝光时长 (Exp_Time)、相机帧率 (FPS)、相机开窗宽度 (Width)、相机开窗高度 (Height)、水平像素 binning (Bin_H)、垂直像素 binning (Bin_V)、保留字段 (Reserved) 共计 11 个字段构成，最小包长度为 18 字节。

UDP 下发分为配置相机、采样控制、请求 UDP 包重发和 UDP 链路连接确认四个功能，由 type 字段定义。当 type 为 0x09 时启动相机采样，当 type 为 0x0a 时停止相机采样，当 type 为 0x0b 时用于请求 UDP 包重发，当 type 为 0xff 时用于 UDP 链路连接确认，其余为相机参数配置。注意当 type 字段为 0x0b 时，上位机请求 FPGA 重发 UDP 数据包 (UDP 数据包 18 字节)，此时 UDP 下发数据包格式需要更改，更改后包格式如表 5-2 所示。

表 5-1 UDP 下发数据包 UDP 字段定义

63 : 56	55 : 48	47 : 40	39 : 32	31 : 24	23 : 16	15 : 8	7 : 0
Exp_Time_H	Exp_Time_L	OffsetY_H	OffsetY_L	OffsetX_H	OffsetX_L	Cam_num	type
Reserved	Reserved	Reserved	Bin_V	Bin_H	Height	Width	Fps
—	—	—	—	—	—	Reserved	Reserved

表 5-2 重发包请求 UDP 字段定义 (18 字节)

63 : 56	55 : 48	47 : 40	39 : 32	31 : 24	23 : 16	15 : 8	7 : 0
fram	fram	fram	fram	0	Block	Cam_num	type
0	0	0	0	0	0	0	0
—	—	—	—	—	—	0	0

表 5-3 UDP 下发数据包 type 字段含义

数据包类型 (type)	含义
0x00	所有字段重配置
0x01	Fps 字段重配置, 其余字段保留原值
0x02	Width 字段重配置, 其余字段保留原值
0x03	Height 字段重配置, 其余字段保留原值
0x04	Bin_H 字段重配置, 其余字段保留原值
0x05	Bin_V 字段重配置, 其余字段保留原值
0x06	Exp_Time_L 和 Exp_Time_H 字段重配置, 其余字段保留原值
0x07	OffsetX_L 和 OffsetX_H 字段重配置, 其余字段保留原值
0x08	OffsetY_L 和 OffsetY_H 字段重配置, 其余字段保留原值
0x09	启动采样
0x0a	停止采样
0x0b	UDP 包重发
0xff	链路确认数据包, 其余字段填充 0

表 5-4 UDP 下发数据包 Cam_num 字段含义

相机编号 (Cam_num)	含义
0x00	指定 0 号相机
0x01	指定 1 号相机
0x02	指定 2 号相机

表 5-5 UDP 下发数据包 OffsetX 字段含义

像素水平移动距离 (OffsetX)	含义
OffsetX_L	像素水平移动距离, 低 8 位
OffsetX_H	像素水平移动距离, 高 8 位

表 5-6 UDP 下发数据包 OffsetY 字段含义

像素垂直移动距离 (OffsetY)	含义
OffsetY_L	像素垂直移动距离, 低 8 位
OffsetY_H	像素垂直移动距离, 高 8 位

表 5-7 UDP 下发数据包 Exp_Time 字段含义

曝光时长 (Exp_Time)	含义
Exp_Time_L	曝光时长, 低 8 位

Exp_Time_H	曝光时长, 高 8 位
------------	-------------

表 5-8 UDP 下发数据包 Fps 字段含义

相机帧率 (Fps)	含义
0x00	200fps
0x01	500fps
0x02	800fps
0x03	1000fps

表 5-9 UDP 下发数据包 Width、Height、Bin_H、Bin_V、Reserved 字段含义

字段	含义
Width	相机开窗大小, 固定为 512
Height	相机开窗大小, 固定为 512
Bin_H	水平像素 binning, 固定为 2
Bin_V	垂直像素 binning, 固定为 2
Reserved	保留字段, 填充 0

(2) UDP 上报数据包

UDP 上报数据包 UDP 字段定义如表 5-10。UDP 上报数据包由数据包类型 (type)、相机编号 (Cam_num)、包编号 (Block)、帧号 (fram)、图像数据 (Data0-Data2047) 五个字段组成。其中 type 固定为 0xaa, 用于上位机过滤 UDP 包。Block 代表包编号, 单个相机单帧图像 (灰度图) 共 256×256 字节, 分为 32 个连续的 Block 发送给上位机, 每个 Block 包含 2048 字节图像数据。fram 代表上位机当前接收图像的帧号, 帧号占 4 字节, 循环计数。

表 5-10 UDP 上报数据包 UDP 字段定义

63 : 56	55 : 48	47 : 40	39 : 32	31 : 24	23 : 16	15 : 8	7 : 0
0	fram	fram	fram	fram	Block	Cam_num	type
Data7	Data6	Data5	Data4	Data3	Data2	Data1	Data0
...	...	...	...	...	...	...	...
Data2047	Data2046	Data2045	Data2044	Data2043	Data2042	Data2041	Data2040

表 5-11 UDP 上报数据包各字段含义

字段	含义
type	数据包类型, 固定为 0xaa
Cam_num	相机开窗大小, 相机编号, 0 - 2
Block	包编号, 0 - 31
fram	帧号, 4 字节, 从 0 计数到 2 的 32 次幂减 1, 循环
Data0-Data2047	图像数据

5.1.2 UDP 链路建立流程

UDP 链路建立由上位机发起, 上位机经由算法处理板广播 type 字段为 0xff 的 UDP 链路确认包。相机采集单元 (任意一个) 接收到 type 字段为 0xff 的数据包后, 向上位机发送一个 type 字段为 0xff 的应答包, 用于链路连接确认, 应答包中包含自检信息。上位机接收应答包并确认其中的自检信息无误后, 标记此链路建立成功。由于系统中存在多个相机采集单元, 上位机需要统计应答包个数和包来源, 直到接收到所有采集单元的响应, 才能确认链路建立完成。

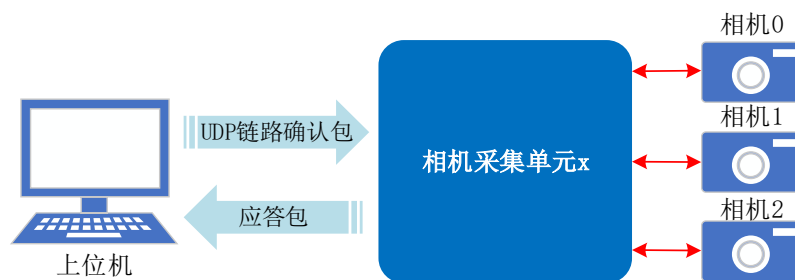


图 2 UDP 链路建立流程图

UDP 链路建立后，上位机开始接收相机数据。如果某个采集单元意外掉线，上位机需要及时捕获掉线状态。因此，在相机数据接收阶段，如果上位机在超时 10ms 后未能收到 UDP 数据包，表明相机掉线。此时，上位机需要跳出 UDP 数据接收，需要重新进行 UDP 链路建立流程。

5.1.3 相机配置

上位机支持对 3 个相机分别进行配置，使用配置表中的 Cam_num 进行区分。Width、Height、Bin_H 和 Bin_V 以固定值配置。Fps 有四档配置值 0x00、0x01、0x02 和 0x03，分别代表 200fps、500fps、800fps 和 1000fps。曝光时长由 Exp_Time_H 和 Exp_Time_L 配置，范围为 0–6000us。像素水平移动由 OffsetX_H 和 OffsetX_L 配置，范围为 0-384，像素垂直移动由 OffsetY_H 和 OffsetY_L 配置，范围为 0-256。

5.1.4 相机数据上报

上位机支持 3 个相机分别进行上报。使用 Cam_num 进行区分。相机上报时，一帧图像（固定为 256×256 大小的灰度图）分为 32 个 Block 进行上报，每个 Block 共有 2048 字节图像数据。上位机需要验证一帧图像的完整性，如果该帧图像缺少 Block 需要指示错误。同时，上报数据包中还含有当前图像的帧号，上位机需要验证帧号的连续性，即不能缺帧，如果缺帧需要指示错误。

5.2 FPGA 实现架构

5.2.1 相机采集单元架构概述

相机采集单元主要负责前端高速相机数据采集和转发的功能，分为 PS 侧（ARM 硬核）和 PL 侧（FPGA 可编程逻辑），具体结构如下图所示。图中绿色部分代表 PS，运行 Linux 操作系统，并且部署相机驱动和 DMA 驱动用于相机的采集和转发；黄色部分代表 PL，负责相机数据的接收和转发，包括 AXI DMA、片上存储单元、EMIO 接口、AURORA 64b/66b。调试接口。

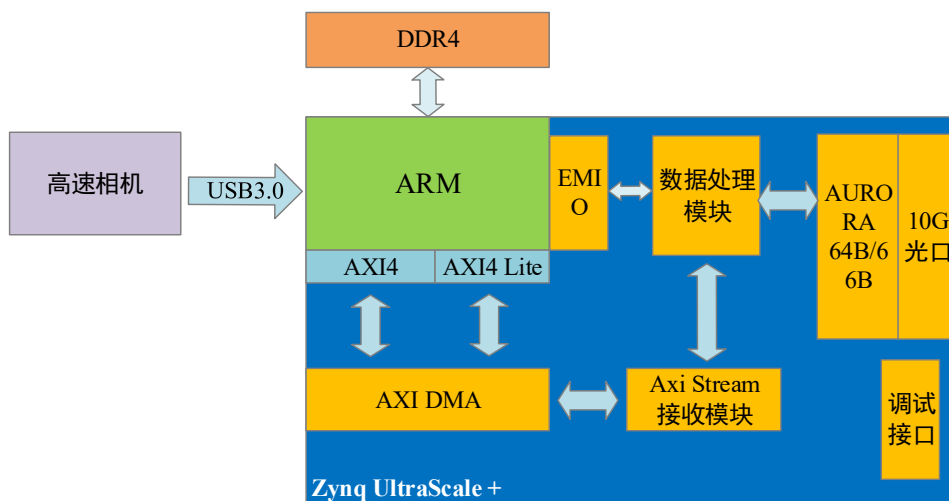


图 3 相机采集单元架构

其中 AXI DMA 用于搬移 PS 侧 DDR4 数据至 PL 侧片上存储阵列；EMIO 接口用于 PS 和 PL 间的 IO 通信；数据处理模块接收 10G 光口下发的配置数据和命令，同时将相机数据转发至上层；AURORA 64B/66B 和 10G 光口合并在一起将数据编码，并利用光纤介质传输至上层。

5.2.2 FPGA 逻辑设计

(1) CPU Block Design

CPU Block Design 利用 Vivado 的图形化编程功能，用于将 ARM 硬核和 AXI DMA 等模块进行总线互联。CPU Block Design 中包含 ARM 硬核、三组 AXI DMA、AXI Bram Ctrl 和 Block Ram，以及相应的 AXI Interconnect 等用于总线互连的模块，具体结构如下图所示。

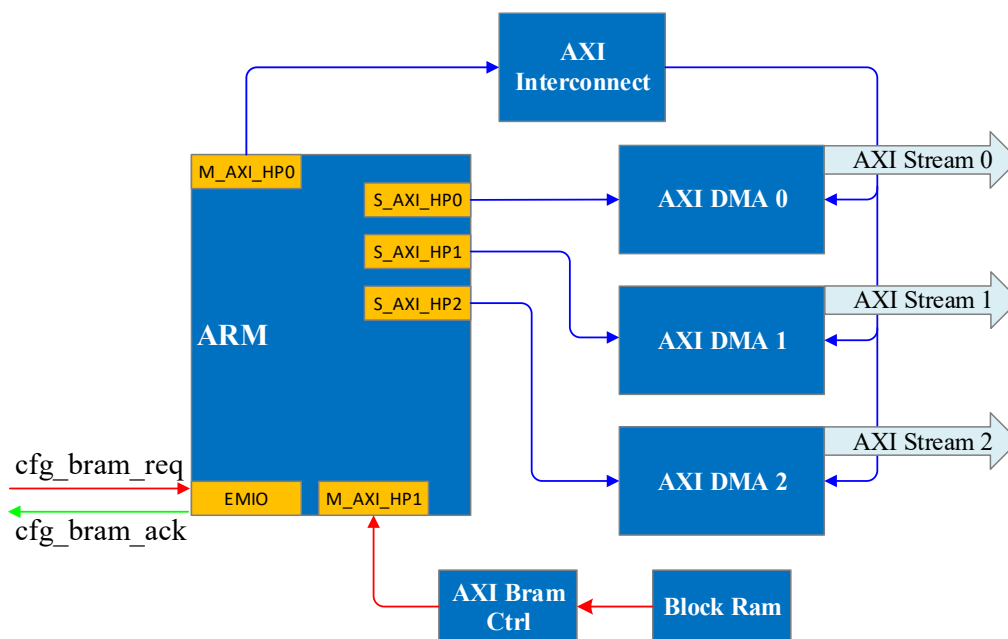


图 4 CPU Block Design 框图

其中 ARM 硬核配置了三组 AXI HP Slave 接口、2 组 AXI HP Master 接口和四组 EMIO 接口。三组 AXI HP Slave 接口分别连接至 AXI DMA 的 AXI4 接口，用于读取 PS 侧 DDR4 内部的数据，位宽设置为 64 比特，同时 AXI DMA 将读取后的数据流转换为 AXI Stream 接口形式的数据流输出至外部。AXI Stream 为 AXI4 协议面向高速数据流传输，采用无地址形式，允许无限制的数据突发传输规模。AXI HP0 Master 用于配置 AXI DMA，AXI DMA 配置接口采用 AXI4 Lite，3 组 AXI4 Lite 接口通过 AXI Interconnect 连接至 AXI HP0 Master。

寄存器配置列表如下所示。

表 5-12 寄存器配置表

Address Space Offset	Name	Description
00h	MM2S_DMCCR	MM2S DMA Control register
04h	MM2S_DMASR	MM2S DMA Status register
08h-14h	Reserved	N/A
18h	MM2S_SA	MM2S Source Address.
1Ch	MM2S_SA_MSB	MM2S Source Address.
28h	MM2S_LENGTH	MM2S Transfer Length (Bytes)
30h	S2MM_DMCCR	S2MM DMA Control register

34h	S2MM_DMASR	S2MM DMA Status register
38h-44h	Reserved	N/A
48h	S2MM_DA	S2MM Destination Address.
4Ch	S2MM_DA_MSB	S2MM Destination Address.
58h	S2MM_LENGTH	S2MM Buffer Length (Bytes)

AXI DMA 分为 MM2S 接口和 S2MM 接口，MM2S 将 PS 侧 DDR4 数据流转换至 PL 侧 AXI Stream 数据流，而 S2MM 则将 PL 侧 AXI Stream 数据流转换至 PS 侧 DDR4 数据流，本设计只使用 MM2S 接口。

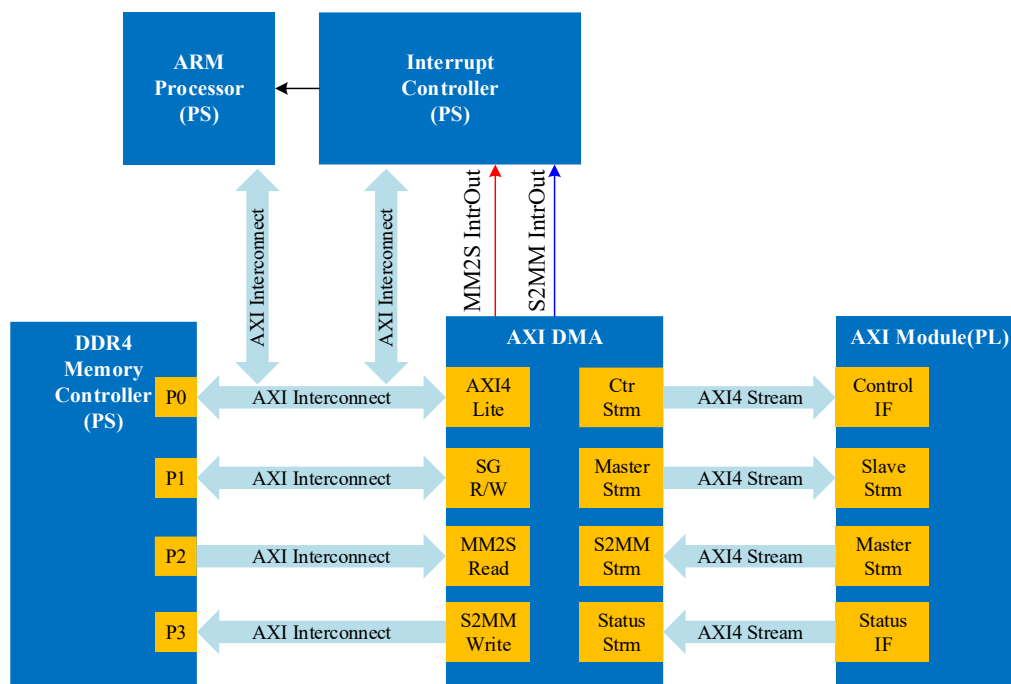


图 5 AXI DMA 常规架构

此外，AXI HP1 Master 用于 PS 读取上位机经 PL 下发的配置参数，每当上层下发配置参数或命令时，PL 都会将配置数据写入 Block Ram 中，并且置位 `cfg_bram_req`。PS 应用层序通过 EMIO 接口读取 `cfg_bram_req` 的值，检测为 1 时，通过 AXI Bram Ctrl 读取 Block Ram 数据，并置位 `cfg_bram_ack` 通知 PL 端配置参数已经被接收。

(2) AXI Stream 接收模块设计

AXI Stream 接收模块用于处理 CPU Block Design 输出的 3 路 AXI Stream 数据流，主要子模块包括 AXI Stream 包头信息提取和接收、数据缓存阵列、跨时钟域处理模块，具体结构如下图所示。

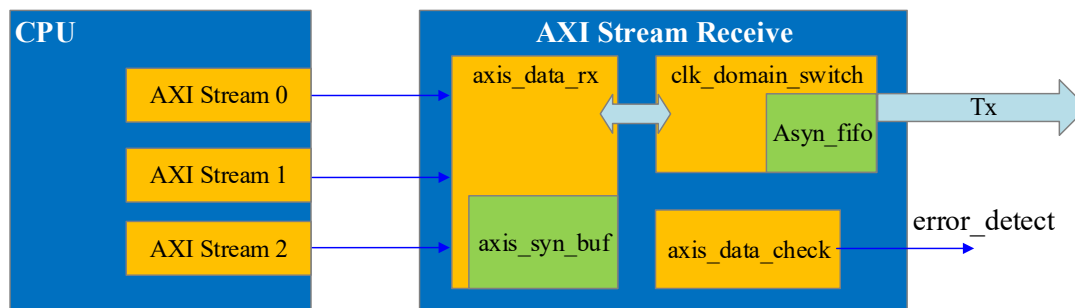


图 6 AXI Stream 接收模块框图

AXI Stream 数据包由 PS 下发至 PL，数据位宽 64 比特，长度 8194，由相机 ID 号、当

前帧号和图像数据（P）组成。相机 ID 号和帧号由 PS 端应用程序添加至数据包前端，相机 ID 号和帧号使用低 32 比特，高 32 比特保留，具体字段定义如下表所示。

表 5-13 AXI Stream 数据包字段定义

高 32 比特				低 32 比特			
Reserved				相机 ID			
Reserved				帧号			
P7	P6	P5	P4	P3	P2	P1	P0
P15	P14	P13	P12	P11	P10	P9	P8
.....							
P65535	P65534	P65533	P65532	P65531	P65530	P65529	P65528

axis_data_rx 模块接收 3 路 AXI Stream 信号，分别提取各自包头信息和数据。同时，将提取的数据进行分包存储，单路 AXI Stream 数据分为 32 个数据包，单个数据包格式为 64x256（位宽 64 比特，长度 256）共 2048 字节，并且每个数据附带相应的包头信息。包头信息除了要包含 PS 设立的相机 ID 和帧号外，还需要包括数据分包 index，总位宽 72 比特，分包包头信息字段如下表所示。

表 5-14 分包包头信息字段定义

71:40	39:8	7:0
相机 ID	帧号	Index

分包后的数据和信息存储在 PL 端片上存储单元 URAM 中，axis_syn_buf 为设计的片上存储阵列模块，其由 fifo 和 Uram 组成。fifo 用于存储包头信息，Uram 通过级联扩展至 64x8192（64 比特，8192 深度）容量，级联后的 Uram 能完整存储 1 帧 256x256 大小的图像。

clk_domain_switch 为设计的跨时钟域单元，AXI Stream 接收模块整体时钟域处在 CPU 侧，由 CPU 提供 200MHz 时钟，而 AURORA 64B/66B 侧的光口发送模块时钟为 156.25MHz。因此，两侧数据传输需要经过跨时钟域进行处理。多比特数据跨时钟域通过异步 fifo 缓存实现，clk_domain_switch 内部采用 64x512（宽度 64，深度 512）的异步 fifo，异步 fifo 输出后的数据传输至 AURORA 64B/66B 光口发送模块。

（3）AURORA 64B/66B 传输模块设计

Aurora 是一种轻量级链路层协议，可通过一个或多个高速串行链路进行点对点通信。本模块采用 Xilinx Aurora 64B/66B IP 核进行实现，包括数据发送缓存 fifo、数据接收缓存 fifo、数据处理模块、64B/66B 编解码等，具体结构如下图所示。

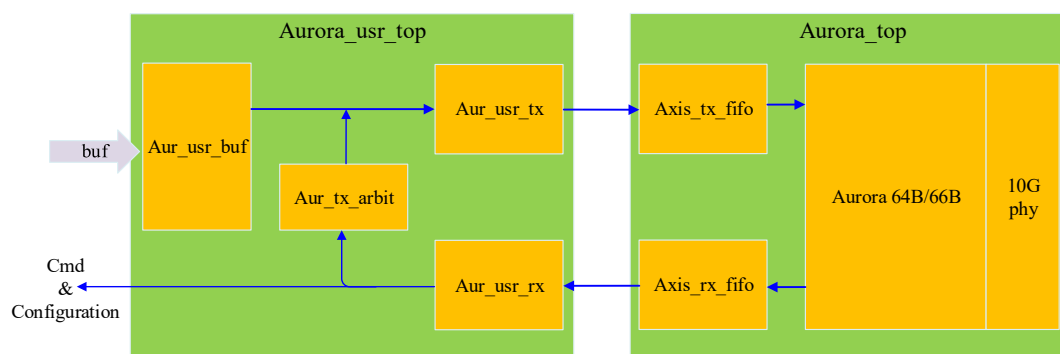


图 7 AURORA 64B/66B 传输模块框图

Aurora 传输模块分为两部分，上图左边为 aurora_usr_top 用于管理图像数据的发送和接收，上图右边为 Aurora 64B/66B IP 核用于将用户数据转换为适应物理器件发送的格式。aurora_usr_top 模块包括 aur_usr_buf、aur_tx_arbit、aur_usr_tx 和 aur_usr_rx，其中 aur_usr_buf 用于缓存从 AXI Stream 接收模块传输的数据，由 fifo 和 Uram 组成，可缓存一帧图像，共 3

组 buf。aur_tx_arbit 为发送仲裁模块，状态转移图如下图所示，各个状态含义如下：

S0：应答信号响应；S1：BUF0 发送；S2：BUF1 发送；S3：BUF2 发送。

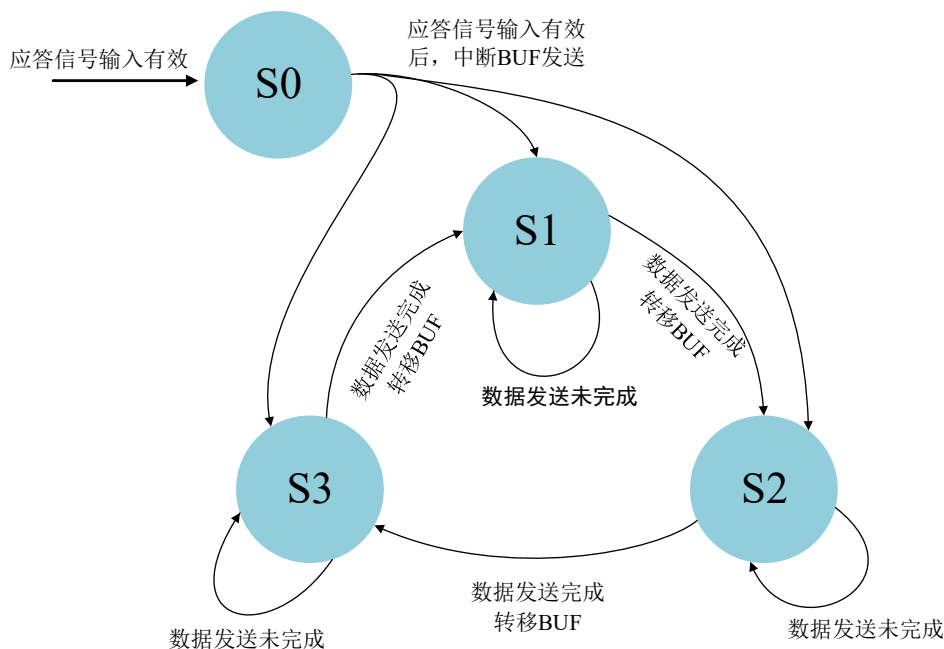


图 8 aur_tx_arbit 发送仲裁模块状态转移图

发送数据源来自 3 组 aur_usr_buf 以及应答信号，3 组 aur_usr_buf 对应 3 路相机数据，而应答信号来自上层 PC 对采集卡下发命令后的回应，由 aur_tx_arbit 为发送仲裁模块进行仲裁。应答信号的发送优先级最高，3 组 aur_usr_buf 采用轮询的方式来发送。从第一路 buf 开始发送数据，当第一路 buf 数据发送完成后，检查第二路 buf 是否有数据，如果有效则发送第二路 buf 数据，否则跳转至第三路 buf，依次流转下去。aur_usr_tx 为数据发送模块，其接收来自 aur_tx_arbit 的数据，以固定间隔发送数据包。aur_usr_rx 为数据接收模块，其包括相机配置模块 camera_config 和应答响应模块 aur_response，aur_usr_rx 根据收到的数据包类型确认是应答数据包，还是配置数据包。

aurora_top 为 Aurora 64B/66B IP 核部分，包括 axis_fifo_tx、axis_fifo_rx 和 Aurora 64B/66B IP 核。axis_fifo_tx 和 axis_fifo_rx 为 axi stream 接口形式的 fifo，以适应 Aurora 64B/66B IP 核接口。Aurora 64B/66B IP 包括以下部分，用户层、PCS 物理编码子层（Aurora 编解码器、64B/66B 解扰码器、变速箱、同步模块）、PMA 物理媒介相关子层（PISO 并行转串行、SIPO 串行转并行、CDR 时钟恢复），具体如下图所示。

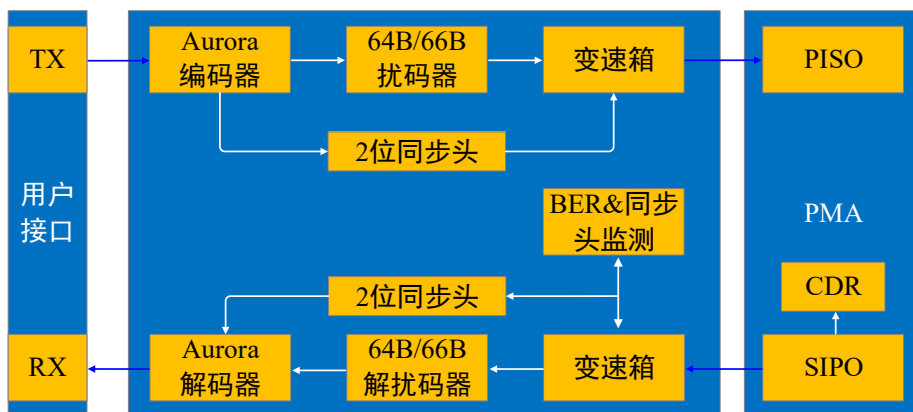


图 9 Aurora 64B/66B IP 框图

5.3 FPGA 实现代码

顶层文件代码如下：

```

module top(
    input          i_ref_100_p      ,
    input          i_ref_100_n      ,
    input          i_ref_sfp_p      ,
    input          i_ref_sfp_n      ,
    input          i_sys_rstn       ,
    input          sfp_rx_p         ,
    input          sfp_rx_n         ,
    output         sfp_tx_p         ,
    output         sfp_tx_n         ,
    output [1:0]   sfp_tx_disable   ,
    output [1:0]   led
);

    wire          udp_clk;
    wire          udp_rst;
    wire          tx_udp_hdr_valid   ;
    wire          tx_udp_hdr_ready   ;
    wire [15:0]   tx_udp_source_port ;
    wire [15:0]   tx_udp_dest_port   ;
    wire [15:0]   tx_udp_length      ;
    wire [63:0]   tx_udp_payload_axis_tdata ;
    wire [7 :0]   tx_udp_payload_axis_tkeep ;
    wire          tx_udp_payload_axis_tvalid ;
    wire          tx_udp_payload_axis_tready ;
    wire          tx_udp_payload_axis_tlast  ;
    wire          tx_udp_payload_axis_tuser  ;
    wire          rx_udp_hdr_valid   ;
    wire          rx_udp_hdr_ready   ;
    wire [15:0]   rx_udp_source_port ;
    wire [15:0]   rx_udp_dest_port   ;
    wire [15:0]   rx_udp_length      ;
    wire [63:0]   rx_udp_payload_axis_tdata ;
    wire [7 :0]   rx_udp_payload_axis_tkeep ;
    wire          rx_udp_payload_axis_tvalid ;
    wire          rx_udp_payload_axis_tready ;
    wire          rx_udp_payload_axis_tlast  ;
    wire          rx_udp_payload_axis_tuser  ;

    wire          o_ref_sfp_clk;
    // wire          o_clk_78p125;

```

```

// wire      o_rst_78p125;
// wire      o_clk_156p25;
// wire      o_rst_156p25;
// wire      o_clk_312p5;
// wire      o_rst_312p5;
wire         o_clk_100;
wire         o_rst_100;
wire         o_clk_200;
wire         o_rst_200;
wire         ddr_clk_lock;
wire         sfp_rx_block_lock;
wire         sfp_tx_bad_block;
wire         sfp_rx_bad_block;

wire [47:0]   local_mac   = 48'h02_00_00_00_00_00;
wire [31:0]   local_ip    = {8'd192, 8'd168, 8'd1,   8'd128};
wire [31:0]   host_ip     = {8'd192, 8'd168, 8'd1,   8'd100};
wire [31:0]   gateway_ip  = {8'd192, 8'd168, 8'd1,   8'd1};
wire [31:0]   subnet_mask = {8'd255, 8'd255, 8'd255, 8'd0};

assign led[0] = ddr_clk_lock;
assign led[1] = sfp_rx_block_lock;
assign sfp_tx_disable = 2'b00;

wire         cfg_bram_cfg      ;
wire         cfg_bram_cfg_done ;
wire [7:0]    cfg_bram_port_we  ;
wire [31:0]   cfg_bram_port_addr;
wire [63:0]   cfg_bram_port_din ;
wire         cfg_bram_port_en  ;
wire [63:0]   cfg_bram_port_dout;
wire [15:0]   udp_cbuf_size ;
wire [1:0]    udp_cbuf_wen  ;
wire [13:0]   udp_cbuf_waddr;
wire [127:0]  udp_cbuf_wdata;
wire [1:0]    udp_cbuf_wlast;
wire [143:0]  udp_cbuf_winfo;
wire [55:0]   device0_id;
wire [55:0]   device1_id;
wire [55:0]   device0_id_d;
wire [55:0]   device1_id_d;

```

```

clock_reset_gen                                u_clock_reset_gen
(
    .i_sys_rstn                                (i_sys_rstn          ),
    .i_ref_sfp_p                                (i_ref_sfp_p          ),
    .i_ref_sfp_n                                (i_ref_sfp_n          ),
    .o_ref_sfp_clk                              (o_ref_sfp_clk        ),
    // .o_clk_78p125                            (o_clk_78p125        ),
    // .o_rst_78p125                            (o_rst_78p125        ),
    // .o_clk_156p25                            (o_clk_156p25        ),
    // .o_rst_156p25                            (o_rst_156p25        ),
    // .o_clk_312p5                             (o_clk_312p5         ),
    // .o_rst_312p5                             (o_rst_312p5         ),
    .ddr_clk_lock                              (ddr_clk_lock         ),
    .i_ref_100_p                                (i_ref_100_p          ),
    .i_ref_100_n                                (i_ref_100_n          ),
    .o_clk_100                                  (o_clk_100           ),
    .o_rst_100                                  (o_rst_100           ),
    .o_clk_200                                  (o_clk_200           ),
    .o_rst_200                                  (o_rst_200           )
);

cpu_top                                          u_cpu_top
(
    .sys_clk_200m                              (o_clk_200            ),
    .sys_rst_200m                              (o_rst_200            ),
    .udp_clk                                    (udp_clk           ),
    .udp_rst                                    (udp_rst           ),
    .cfg_bram_cfg                              (cfg_bram_cfg        ),
    .cfg_bram_cfg_done                        (cfg_bram_cfg_done    ),
    .cfg_bram_port_we                         (cfg_bram_port_we     ),
    .cfg_bram_port_addr                      (cfg_bram_port_addr    ),
    .cfg_bram_port_din                      (cfg_bram_port_din      ),
    .cfg_bram_port_en                       (cfg_bram_port_en       ),
    .cfg_bram_port_dout                     (cfg_bram_port_dout      ),
    .device0_id                              (device0_id          ),
    .device1_id                              (device1_id          ),
    .udp_cbuf_size                           (udp_cbuf_size        ),
    .udp_cbuf_wen                            (udp_cbuf_wen          ),
    .udp_cbuf_waddr                         (udp_cbuf_waddr         ),
    .udp_cbuf_wdata                         (udp_cbuf_wdata         ),
    .udp_cbuf_wlast                         (udp_cbuf_wlast         ),
    .udp_cbuf_winfo                         (udp_cbuf_winfo         )
);

```

```

shift_reg #
(
    .WIDTH      (112),
    .LATENCY    (3)
)
u_shift_reg
(
    .clk         (udp_clk                ),
    .din         ({device1_id,device0_id  }),
    .dout        ({device1_id_d,device0_id_d} )
);

udp_usr_top                                u_udp_usr_top
(
    .clk                (udp_clk                ),
    .rst                (udp_rst                ),
    .device0_id         (device0_id_d          ),
    .device1_id         (device1_id_d          ),
    .udp_cbuf_size      (udp_cbuf_size         ),
    .udp_cbuf_wen       (udp_cbuf_wen         ),
    .udp_cbuf_waddr     (udp_cbuf_waddr        ),
    .udp_cbuf_wdata     (udp_cbuf_wdata        ),
    .udp_cbuf_wlast     (udp_cbuf_wlast        ),
    .udp_cbuf_winfo     (udp_cbuf_winfo        ),
    .udp_usr_tx_hdr_vld (tx_udp_hdr_valid      ),
    .udp_usr_tx_hdr_rdy (tx_udp_hdr_ready      ),
    .udp_usr_tx_source_port (tx_udp_source_port ),
    .udp_usr_tx_dest_port (tx_udp_dest_port    ),
    .udp_usr_tx_length  (tx_udp_length         ),
    .udp_usr_tx_axis_tdata (tx_udp_payload_axis_tdata ),
    .udp_usr_tx_axis_tkeep (tx_udp_payload_axis_tkeep ),
    .udp_usr_tx_axis_tvalid (tx_udp_payload_axis_tvalid),
    .udp_usr_tx_axis_tready (tx_udp_payload_axis_tready),
    .udp_usr_tx_axis_tlast (tx_udp_payload_axis_tlast ),
    .udp_usr_tx_axis_tuser (tx_udp_payload_axis_tuser ),
    .udp_usr_rx_hdr_vld  (rx_udp_hdr_valid      ),
    .udp_usr_rx_hdr_rdy  (rx_udp_hdr_ready      ),
    .udp_usr_rx_source_port (rx_udp_source_port ),
    .udp_usr_rx_dest_port (rx_udp_dest_port    ),
    .udp_usr_rx_length   (rx_udp_length         ),
    .udp_usr_rx_axis_tdata (rx_udp_payload_axis_tdata ),
    .udp_usr_rx_axis_tkeep (rx_udp_payload_axis_tkeep ),
    .udp_usr_rx_axis_tvalid (rx_udp_payload_axis_tvalid),

```

```

        .udp_usr_rx_axis_tready      (rx_udp_payload_axis_tready),
        .udp_usr_rx_axis_tlast      (rx_udp_payload_axis_tlast ),
        .udp_usr_rx_axis_tuser      (rx_udp_payload_axis_tuser ),
        .cfg_bram_cfg                (cfg_bram_cfg                ),
        .cfg_bram_cfg_done           (cfg_bram_cfg_done           ),
        .cfg_bram_port_we            (cfg_bram_port_we            ),
        .cfg_bram_port_addr          (cfg_bram_port_addr          ),
        .cfg_bram_port_din           (cfg_bram_port_din           ),
        .cfg_bram_port_en            (cfg_bram_port_en            ),
        .cfg_bram_port_dout          (cfg_bram_port_dout          )
    );

    udp_top                                u_udp_top
    (
        /*
         * Ethernet Serial data
         */
        .sfp_ctrl_clk                  (o_clk_100                  ), //free
run clk
        .sfp_ctrl_rst                  (o_rst_100                  ),
//Synchronous
        .sfp_ref_clk                   (o_ref_sfp_clk              ), //mgt ref
clk
        .sfp_rx_block_lock             (sfp_rx_block_lock         ),
        .sfp_tx_bad_block               (sfp_tx_bad_block          ),
        .sfp_rx_bad_block               (sfp_rx_bad_block          ),
        .sfp_tx_p                       (sfp_tx_p                  ),
        .sfp_tx_n                       (sfp_tx_n                  ),
        .sfp_rx_p                       (sfp_rx_p                  ),
        .sfp_rx_n                       (sfp_rx_n                  ),
        // Clock: 156.25MHz Synchronous reset
        .clk                           (udp_clk                    ),
        .rst                           (udp_rst                    ),
        // UDP frame input
        .tx_udp_hdr_valid               (tx_udp_hdr_valid          ),
        .tx_udp_hdr_ready               (tx_udp_hdr_ready          ),
        .tx_udp_source_port             (tx_udp_source_port        ),
        .tx_udp_dest_port               (tx_udp_dest_port          ),
        .tx_udp_length                  (tx_udp_length             ),
        .tx_udp_payload_axis_tdata      (tx_udp_payload_axis_tdata ),
        .tx_udp_payload_axis_tkeep      (tx_udp_payload_axis_tkeep ),
        .tx_udp_payload_axis_tvalid     (tx_udp_payload_axis_tvalid ),
        .tx_udp_payload_axis_tready     (tx_udp_payload_axis_tready ),
        .tx_udp_payload_axis_tlast      (tx_udp_payload_axis_tlast ),

```



```

        .tx_udp_payload_axis_tuser      (tx_udp_payload_axis_tuser    ),
// UDP frame output
        .rx_udp_hdr_valid_reg          (rx_udp_hdr_valid            ),
        .rx_udp_hdr_ready              (rx_udp_hdr_ready            ),
        .rx_udp_source_port_reg        (rx_udp_source_port          ),
        .rx_udp_dest_port_reg          (rx_udp_dest_port            ),
        .rx_udp_length_reg             (rx_udp_length               ),
        .rx_udp_payload_axis_tdata_reg (rx_udp_payload_axis_tdata    ),
        .rx_udp_payload_axis_tkeep_reg (rx_udp_payload_axis_tkeep    ),
        .rx_udp_payload_axis_tvalid_reg(rx_udp_payload_axis_tvalid   ),
        .rx_udp_payload_axis_tready    (rx_udp_payload_axis_tready   ),
        .rx_udp_payload_axis_tlast_reg (rx_udp_payload_axis_tlast    ),
        .rx_udp_payload_axis_tuser_reg (rx_udp_payload_axis_tuser    ),
// Configuration
        .local_mac                    (local_mac                    ),
        .local_ip                     (local_ip                     ),
        .host_ip                      (host_ip                      ),
        .gateway_ip                   (gateway_ip                   ),
        .subnet_mask                   (subnet_mask                 )
    );

    reg [31:0] udp_tx_cnt;
    wire      udp_tx_cnt_clear;
    reg      udp_vld;
    reg [7:0] udp_data_cnt;
    reg [31:0] error;
    reg      hdr_vld;
    reg [31:0] low;

    always@(posedge udp_clk) begin
        hdr_vld <= (tx_udp_hdr_valid&tx_udp_hdr_ready);

        if(udp_rst == 1'b1 || tx_udp_payload_axis_tlast==1'b1)
            udp_vld <= 1'b0;
        else if(hdr_vld)
            udp_vld <= 1'b1;
    end

    always@(posedge udp_clk) begin
        if(udp_rst == 1'b1 || tx_udp_payload_axis_tlast==1'b1)
            udp_data_cnt <= 'd0;
        else if(udp_vld ==
1'b1&&tx_udp_payload_axis_tvalid&&tx_udp_payload_axis_tready)
            udp_data_cnt<=udp_data_cnt+1;
    end

```

```

end

always@(posedge udp_clk) begin
    if(udp_rst == 1'b1)
        error <= 'd0;
    else if(udp_vld&&tx_udp_payload_axis_tlast&&udp_data_cnt!=8'd129)
        error<=error+1;
end

always@(posedge udp_clk) begin
    if(udp_rst == 1'b1)
        low <= 'd0;
    else
        if(udp_vld&&(tx_udp_payload_axis_tvalid==0 | tx_udp_payload_axis_tready==0))
            low<=low+1;
end

always@(posedge udp_clk) begin
    if(udp_rst == 1'b1 | udp_tx_cnt_clear)
        udp_tx_cnt <= 32'd0;
    else if(tx_udp_hdr_valid&&tx_udp_hdr_ready)
        udp_tx_cnt <= udp_tx_cnt + 32'd1;
end

ila_0      u_ila_0
(
    .clk      (udp_clk      ),// input wire clk
    .probe0    ({rx_udp_hdr_valid,rx_udp_hdr_ready}),// 2
    .probe1    ({rx_udp_payload_axis_tvalid,rx_udp_payload_axis_tready,rx_udp_payload_axis_tlast} ),// 3
    .probe2    (rx_udp_payload_axis_tdata      ),// 64
    .probe3    (rx_udp_payload_axis_tkeep),//8
    .probe4    ({tx_udp_hdr_valid,tx_udp_hdr_ready,tx_udp_payload_axis_tvalid,tx_udp_payload_axis_tready,tx_udp_payload_axis_tlast} ),// 5
    .probe5    (tx_udp_payload_axis_tdata),// 64
    .probe6    (udp_tx_cnt),// 32
    .probe7    (cfg_bram_port_we),// 8
    .probe8    (cfg_bram_port_addr),// 32
    .probe9    (cfg_bram_port_din),// 64
    .probe10    ({cfg_bram_cfg,cfg_bram_cfg_done,cfg_bram_port_en}),// 3
    .probe11    ({low,error}), // 64
    .probe12    ({sfp_rx_block_lock,sfp_tx_bad_block,sfp_rx_bad_block}) // 3
)

```

```

);

vio_1      u_vio_1
(
    .clk      (udp_clk      ),// input wire clk
    .probe_out0 (udp_tx_cnt_clear      )// output wire [0 : 0] probe_out0
);

endmodule

```

5.4 FPGA 硬件调试

5.4.1 调试环境及流程

搭建 1×3 相机阵列进行相机采集控制实验，验证上位机控制效果、显示效果及系统处理性能。实验平台硬件连接图如下图。相机采集单元一端通过 USB3.0 接口，经高速线缆连接 3 个高速相机，另一端通过 SPF+ 接口和光纤连接上位机。12V 直流电源为相机采集单元供电。HDMI 显示器连接主机，显示采集到的图像数据。

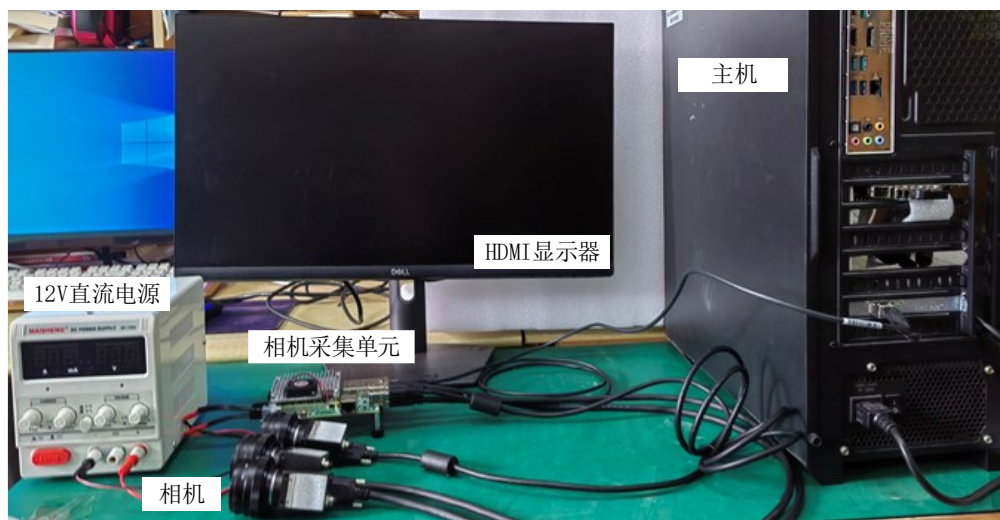


图 10 相机采集控制实验硬件连接图

实验流程：上位机下发 UDP 链路确认包，待收到相机采集单元应答后开始相机配置，相机配置参数如下图所示。为监控上位机与相机采集单元的通信过程，使用 Wireshark 软件抓取主机 192.168.1.100 地址下的 UDP 包，包括 UDP 链路确认包、应答包、参数配置包、启动采样包。随后，控制相机采集单元分别以 500fps 和 1000fps 的帧率进行数据采集和上报，实时显示视频。下面各图展示了上述各个阶段 Wireshark 抓包结果，数据下发准确无误。

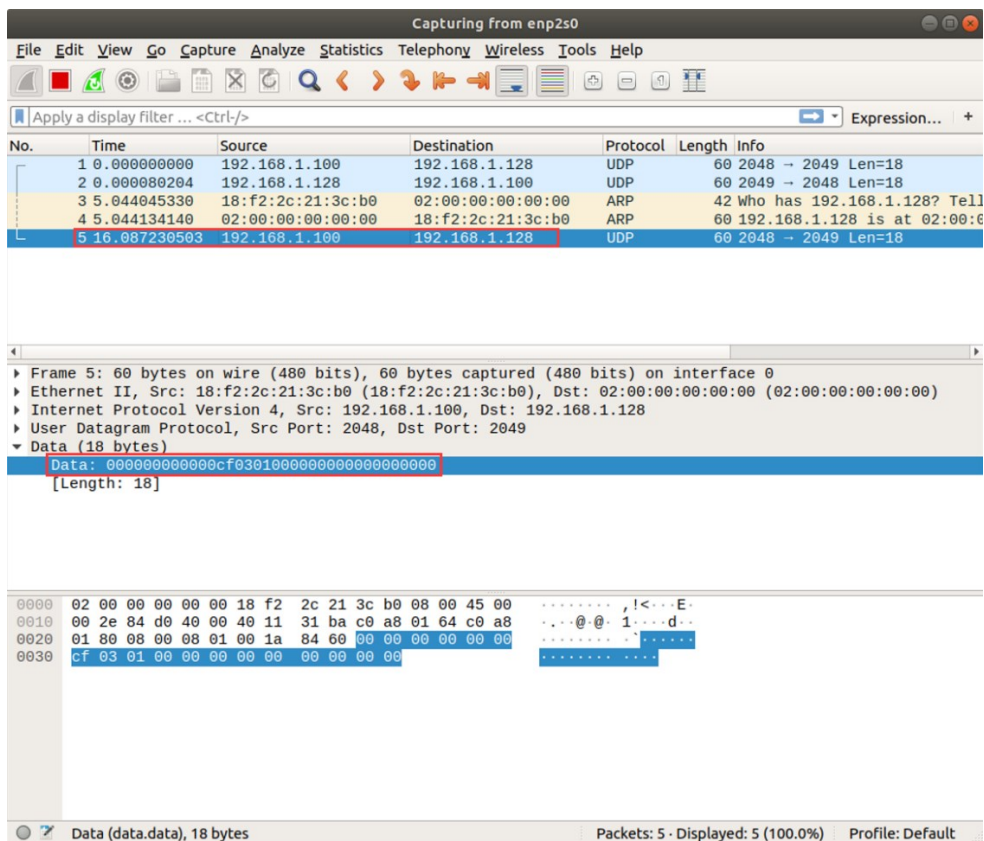


图 13 相机配置命令 UDP 数据包

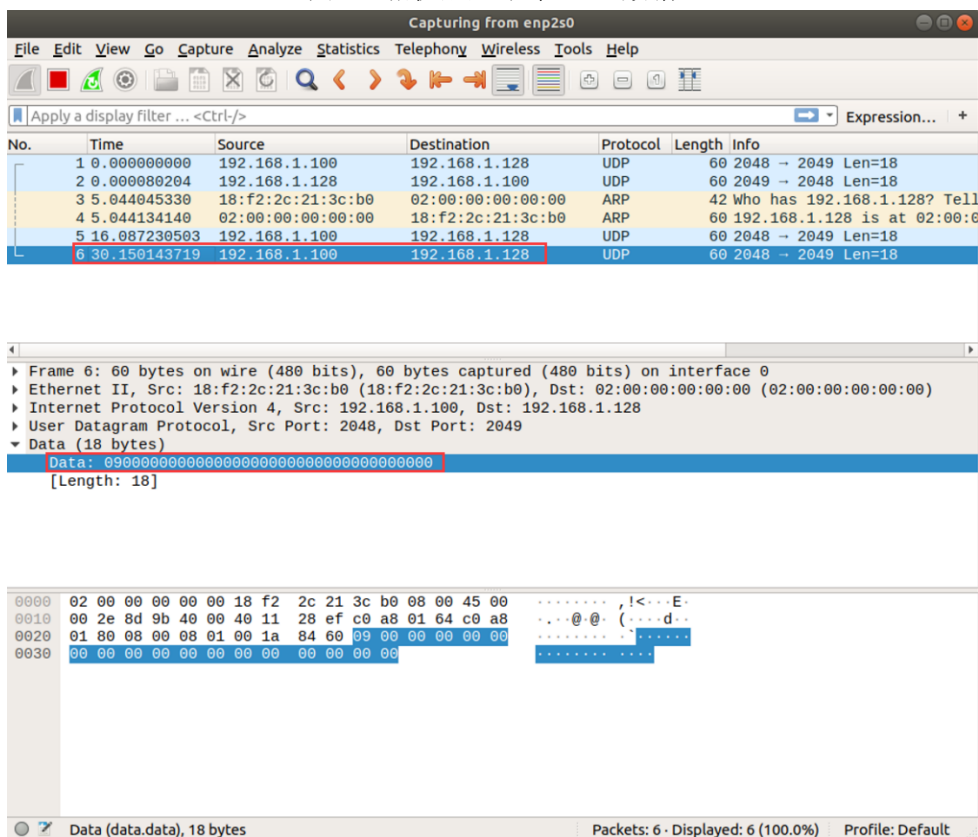


图 14 启动采样命令 UDP 数据包

5.4.2 实验结果及分析

在 1000fps 帧率下，上位机视频显示效果如下图。上位机实时显示 3 个相机画面，画质清晰流畅。使用 nload 网速检测工具查看系统通信速率，结果如下图 16 和图 17 所示。由图可知，在 500fps 帧率下，系统传输速率为 785.12MBit/s，在 1000fps 帧率下，系统传输速率为 1.51GBit/s。

本文单个相机工作分辨率固定为 256×256 像素，8bit 灰度图像，相机采集单元每采集 3 个相机的一帧图像后进行一次数据上报，在 500fps 帧率条件下，单个采集单元数据上报速率理论值为：

$$256 \times 256 \times 8 \times 500 \times 3 \approx 786.43\text{MBps}$$

在 1000fps 帧率条件下，单个采集单元数据上报速率理论值为：

$$256 \times 256 \times 8 \times 1000 \times 3 \approx 1.57\text{GBps}$$

对比实际测速结果与理论结果计算，在 500fps 与 1000fps 帧率条件下，系统性能符合设计预期，但仍有改进空间。

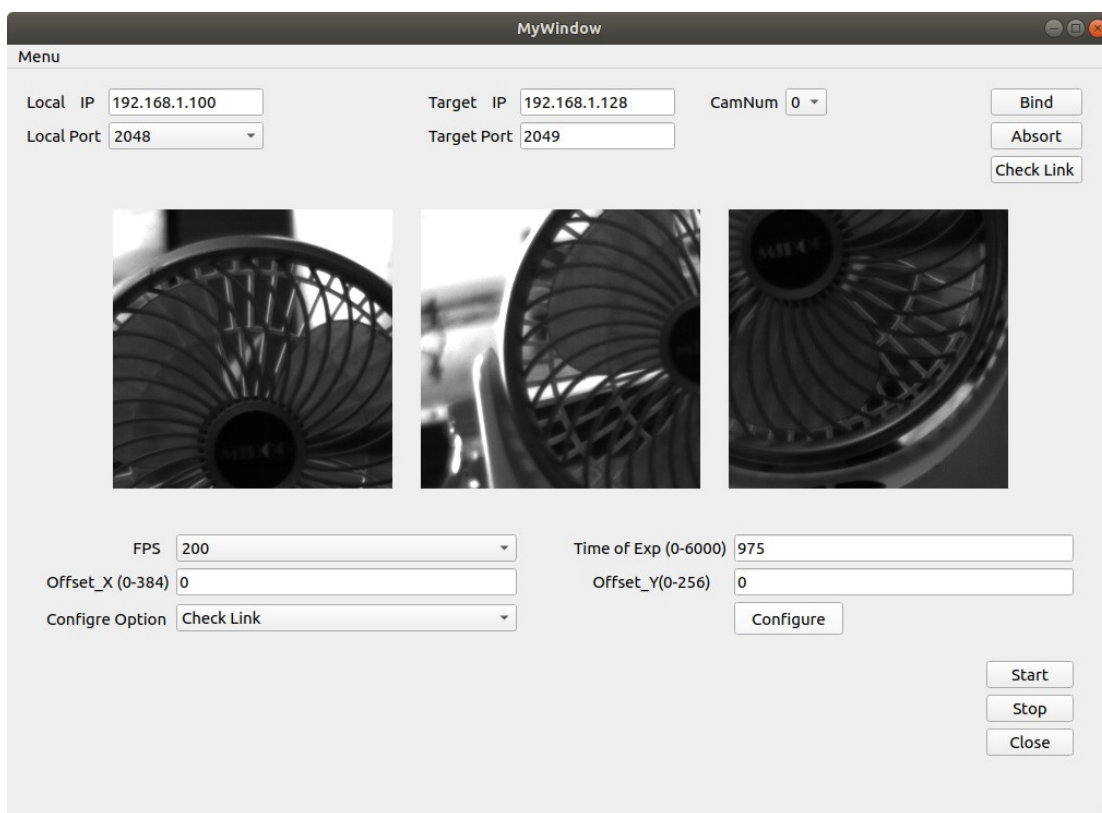


图 15 上位机及图像显示结果


```
xzx@xilinx: ~  
File Edit View Search Terminal Help  
Device enp2s0 [192.168.1.100] (2/3):  
===== Incoming: #####  
#####  
#####  
#####  
#####  
##### Curr: 785.12 MBit/s  
##### Avg: 565.13 MBit/s  
##### Min: 0.00 Bit/s  
##### Max: 3.01 GBit/s  
##### Ttl: 35.74 GByte  
Outgoing:  
  
Curr: 0.00 Bit/s  
Avg: 8.00 Bit/s  
Min: 0.00 Bit/s  
Max: 1.83 kBit/s  
Ttl: 8.30 kByte
```

图 16 500fps 条件下数据传输速率

```
xzx@xilinx: ~  
File Edit View Search Terminal Help  
Device enp2s0 [192.168.1.100] (2/3):  
===== Incoming: #####  
#####  
#####  
#####  
#####  
##### Curr: 1.51 GBit/s  
##### Avg: 229.97 MBit/s  
##### Min: 0.00 Bit/s  
##### Max: 3.01 GBit/s  
##### Ttl: 23.45 GByte  
Outgoing:  
  
Curr: 0.00 Bit/s  
Avg: 24.00 Bit/s  
Min: 0.00 Bit/s  
Max: 1.83 kBit/s  
Ttl: 7.81 kByte
```

图 17 1000fps 条件下数据传输速率

5.5 参考文献

[1]杨朋飞. 阵列相机图像采集控制与处理技术研究[D]. 西安工业大学. 2020.

6. 基于 FPGA 的 flash 在线 jtag 编码与识别

6.1 算法原理

当 FPGA 主芯片通过 FPGA 的 USB 下载器接入电脑时，电脑设备管理器中常常出现的 FPGA 的 USB 下载器的编号的顺序与我们插拔 FPGA 的 USB 下载器的顺序有很大关系，然而这些编号并不能反应我们要操作的具体 FPGA 型号，这无疑使我们在下载 FPGA 程序进入 FPGA 芯片时对着号，故在这种背景下考虑开发一种在线编码识别 FPGA 芯片的方法，使得 FPGA 开发板设备编号与 USB 端口批量匹配。以前传统实验教学时，学生都是人手一套实验开发板和一台电脑，接入电脑时所对应的 FPGA 的 USB 下载器必是所连的开发板，这一点大家都是看作理所应当，然而接入多篇 FPGA 芯片时，便没有考虑编号所对应的具体芯片，人为的去插拔识别不同 FPGA 的芯片实在是费力麻烦。

本算法利用本地实验开发板 FPGA 芯片外挂了一片非配置用的 Flash 芯片,Flash 芯片可以通过编码的比特流文件来存储设备编码,然后单个开发板通过批处理命令下载进 FPGA 实验设备上的 FPGA 芯片中,FPGA 通过 SPI 接口往 Flash 芯片里写入 8 位的设备编码(往 flash 里面写入数据前,必须先擦除)。然后用 python+bat+tcl 生成的 exe 文件运行,exe 文件里面核心算法步骤为:首先探寻本地挂载的 FPGA 下载器的数量,然后往本地实验开发板 FPGA 主芯片加载相同的配置文件,配置逻辑通过 SPI 总线读取 Flash 芯片中设备编码信息并进行校验,并且使用基于 JTAG 虚拟 IO 模块的调试技术(此处实现时用到了具体的 in-sysystem sources and probes IP 核),此技术不需要额外的 FPGA 引脚,并且将读取的设备编码信息传入 JTAG 虚拟 IO 模块.exe 可执行程序然后根据搜索的本地挂载的 FPGA 下载器的数量多次执行基于虚拟 JTAG IO 相关的 Tcl 脚本命令,实时获得实验板中的所有设备编号,构建 FPGA 下载器编号与所获取的开发板设备编号一一对应的表格文件。

6.2 python、tcl 验证代码

擦除 flash, 以及往 flash 里面写入相应的编码的批处理命令:

```
%QUARTUS_ROOTDIR%\bin64\quartus_pgm.exe -m jtag -c USB-Blaster[USB-0] -o p;e1.sof
pause
%QUARTUS_ROOTDIR%\bin64\quartus_pgm.exe -m jtag -c USB-Blaster[USB-0] -o
p;big_write.sof
pause
```

其中 e1.sof 和 big_write.sof 分别为全擦除 flash 和往 flash 指定区域写入 8 位编码的下载文件。

```
python 代码;
import os
import re
c=os.path.abspath('.')#得到当前目录
d=c+"\\tcl1.tcl"
d1=d.replace("\\", "\\")#得到当前脚本目录 C:\\Users\\YR_LSN\\Desktop\\test2\\tcl1.tcl, 将
单斜杠变成双斜杠
#print(d1)
if os.path.exists("4.txt")==1: #每次运行时, 删除 4.txt, 最后会生成最新的 4.txt, 器件编
```


码就在 4.txt 中

```

os.remove('4.txt')
os.system("""%QUARTUS_ROOTDIR%\bin64\quartus_pgm.exe -l">1.txt) #判断连接的
的 usb—blaster 的数量，输出信息到 1.txt 中（1.txt 可删）
txt=open(r"1.txt", "r").read() #.read()函数作用是读取打开的 txt 文档的内容
p1=r"USB-[0-9]"#这是我们写的正则表达式规则
pat=re.compile(p1) # 编译
#print(pat.findall(txt)) #打印匹配的 usb-[?] 这一段的作用就是从 1.txt 中找出 usb 的数
量，只能这样
f=open('tcl1.tcl', 'r') #就是修改 usb[0]到 usb[?],否则取不到除 usb[0]之外的其他器件的
值
list1 = f.readlines() #得到一个列表，列表中每个值就是每一行的字符串，我们只改变
第一行的元素
f.close()
for i in range(len(pat.findall(txt))): #每次下载一个 sof 得到一个编码值
ins= "%QUARTUS_ROOTDIR%\bin64\quartus_pgm.exe -m jtag -c USB-Blaster[USB-
"+str(i)+" ] -o p;test.sof "
os.system(ins) #给 usb[?]下载 test.sof
ins1="set usb [index [get _hardware_names] "+str(i)+" ]\n" #这就是要修改的 tcl1.tcl
的第一行的字符串
list1[0]=ins1 #替换后写入
f1 = open('tcl1.tcl', 'w') #写会覆盖
f1.writelines(list1)
f1.close()

result=os.popen(r"cd %QUARTUS_ROOTDIR%\bin64 && quartus_stp -t"+d1) #此
处的 d1 就是运用了相对路径
y1=result.readlines()
#print(y1[-7]) #y1[-7]就是我们需要的编码值
f4=open('4.txt','a+') #写入 4.txt 中
f4.write(y1[-7])
f4.close() #关闭文件
print("运行成功")
#下面代码纯属多余，只为让结果更加美观
duo1=open(r"1.txt", "r")
duo2=open(r"4.txt", "r")
d1=duo1.readlines()
d2=duo2.readlines()
duo3=open(r"5.txt", "w")
del d1[len(d2):]
for i in range(len(d2)):
duo3.write(d1[i].replace('\n', "").replace('\r', "")+" : "+d2[i])
duo1.close() #关闭文件
duo2.close() #关闭文件

```

```
duo3.close() #关闭文件
```

Tcl 代码:

```
set usb [lindex [get_hardware_names] 0]
```

```
set device_name [lindex [get_device_names -hardware_name $usb] 0]
```

```
proc read {} {
```

```
    global device_name usb
```

```
    start_insystem_source_probe -device_name $device_name -hardware_name $usb
```

```
    set x [read_probe_data -instance_index 0]
```

```
    end_insystem_source_probe
```

```
    puts $x
```

```
    return $x
```

```
}
```

```
Read
```

6.3 FPGA 实现架构

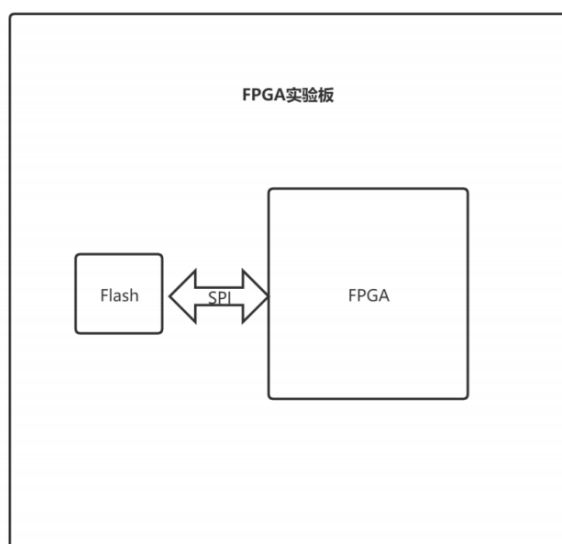


图 1 单块开发板架构图

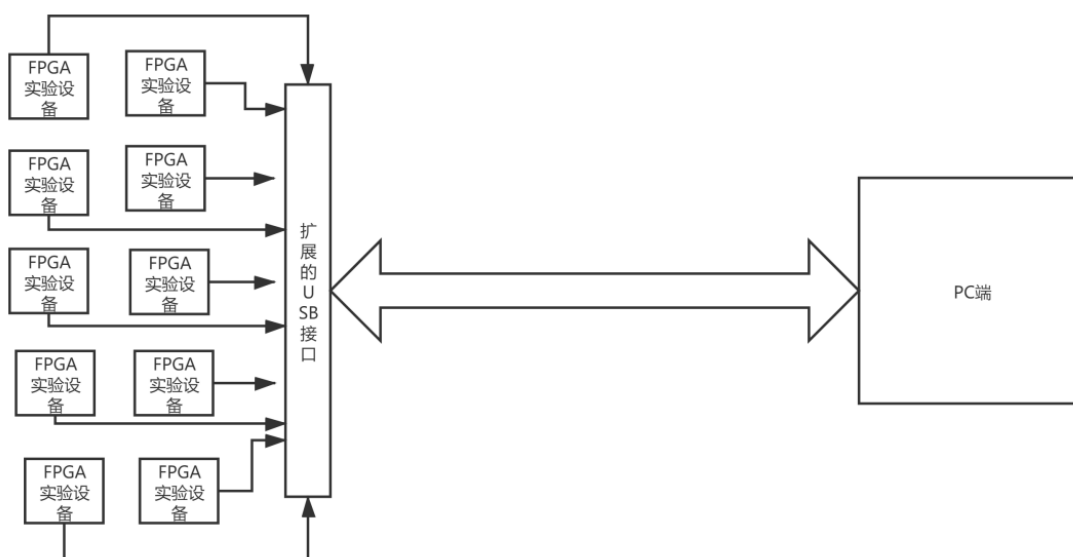


图 2 多块开发板整体接入架构图

6.4 FPGA 实现代码

全擦除代码:

```

module spi_flash_be
(
    input wire sys_clk      , //系统时钟, 频率 50MHz
    input wire sys_rst_n    , //复位信号,低电平有效
    //input wire pi_key      , //按键输入信号
    output wire cs_n        , //片选信号
    output wire sck         , //串行时钟
    output wire mosi        //主输出从输入数据
);
//*****
//***** Parameter and Internal Signal *****//
//*****
//wire define
wire po_key ;
//*****
//***** Instantiation *****//
//*****
key_moni key_moni_1

(
    .clk      (sys_clk      ), //系统时钟, 频率 50MHz
    .ret      (sys_rst_n    ), //复位信号,低电平有效

    .key      (po_key       ) //消抖后信号,高一个时钟
);

//----- flash_be_ctrl_inst -----
flash_be_ctrl flash_be_ctrl_inst
(
    .sys_clk      (sys_clk      ), //系统时钟, 频率 50MHz
    .sys_rst_n    (sys_rst_n    ), //复位信号,低电平有效
    .key          (po_key       ), //按键输入信号

    .sck          (sck         ), //片选信号
    .cs_n         (cs_n        ), //串行时钟
    .mosi         (mosi        ) //主输出从输入数据
);
Endmodule

```

```
module key_moni(
input clk,
input ret,
output reg key
);
reg [9:0] count_1;
reg [1:0] count_2;

reg [2:0] state;
parameter idle=3'b001;
parameter start_1=3'b010;
parameter start_2=3'b100;

always@(posedge clk or negedge ret)
if(!ret)
count_1<=0;
else if(count_1==d100)
count_1<=0;
else if(state==start_2)
count_1<=count_1;
else
count_1<=count_1+1;

always@(posedge clk or negedge ret)
if(!ret)
count_2<=0;
else if(count_1==d100)
count_2<=count_2+1;

always@(posedge clk or negedge ret)
if(!ret)
state<=idle;
else
case(state)
idle:if(count_1==d96&&count_2==2'd2)
state<=start_1;
start_1:if(count_1==d97&&count_2==2'd2)
state<=start_2;
start_2:
state<=start_2;
default:state<=idle;
endcase
```

```

always@(posedge clk or negedge ret)
if(!ret)
key<=0;
else if(state==start_1)
key<=1;
else
key<=0;
endmodule

module flash_be_ctrl
(
    input  wire      sys_clk      , //系统时钟, 频率 50MHz
    input  wire      sys_rst_n    , //复位信号,低电平有效
    input  wire      key          , //按键输入信号

    output reg        cs_n        , //片选信号
    output reg        sck         , //串行时钟
    output reg        mosi        //主输出从输入数据
);

//*****
//***** Parameter and Internal Signal *****
//*****

//parameter define
parameter IDLE      = 4'b0001, //初始状态
           WR_EN     = 4'b0010, //写状态
           DELAY     = 4'b0100, //等待状态
           BE        = 4'b1000; //全擦除状态
parameter WR_EN_INST = 8'b0000_0110, //写使能指令
           BE_INST   = 8'b1100_0111; //全擦除指令

//reg define
reg [2:0] cnt_byte; //字节计数器
reg [3:0] state; //状态机状态
reg [4:0] cnt_clk; //系统时钟计数器
reg [1:0] cnt_sck; //串行时钟计数器
reg [2:0] cnt_bit; //比特计数器
//*****
//***** Main Code *****
//*****

//cnt_clk:系统时钟计数器, 用以记录单个字节
always@(posedge sys_clk or negedge sys_rst_n)
if(sys_rst_n == 1'b0)

```

```

        cnt_clk <= 5'd0;
    else if(state != IDLE)
        cnt_clk <= cnt_clk + 1'b1;
//cnt_byte:记录输出字节个数和等待时间
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        cnt_byte <= 3'd0;
    else if((cnt_clk == 5'd31) && (cnt_byte == 3'd6))
        cnt_byte <= 3'd0;
    else if(cnt_clk == 31)
        cnt_byte <= cnt_byte + 1'b1;

//cnt_sck:串行时钟计数器，用以生成串行时钟
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        cnt_sck <= 2'd0;
    else if((state == WR_EN) && (cnt_byte == 1'b1))
        cnt_sck <= cnt_sck + 1'b1;
    else if((state == BE) && (cnt_byte == 3'd5))
        cnt_sck <= cnt_sck + 1'b1;

//cs_n:片选信号
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        cs_n <= 1'b1;
    else if(key == 1'b1)
        cs_n <= 1'b0;
    else if((cnt_byte == 3'd2) && (cnt_clk == 5'd31) && (state == WR_EN))
        cs_n <= 1'b1;
    else if((cnt_byte == 3'd3) && (cnt_clk == 5'd31) && (state == DELAY))
        cs_n <= 1'b0;
    else if((cnt_byte == 3'd6) && (cnt_clk == 5'd31) && (state == BE))
        cs_n <= 1'b1;

//sck:输出串行时钟
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        sck <= 1'b0;
    else if(cnt_sck == 2'd0)
        sck <= 1'b0;
    else if(cnt_sck == 2'd2)
        sck <= 1'b1;

//cnt_bit:高低位对调，控制 mosi 输出

```

```

always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        cnt_bit <= 3'd0;
    else if(cnt_sck == 2'd2)
        cnt_bit <= cnt_bit + 1'b1;

//state:两段式状态机第一段，状态跳转
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        state <= IDLE;
    else
        case(state)
            IDLE: if(key == 1'b1)
                    state <= WR_EN;
            WR_EN: if((cnt_byte == 3'd2) && (cnt_clk == 5'd31))
                    state <= DELAY;
            DELAY: if((cnt_byte == 3'd3) && (cnt_clk == 5'd31))
                    state <= BE;
            BE: if((cnt_byte == 3'd6) && (cnt_clk == 5'd31))
                    state <= IDLE;
            default: state <= IDLE;
        endcase

//mosi:两段式状态机第二段，逻辑输出
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        mosi <= 1'b0;
    else if((state == WR_EN) && (cnt_byte == 3'd2))
        mosi <= 1'b0;
    else if((state == BE) && (cnt_byte == 3'd6))
        mosi <= 1'b0;
    else if((state == WR_EN) && (cnt_byte == 3'd1) && (cnt_sck == 5'd0))
        mosi <= WR_EN_INST[7 - cnt_bit]; //写使能指令
    else if((state == BE) && (cnt_byte == 3'd5) && (cnt_sck == 5'd0))
        mosi <= BE_INST[7 - cnt_bit]; //全擦除指令

Endmodule

```

SignalTap 仿真波形：

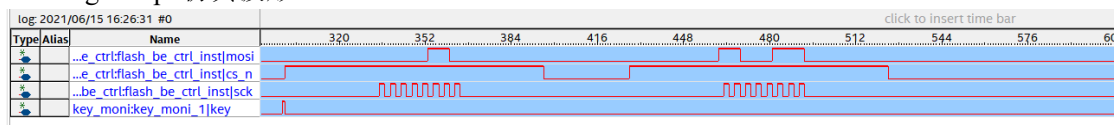


图3 全擦除 flash 波形图

往 flash 写入 8bit 数的代码：

```
module top(
```

```

        input clk,
        input ret,

        output wire sck, //串行时钟
        output wire cs_n, //片选信号
        output wire mosi //主输出从输入数据

    );
    wire key;
    parameter data=8'b0000_0001;

    key_moni key_moni(
        .clk(clk),
        .ret(ret),
        .key(key)
    );

    flash_seq_wr_ctrl flash_seq_wr_ctrl(
        .sys_clk(clk), //系统时钟, 频率 50MHz
        .sys_rst_n(ret), //复位信号,低电平有效
        .pi_flag(key), //数据标志信号
        .pi_data(data), //写入数据
        .sck(sck), //串行时钟
        .cs_n(cs_n), //片选信号
        .mosi(mosi) //主输出从输入数据
    );

    endmodule

    module key_moni(
    input clk,
    input ret,
    output reg key
    );
    reg [9:0] count_1;
    reg [1:0] count_2;

    reg [2:0] state;
    parameter idle=3'b001;
    parameter start_1=3'b010;
    parameter start_2=3'b100;
    always@(posedge clk or negedge ret)
    if(!ret)
    count_1<=0;

```



```

else if(count_1=='d100)
count_1<=0;
else if(state==start_2)
count_1<=count_1;
else
count_1<=count_1+1;

always@(posedge clk or negedge ret)
if(!ret)
count_2<=0;
else if(count_1=='d100)
count_2<=count_2+1;

always@(posedge clk or negedge ret)
if(!ret)
state<=idle;
else
case(state)
idle:if(count_1=='d96&&count_2==2'd2)
state<=start_1;
start_1:if(count_1=='d97&&count_2==2'd2)
state<=start_2;
start_2:
state<=start_2;
default:state<=idle;
endcase

always@(posedge clk or negedge ret)
if(!ret)
key<=0;
else if(state==start_1)
key<=1;
else
key<=0;

endmodule

module flash_seq_wr_ctrl(

    input  wire      sys_clk      , //系统时钟, 频率 50MHz
    input  wire      sys_rst_n   , //复位信号,低电平有效
    input  wire      pi_flag     , //数据标志信号
    input  wire [7:0] pi_data    , //写入数据

    output reg       sck         , //串行时钟

```

```

output reg          cs_n      ,    //片选信号
output reg          mosi      //主输出从输入数据

);

//*****
//***** Parameter and Internal Signal *****
//*****
//parameter define
parameter IDLE      = 4'b0001 ,    //初始状态
           WR_EN     = 4'b0010 ,    //写状态
           DELAY     = 4'b0100 ,    //等待状态
           PP        = 4'b1000 ;    //扇区擦除状态
parameter WR_EN_INST = 8'b0000_0110, //写使能指令
           PP_INST    = 8'b0000_0010; //扇区擦除指令
parameter ADDR       = 24'h00_04_25; //数据写入地址

//reg define
reg [23:0] addr_reg; //数据写入地址寄存器
reg [23:0] addr      ; //数据写入地址

reg [4:0] cnt_clk ; //系统时钟计数器
reg [3:0] state   ; //状态机状态
reg [3:0] cnt_byte; //字节计数器
reg [1:0] cnt_sck ; //串行时钟计数器
reg [2:0] cnt_bit ; //比特计数器

//*****
//***** Main Code *****
//*****

//cnt_clk:系统时钟计数器，用以记录单个字节
always@(posedge sys_clk or  negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        cnt_clk  <= 5'd0;
    else if(state != IDLE)
        cnt_clk  <= cnt_clk + 1'b1;

//cnt_byte:记录输出字节个数和等待时间
always@(posedge sys_clk or  negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        cnt_byte  <= 4'd0;
    else if((cnt_clk == 5'd31) && (cnt_byte == 4'd10))
        cnt_byte  <= 4'd0;

```

```

else    if(cnt_clk == 31)
    cnt_byte    <=  cnt_byte + 1'b1;

//cnt_sck:串行时钟计数器，用以生成串行时钟
always@(posedge sys_clk or  negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        cnt_sck <=  2'd0;
    else    if((state == WR_EN) && (cnt_byte == 1'b1))
        cnt_sck <=  cnt_sck + 1'b1;
    else    if((state == PP) && (cnt_byte >= 4'd5) && (cnt_byte <= 4'd9))
        cnt_sck <=  cnt_sck + 1'b1;

//addr_reg:数据写入地址寄存器
always@(posedge sys_clk or  negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        addr_reg    <=  ADDR;
    else    if(pi_flag == 1'b1)
        addr_reg    <=  addr_reg + 1'b1 ;

//addr:数据写入地址
always@(posedge sys_clk or  negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        addr    <=  24'd0;
    else    if(pi_flag == 1'b1)
        addr    <=  addr_reg;

//cs_n:片选信号
always@(posedge sys_clk or  negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        cs_n    <=  1'b1;
    else    if(pi_flag == 1'b1)
        cs_n    <=  1'b0;
    else    if((cnt_byte == 4'd2) && (cnt_clk == 5'd31) && (state == WR_EN))
        cs_n    <=  1'b1;
    else    if((cnt_byte == 4'd3) && (cnt_clk == 5'd31) && (state == DELAY))
        cs_n    <=  1'b0;
    else    if((cnt_byte == 4'd10) && (cnt_clk == 5'd31) && (state == PP))
        cs_n    <=  1'b1;

//sck:输出串行时钟
always@(posedge sys_clk or  negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        sck <=  1'b0;
    else    if(cnt_sck == 2'd0)

```

```

        sck <= 1'b0;
    else    if(cnt_sck == 2'd2)
        sck <= 1'b1;

//cnt_bit:高低位对调, 控制 mosi 输出
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        cnt_bit <= 3'd0;
    else    if(cnt_sck == 2'd2)
        cnt_bit <= cnt_bit + 1'b1;

//state:两段式状态机第一段, 状态跳转
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        state <= IDLE;
    else
        case(state)
            IDLE:    if(pi_flag == 1'b1)
                state <= WR_EN;
            WR_EN:    if((cnt_byte == 4'd2) && (cnt_clk == 5'd31))
                state <= DELAY;
            DELAY:    if((cnt_byte == 4'd3) && (cnt_clk == 5'd31))
                state <= PP;
            PP:        if((cnt_byte == 4'd10) && (cnt_clk == 5'd31))
                state <= IDLE;
            default:    state <= IDLE;
        endcase

//mosi:两段式状态机第二段, 逻辑输出
always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        mosi <= 1'b0;
    else    if((state == WR_EN) && (cnt_byte == 4'd2))
        mosi <= 1'b0;
    else    if((state == PP) && (cnt_byte == 4'd10))
        mosi <= 1'b0;
    else    if((state == WR_EN) && (cnt_byte == 4'd1) && (cnt_sck == 5'd0))
        mosi <= WR_EN_INST[7 - cnt_bit];    //写使能指令
    else    if((state == PP) && (cnt_byte == 4'd5) && (cnt_sck == 5'd0))
        mosi <= PP_INST[7 - cnt_bit];        //扇区擦除指令
    else    if((state == PP) && (cnt_byte == 4'd6) && (cnt_sck == 5'd0))
        mosi <= addr[23 - cnt_bit];            //扇区地址
    else    if((state == PP) && (cnt_byte == 4'd7) && (cnt_sck == 5'd0))
        mosi <= addr[15 - cnt_bit];            //页地址

```

```

else    if((state == PP) && (cnt_byte == 4'd8) && (cnt_sck == 5'd0))
    mosi    <=  addr[7 - cnt_bit];           //字节地址
else    if((state == PP) && (cnt_byte == 4'd9) && (cnt_sck == 5'd0))
    mosi    <=  pi_data[7 - cnt_bit];       //写入数据

```

endmodule

SignalTap 仿真波形:



图 4 往 flash 写入 0000_0001 的波形图

读取 flash 编码及运用 in-sysytem sources and probes 的代码:

```

module top(
    input clk,
    input ret,

    input wire miso, //读出 flash 数据

    output wire sck, //串行时钟
    output wire cs_n, //片选信号
    output wire mosi //主输出从输入数据

);
wire [7:0] data_1;
wire key;
wire tx_flag;
reg [7:0] data_2;
//assign data_2=(tx_falg==1)?data_1
always@(posedge clk)
if(tx_flag==1)
data_2<=data_1;
else
data_2<=data_2;

key_moni key_moni (
    .clk(clk),
    .ret(ret),
    .key( key)
);

flash_read_ctrl flash_read_ctrl(

```

```

        .sys_clk( clk)      ,    //系统时钟, 频率 50MHz
        .sys_rst_n( ret)    ,    //复位信号,低电平有效
        .key( key)         ,    //按键输入信号
        .miso( miso)       ,    //读出 flash 数据

        .sck( sck)         ,    //串行时钟
        .cs_n( cs_n)       ,    //片选信号
        .mosi( mosi)       ,    //主输出从输入数据
        .tx_flag(tx_flag ) ,    //输出数据标志信号
        .tx_data( data_1)   //输出数据
    );
    probe1 u0(
        .probe(data_2));
endmodule

```

```

module key_moni(
input clk,
input ret,
output reg  key
);
reg [9:0] count_1;
reg [1:0] count_2;

reg [2:0] state;
parameter idle=3'b001;
parameter start_1=3'b010;
parameter start_2=3'b100;

```

```

always@(posedge clk or negedge ret)
if(!ret)
    count_1<=0;
else if(count_1=='d100)
    count_1<=0;
else if(state==start_2)
    count_1<=count_1;
else
    count_1<=count_1+1;

```

```

always@(posedge clk or negedge ret)
if(!ret)
    count_2<=0;
else if(count_1=='d100)

```

```
count_2<=count_2+1;
```

```
always@(posedge clk or negedge ret)
if(!ret)
state<=idle;
else
case(state)
idle:if(count_1=='d96&&count_2==2'd2)
state<=start_1;
start_1:if(count_1=='d97&&count_2==2'd2)
state<=start_2;
start_2:
state<=start_2;
default:state<=idle;
endcase
```

```
always@(posedge clk or negedge ret)
if(!ret)
key<=0;
else if(state==start_1)
key<=1;
else
key<=0;
```

```
endmodule
```

```
module flash_read_ctrl(
```

```
    input  wire      sys_clk      , //系统时钟, 频率 50MHz
    input  wire      sys_rst_n   , //复位信号,低电平有效
    input  wire      key         , //按键输入信号
    input  wire      miso        , //读出 flash 数据

    output reg       sck         , //串行时钟
    output reg       cs_n        , //片选信号
    output reg       mosi        , //主输出从输入数据
    output wire      tx_flag     , //输出数据标志信号
    output wire [7:0] tx_data    , //输出数据
```

```
);
```

```
/* synthesis noprunce */
```

```

//*****
//***** Parameter and Internal Signal *****
//*****

//parameter define
parameter IDLE      = 3'b001 , //初始状态
           READ      = 3'b010 ; //数据读状态
           //SEND     = 3'b100 ; //数据发送状态

parameter READ_INST = 8'b0000_0011; //读指令
parameter NUM_DATA   = 1'd1 ; //读出数据个数
parameter SECTOR_ADDR = 8'b0000_0000, //扇区地址
           PAGE_ADDR  = 8'b0000_0100, //页地址
           BYTE_ADDR  = 8'b0010_0101; //字节地址
//parameter CNT_WAIT_MAX= 16'd6_00_00;

//wire define
//wire [14:0] fifo_data_num ; //fifo 内数据个数
//reg define
reg [4:0] cnt_clk ; //系统时钟计数器
reg [2:0] state ; //状态机状态
reg [15:0] cnt_byte ; //字节计数器
reg [1:0] cnt_sck ; //串行时钟计数器
reg [2:0] cnt_bit ; //比特计数器
reg miso_flag ; //miso 提取标志信号
reg [7:0] data ; //拼接数据
reg po_flag_reg ; //输出数据标志信号
reg po_flag ; //输出数据
reg [7:0] po_data ; //输出数据

reg [15:0] cnt_wait ; //等待计数器

//*****
//***** Main Code *****
//*****
//cnt_clk: 系统时钟计数器，用以记录单个字节

assign tx_data=po_data;
assign tx_flag=po_flag;

always@(posedge sys_clk or negedge sys_rst_n)
    if(sys_rst_n == 1'b0)

```



```

        cnt_clk <= 5'd0;
    else    if(state == READ)
        cnt_clk <= cnt_clk + 1'b1;

//cnt_byte: 记录输出字节个数和等待时间
always@(posedge sys_clk or  negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        cnt_byte <= 16'd0;
    else    if((cnt_clk == 5'd31) && (cnt_byte == NUM_DATA + 16'd3))
        cnt_byte <= 16'd0;
    else    if(cnt_clk == 5'd31)
        cnt_byte <= cnt_byte + 1'b1;

//cnt_sck: 串行时钟计数器, 用以生成串行时钟
always@(posedge sys_clk or  negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        cnt_sck <= 2'd0;
    else    if(state == READ)
        cnt_sck <= cnt_sck + 1'b1;

//cs_n: 片选信号
always@(posedge sys_clk or  negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        cs_n <= 1'b1;
    else    if(key == 1'b1)
        cs_n <= 1'b0;
    else    if((cnt_byte == NUM_DATA + 16'd3) && (cnt_clk == 5'd31) && (state ==
READ))
        cs_n <= 1'b1;

//sck: 输出串行时钟
always@(posedge sys_clk or  negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        sck <= 1'b0;
    else    if(cnt_sck == 2'd0)
        sck <= 1'b0;
    else    if(cnt_sck == 2'd2)
        sck <= 1'b1;

//cnt_bit: 高低位对调, 控制 mosi 输出
always@(posedge sys_clk or  negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        cnt_bit <= 3'd0;
    else    if(cnt_sck == 2'd2)

```

```
cnt_bit <= cnt_bit + 1'b1;
```

//state: 两段式状态机第一段，状态跳转

```
always@(posedge sys_clk or negedge sys_rst_n)
  if(sys_rst_n == 1'b0)
    state <= IDLE;
  else
    case(state)
      IDLE: if(key == 1'b1)
              state <= READ;
      READ: if((cnt_byte == NUM_DATA + 16'd3) && (cnt_clk == 5'd31))
              //state <= SEND;
      //SEND: if((read_data_num == NUM_DATA)
              // /*&& ((cnt_wait == (CNT_WAIT_MAX - 1'b1))))*/
              state <= IDLE;
      default: state <= IDLE;
    endcase
```

//mosi: 两段式状态机第二段，逻辑输出

```
always@(posedge sys_clk or negedge sys_rst_n)
  if(sys_rst_n == 1'b0)
    mosi <= 1'b0;
  else if((state == READ) && (cnt_byte >= 16'd4))
    mosi <= 1'b0;
  else if((state == READ) && (cnt_byte == 16'd0) && (cnt_sck == 2'd0))
    mosi <= READ_INST[7 - cnt_bit]; //读指令
  else if((state == READ) && (cnt_byte == 16'd1) && (cnt_sck == 2'd0))
    mosi <= SECTOR_ADDR[7 - cnt_bit]; //扇区地址
  else if((state == READ) && (cnt_byte == 16'd2) && (cnt_sck == 2'd0))
    mosi <= PAGE_ADDR[7 - cnt_bit]; //页地址
  else if((state == READ) && (cnt_byte == 16'd3) && (cnt_sck == 2'd0))
    mosi <= BYTE_ADDR[7 - cnt_bit]; //字节地址
```

//miso_flag: miso 提取标志信号

```
always@(posedge sys_clk or negedge sys_rst_n)
  if(sys_rst_n == 1'b0)
    miso_flag <= 1'b0;
  else if((cnt_byte >= 16'd4) && (cnt_sck == 2'd1))
    miso_flag <= 1'b1;
  else
    miso_flag <= 1'b0;
```

//data: 拼接数据

```
always@(posedge sys_clk or negedge sys_rst_n)
```

```

    if(sys_rst_n == 1'b0)
        data    <= 8'd0;
    else    if(miso_flag == 1'b1)
        data    <= {data[6:0],miso};

//po_flag_reg:输出数据标志信号
always@(posedge sys_clk or  negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        po_flag_reg <= 1'b0;
    else    if((cnt_bit == 3'd7) && (miso_flag == 1'b1))
        po_flag_reg <= 1'b1;
    else
        po_flag_reg <= 1'b0;

//po_flag:输出数据标志信号
always@(posedge sys_clk or  negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        po_flag <= 1'b0;
    else
        po_flag <= po_flag_reg;

//po_data:输出数据
always@(posedge sys_clk or  negedge sys_rst_n)
    if(sys_rst_n == 1'b0)
        po_data <= 8'd0;
    else    if(po_flag_reg == 1'b1)
        po_data <= data;
    else
        po_data <= po_data;
endmodule

```

SignalTap 仿真波形:

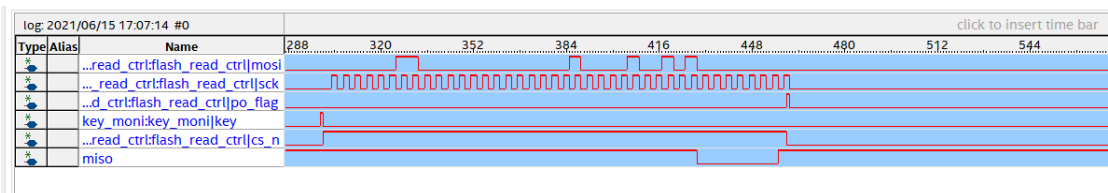


图 5 读取 flash 里面数据的波形图

6.5 FPGA 硬件调试

运行擦除 flash 的 bat 文件运行结果:

```

C:\Windows\system32\cmd.exe
C:\Users\25339\Desktop\duxie_flash>C:\intelFPGA\18.1\quartus\bin64\quartus_pgm.exe -m jtag -c USB-Blaster[USB-0] -o p;el.sof
Info: *****
Info: Running Quartus Prime Programmer
Info: Version 18.1.0 Build 625 09/12/2018 SJ Standard Edition
Info: Copyright (C) 2018 Intel Corporation. All rights reserved.
Info: Your use of Intel Corporation's design tools, logic functions
Info: and other software and tools, and its AMPP partner logic
Info: functions, and any output files from any of the foregoing
Info: (including device programming or simulation files), and any
Info: associated documentation or information are expressly subject
Info: to the terms and conditions of the Intel Program License
Info: Subscription Agreement, the Intel Quartus Prime License Agreement,
Info: the Intel FPGA IP License Agreement, or other applicable license
Info: agreement, including, without limitation, that your use is for
Info: the sole purpose of programming logic devices manufactured by
Info: Intel and sold by Intel or its authorized distributors. Please
Info: refer to the applicable agreement for further details.
Info: Processing started: Tue Jun 15 17:04:19 2021
Info: Command: quartus_pgm -m jtag -c USB-Blaster[USB-0] -o p;el.sof
Info (213045): Using programming cable "USB-Blaster [USB-0]"
Info (213011): Using programming file el.sof with checksum 0x000E578D for device 10C006YU25601
Info (209060): Started Programmer operation at Tue Jun 15 17:04:19 2021
Info (209016): Configuring device index 1
Info (209017): Device 1 contains JTAG ID code 0x020F10DD
Info (209007): Configuration succeeded -- 1 device(s) configured
Info (209011): Successfully performed operation(s)
Info (209061): Ended Programmer operation at Tue Jun 15 17:04:20 2021
Info: Quartus Prime Programmer was successful. 0 errors, 0 warnings
Info: Peak virtual memory: 4417 megabytes
Info: Processing ended: Tue Jun 15 17:04:20 2021
Info: Elapsed time: 00:00:01
Info: Total CPU time (on all processors): 00:00:00
C:\Users\25339\Desktop\duxie_flash>pause
请按任意键继续. . .

```

图 6 批处理下载全擦除 flash 的运行结果

运行往 flash 里面写入相应的编码的 bat 文件运行结果:

```

C:\Windows\system32\cmd.exe
Info: Your use of Intel Corporation's design tools, logic functions
Info: and other software and tools, and its AMPP partner logic
Info: functions, and any output files from any of the foregoing
Info: (including device programming or simulation files), and any
Info: associated documentation or information are expressly subject
Info: to the terms and conditions of the Intel Program License
Info: Subscription Agreement, the Intel Quartus Prime License Agreement,
Info: the Intel FPGA IP License Agreement, or other applicable license
Info: agreement, including, without limitation, that your use is for
Info: the sole purpose of programming logic devices manufactured by
Info: Intel and sold by Intel or its authorized distributors. Please
Info: refer to the applicable agreement for further details.
Info: Processing started: Tue Jun 15 17:06:46 2021
Info: Command: quartus_pgm -m jtag -c USB-Blaster[USB-0] -o p;big_write.sof
Info (213045): Using programming cable "USB-Blaster [USB-0]"
Info (213011): Using programming file big_write.sof with checksum 0x000ECSD1 for device 10C006YU25601
Info (209060): Started Programmer operation at Tue Jun 15 17:06:46 2021
Info (209016): Configuring device index 1
Info (209017): Device 1 contains JTAG ID code 0x020F10DD
Info (209007): Configuration succeeded -- 1 device(s) configured
Info (209011): Successfully performed operation(s)
Info (209061): Ended Programmer operation at Tue Jun 15 17:06:47 2021
Info: Quartus Prime Programmer was successful. 0 errors, 0 warnings
Info: Peak virtual memory: 4417 megabytes
Info: Processing ended: Tue Jun 15 17:06:47 2021
Info: Elapsed time: 00:00:01
Info: Total CPU time (on all processors): 00:00:00
C:\Users\25339\Desktop\duxie_flash>pause
请按任意键继续. . .

```

图 7 批处理下载往 flash 下载 0000_0001 的运行结果

运行 exe 程序生成的 txt 文件:

```

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
00000001: USB-Blaster [USB-0]

```

图 8 运行 exe 生成的 txt 文件内容

可以看到 USB-Blaster [USB-0]这个在 PC 端设备管理器中设备符号与具体的 FPGA 相匹配, 同理当多块开发板接入 PC 端时可以得到下面的 txt 文件:

```
00000003:USB-Blaster [USB-0]
00000001:USB-Blaster [USB-1]
00000002:USB-Blaster [USB-2]
00000005:USB-Blaster [USB-3]
00000007:USB-Blaster [USB-4]
00000009:USB-Blaster [USB-5]
00000004:USB-Blaster [USB-6]
00000008:USB-Blaster [USB-7]
0000000A:USB-Blaster [USB-8]
00000006:USB-Blaster [USB-9]
```

图 9 10 块开发板 exe 运行生成的 txt 文件内容

6.6 参考文献

<https://blog.csdn.net/wuzhikaidetb/article/details/122358545>

<http://blog.chinaaet.com/ilove314/p/31841>

7. 基于 FPGA 的 IIR 滤波器设计

7.1 算法原理

无限冲激响应（IIR: Infinite Impulse Response）是一种适用于许多线性时不变系统的属性，这些系统的特征具有一个冲激响应 $h(t)$ ，该冲激响应 $h(t)$ 不会在特定点上完全变为零，而是无限期地持续，这与有限冲激响应（FIR: Finite Impulse Response）系统形成对比。在对信号进行处理时，通常先通过采样将连续的时间信号转换为离散的时间序列，而 IIR 滤波器的时域差分方程如(式 1)所示，其中时间序列 $x(n)$ 为滤波器的输入，时间序列 $y(n)$ 为滤波器的输出，第一个样本点在 $n=0$ 处。可以从时域方程中看书，IIR 滤波器使用了一些输出作为反馈再次输入，这使得 IIR 滤波器的算法成为了一个递归函数。

$$y(n) = \sum_{k=0}^N a(k)x(n-k) + \sum_{j=0}^P b(j)y(n-j) \quad (\text{式 1})$$

通过 Z 变换将 IIR 滤波器的时域差分方程变换到频域，可以得到如(式 2)所示的系统函数 $H(z)$ 。

$$H(z) = \frac{\sum_{k=0}^N a_k z^{-k}}{1 - \sum_{j=0}^P b_j z^{-j}} \quad (\text{式 2})$$

为了减少资源的消耗，也可以由此设计出如图 1 所示的直接 II 型 IIR 滤波器的结构图。图中左侧为极点部分，对应系统函数的分母，右侧为零点部分，对应系统函数的分子。因此 IIR 滤波器的实质就是两个 FIR 的叠加，也就是零点部分的模块进行一个 N 位的移位，极点部分的模块进行一个 N-1 位的移位。

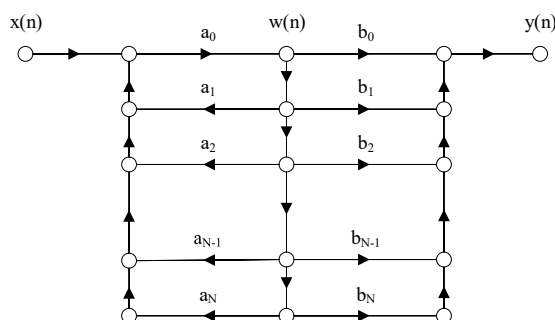


图 1 直接 II 型 IIR 滤波器结构图

7.2 Matlab 验证代码

使用 Matlab 编写生成频率为 200Hz 与 2000Hz 混叠的信号用于验证 IIR 滤波器设计的滤波效果，并将信号保存到文本文档内便于存入 FPGA 开发板的 ROM 中。产生输入信号的 Matlab 代码如下：

```
f1 = 200;%频率 1
```

```

f2 = 2000;%频率 2
Fs = 5000;%采样频率
N = 10;
End = 200;
t = 0:1/Fs:End-1/Fs;
c1 = 2*pi*f1*t;
c2 = 2*pi*f2*t;
s1 = sin(c1);
s2 = 0.5*sin(c2);
s = s1+s2;
s = s/max(abs(s));
Q_s = round(s*(2^(N-1) - 1));
fid = fopen('sin_ten.txt','w');
fprintf(fid, '%10d\r\n',Q_s);
fprintf(fid, ';');
fclose(fid);
fid = fopen('sin.txt','w');
for k=1:length(Q_s)%二进制变换
B_s=dec2bin(Q_s(k)+((Q_s(k))<0)*2^N,N);
for j=1:N
if B_s(j)=='1'
tb=1;
else
tb=0;
end
fprintf(fid,'%d',tb);
end
fprintf(fid,'\r\n');
end
fprintf(fid, ';');
fclose(fid);

```

使用 Matlab 编写设计一个 7 阶、采样频率为 5000Hz、截止频率为 1000Hz、阻带衰减 70dB 的直接 II 型切比雪夫 IIR 低通滤波器，求得它的各个零点和极点的系数并进行量化，代码如下：

```

clear ;
clc;
f_sample = 5000;%采样频率
f_stop = 1000;%截止频率
f_falling = 70;%db 衰减
num_filter = 7;%设置滤波器阶数
[Num, Den] = cheby2(num_filter, f_falling, f_stop*2/f_sample, "low");
max_den_num = max(max(abs(Num), abs(Den)));
mid_compute = log2(max_den_num/Den(1));
Number = floor(mid_compute);

```

```

if Number < log2(max_den_num/Den(1))
    Number = Number + 1;
end
Number = 2^Number;
Q_den = round(Den/Number * (2^13-1));
Q_num = round(Num/Number * (2^13-1));
fprintf("Num = \n")
for i = 1:1:(num_filter+1)
    fprintf("\t %f",Num(i));
end
fprintf("\n");
fprintf("Den = \n")
for i = 1:1:(num_filter+1)
    fprintf("\t %f",Den(i));
end
fprintf("\n");
fprintf("Q_num = \n")
for i = 1:1:(num_filter+1)
    fprintf("\t %f",Q_num(i));
end
fprintf("\n");
fprintf("Q_den = \n")
for i = 1:1:(num_filter+1)
    fprintf("\t %f",Q_den(i));
end
fprintf("\n");

```

也可以使用如图 2 所示的 Digital Filter Design 模块直接进行设计，分别输入所需的响应类型、设计方法、阶数、截止频率、阻带衰减等参数即可。

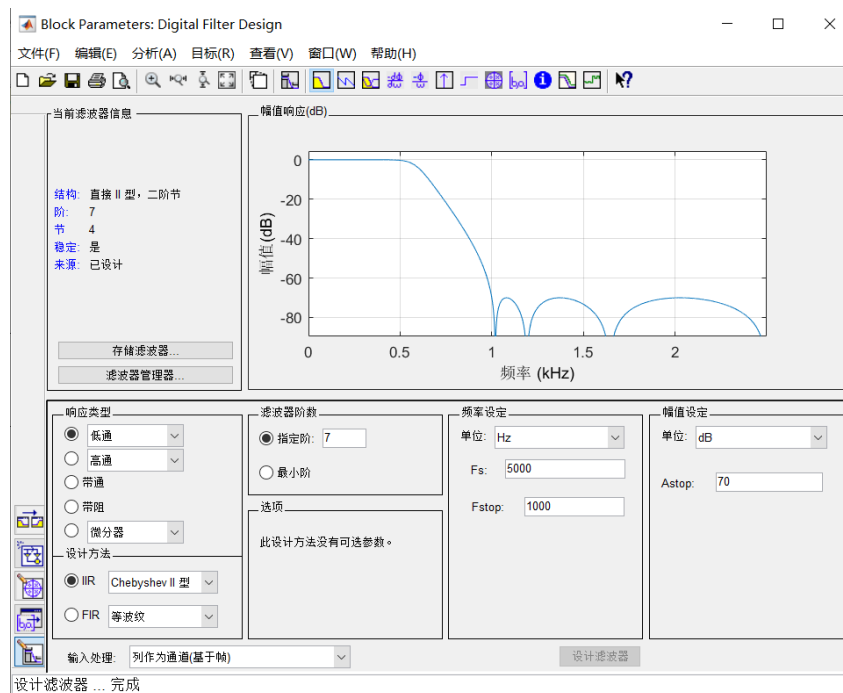


图 2 使用设计工具进行 IIR 滤波器设计

设计完成的 IIR 低通滤波器的幅频响应如图 3 所示。

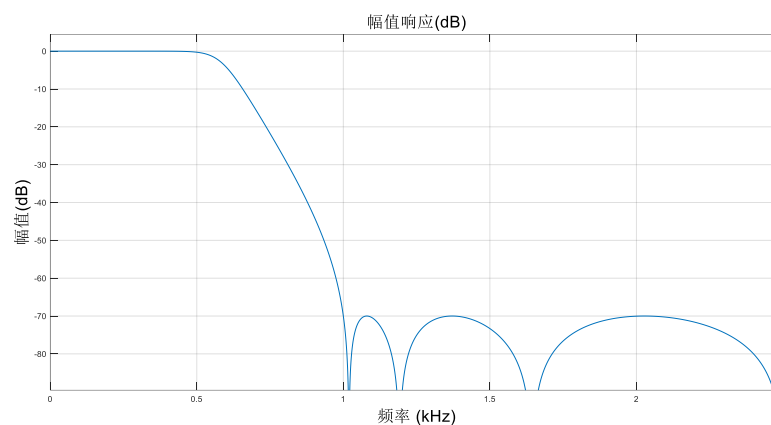


图 3 IIR 滤波器的幅频响应

为了便于在 Matlab 中进行验证, 使用了 Simulink 工具搭建了如图 4 所示的结构进行仿真。输入为一个幅值为 1V 频率为 200Hz 的正弦信号和一个幅值为 0.5V 频率为 2000Hz 的正弦信号的叠加, 叠加后的信号输入设计好的 IIR 低通滤波器, 最后将滤波器的输入和输出都接到示波器上观察结果。

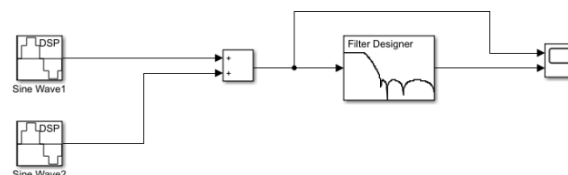


图 4 IIR 滤波器的幅频响应

Simulink 内仿真的结果如图 5 所示, 可以从示波器上看到图中上方在 200Hz 上叠加的

2000Hz 的高频信号已经被阻带频率为 1000Hz 的 IIR 低通滤波器滤除，图中下方为剩下的频率为 200Hz 信号。

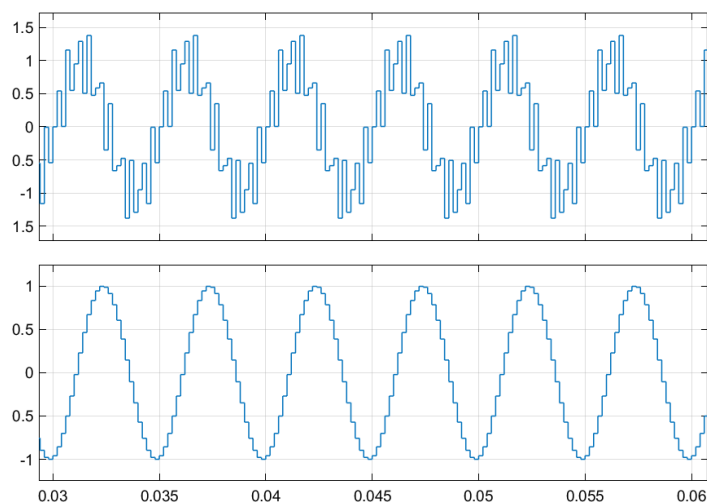


图 5 IIR 滤波器的输入和输出

7.3 FPGA 实现架构

FPGA 设计的整体架构如图 6 所示，本次设计将 IIR 滤波器分为零点模块和极点模块两个模块，零点模块用于计算各个零点与系数相乘的结果，极点模块用于计算各个极点与系数相乘的结果，零点模块的输出减去极点模块的输出再除以第一个极点量化后的系数就可以求得 $y(n)$ 。

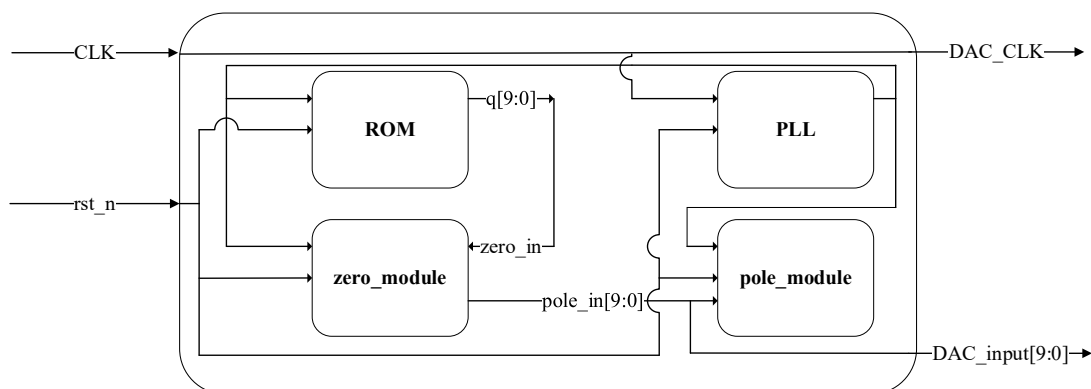


图 6 IIR 滤波器设计框图

为了实现结果的展示，将 Matlab 生成好的不同频率叠加信号保存在调用的 rom 的 ip 核内，将从 ip 核内按顺序读取每个点的信号幅值座位零点模块的输入，将极点模块的输入作为顶层 IIR 滤波器的输出。最后通过锁相环分频输出特定频率的时钟信号作为零点模块、极点模块、ROM 的 ip 核以及 DAC 模块的时钟，将 10 位滤波后的结果转换成模拟信号输出。

7.4 FPGA 实现代码

根据上文对直接 II 型 IIR 滤波器的算法分析，在 FPGA 实现时可以将整个算法分为零点模块和极点模块两个部分，零点模块代码如下：

```
`timescale 1ns/1ps
module zero_module
```

```

#( parameter INPUT_LENGTH = 10)
(
    input clk
    input rst_n
    input signed [INPUT_LENGTH-1:0] zero_input
    output signed [INPUT_LENGTH+9+1:0] zero_output
);

parameter U_DLY = 1;//延时

reg signed [INPUT_LENGTH-1:0] data_reg [6:0];

always@(posedge clk or negedge rst_n) begin
    if(rst_n == 1'b0) begin
        data_reg[0] <= #U_DLY 12'b0 ;
        data_reg[1] <= #U_DLY 12'b0 ;
        data_reg[2] <= #U_DLY 12'b0 ;
        data_reg[3] <= #U_DLY 12'b0 ;
        data_reg[4] <= #U_DLY 12'b0 ;
        data_reg[5] <= #U_DLY 12'b0 ;
        data_reg[6] <= #U_DLY 12'b0 ;
    end
    else
    begin
        data_reg[6] <= #U_DLY data_reg[5] ;
        data_reg[5] <= #U_DLY data_reg[4] ;
        data_reg[4] <= #U_DLY data_reg[3] ;
        data_reg[3] <= #U_DLY data_reg[2] ;
        data_reg[2] <= #U_DLY data_reg[1] ;
        data_reg[1] <= #U_DLY data_reg[0] ;
        data_reg[0] <= #U_DLY zero_input ;
    end
end

wire signed [INPUT_LENGTH:0] add_reg [3:0];//加符号位 1 位 11 位

assign add_reg[0] = {zero_input[9], zero_input} + {data_reg[6][9], data_reg[6]} ;
assign add_reg[1] = {data_reg[0][9], data_reg[0]} + {data_reg[5][9], data_reg[5]} ;
assign add_reg[2] = {data_reg[1][9], data_reg[1]} + {data_reg[4][9], data_reg[4]} ;
assign add_reg[3] = {data_reg[2][9], data_reg[2]} + {data_reg[3][9], data_reg[3]} ;

wire signed [INPUT_LENGTH + 9:0] mult_reg [3:0];//11 位 x9 位 20 位

wire signed [8:0] coe_0 = 'd3;//a0

```

```

wire signed [8:0] coe_1 = 'd4;//a1
wire signed [8:0] coe_2 = 'd8;//a2
wire signed [8:0] coe_3 = 'd9;//a3

assign mult_reg[0] = coe_0 * add_reg[0];
assign mult_reg[1] = coe_1 * add_reg[1];
assign mult_reg[2] = coe_2 * add_reg[2];
assign mult_reg[3] = coe_3 * add_reg[3];

assign zero_output = {mult_reg[0][19], mult_reg[0]}
                    + {mult_reg[1][19], mult_reg[1]}
                    + {mult_reg[2][19], mult_reg[2]}
                    + {mult_reg[3][19], mult_reg[3]};

endmodule

```

零点模块用于将各个零点和由 Matlab 生成的各零点系数进行卷积，求得差分方程 $y(n)$ 中前半部分的卷积。由于 7 阶 IIR 滤波器各零点系数存在对称的规律，所以零点模块中只需要使用 4 个加法器和 4 个乘法器即可计算出 8 个零点的卷积。极点模块用于计算各个极点和由 Matlab 生成的各个极点系数的卷积并求和，求得差分方程 $y(n)$ 中后半部分的卷积，极点模块的代码如下：

```

`timescale 1ns/1ps
module pole_module
#( parameter OUTPUT_LENGTH = 10)
(
    input clk
    ,
    input rst_n
    ,
    input signed [OUTPUT_LENGTH - 1:0] pole_input
    ,
    output signed [OUTPUT_LENGTH + 16:0] pole_output
);

parameter U_DLY = 1;//延时

reg signed [OUTPUT_LENGTH - 1:0] data_reg [6:0];

always@(posedge clk or negedge rst_n)
    if(rst_n == 1'b0) begin
        data_reg[0] <= #U_DLY 12'd0 ;
        data_reg[1] <= #U_DLY 12'd0 ;
        data_reg[2] <= #U_DLY 12'd0 ;
        data_reg[3] <= #U_DLY 12'd0 ;
        data_reg[4] <= #U_DLY 12'd0 ;
        data_reg[5] <= #U_DLY 12'd0 ;
        data_reg[6] <= #U_DLY 12'd0 ;
    end

```

```

end
else begin
    data_reg[6] <= #U_DLY data_reg[5]    ;
    data_reg[5] <= #U_DLY data_reg[4]    ;
    data_reg[4] <= #U_DLY data_reg[3]    ;
    data_reg[3] <= #U_DLY data_reg[2]    ;
    data_reg[2] <= #U_DLY data_reg[1]    ;
    data_reg[1] <= #U_DLY data_reg[0]    ;
    data_reg[0] <= #U_DLY pole_input     ;
end

wire signed [OUTPUT_LENGTH + 16 :0] mult_pole [6:0]; //17 位 x10 位 27 位

wire signed [16:0] coe_0 = -'sd3627; //b0
wire signed [16:0] coe_1 = 'sd6046; //b1
wire signed [16:0] coe_2 = -'sd5911; //b2
wire signed [16:0] coe_3 = 'sd3623; //b3
wire signed [16:0] coe_4 = -'sd1378; //b4
wire signed [16:0] coe_5 = 'sd300; //b5
wire signed [16:0] coe_6 = -'sd29; //b6

assign mult_pole[0] = coe_0 * data_reg[0];
assign mult_pole[1] = coe_1 * data_reg[1];
assign mult_pole[2] = coe_2 * data_reg[2];
assign mult_pole[3] = coe_3 * data_reg[3];
assign mult_pole[4] = coe_4 * data_reg[4];
assign mult_pole[5] = coe_5 * data_reg[5];
assign mult_pole[6] = coe_6 * data_reg[6];

assign pole_output = {mult_pole[0] +
                      mult_pole[1] +
                      mult_pole[2] +
                      mult_pole[3] +
                      mult_pole[4] +
                      mult_pole[5] +
                      mult_pole[6] };

endmodule

```

最后编写顶层模块，将 Matlab 生成的 200Hz 和 2000Hz 叠加的正弦信号保存至 ROM 的 ip 核内作为零点模块的输入，将零点模块的输出作为极点模块的输入，将极点模块的输入作为整个 IIR 滤波器的输出，代码如下：

```

`timescale 1ns/1ps
module iir_top(
    input clk
    input rst_n

```

```

        output DAC_clk,
        output reg [11:0] address_sig,
        output reg signed [9:0] IIR_output
    );

    parameter U_DLY = 1;//延时
    wire clk_cal;
    ip_pll ip_pll_inst (
        .inclk0 ( clk ),
        .c0 ( clk_cal )
    );

    assign DAC_clk = clk;

    wire signed [ 9:0] IIR_input ;
    wire signed [20:0] zero_out  ;
    wire signed [ 9:0] pole_in   ;
    wire signed [26:0] pole_out  ;
    wire signed [17:0] pole_res  ;

    always@(posedge clk_cal or negedge rst_n)begin
        if(rst_n == 1'b0) address_sig <= 1'b0;
        else if(address_sig == 12'd24) address_sig <= 1'b0;
        else address_sig <= address_sig + 1'b1;
    end

    ip_rom ip_rom_inst (
        .address ( address_sig ),
        .clock ( clk_cal ),
        .q ( IIR_input )
    );

    always@(posedge clk_cal or negedge rst_n)
    begin
        if(rst_n == 1'b0)begin
            IIR_output <= #U_DLY 12'b0;    //复位信号，下降沿有效
        end
        else begin
            IIR_output <= #U_DLY {~pole_res[9],pole_res[8:0]};
        end
    end

    zero_module zero
    (
        .rst_n (rst_n)
        ,
        .clk (clk_cal)
        ,

```

```

.zero_input (IIR_input)      ,
.zero_output(zero_out)
);

pole_module pole
(
.clk (clk_cal)                ,
.rst_n (rst_n)                ,
.pole_input (pole_in)         ,
.pole_output(pole_out)
);

wire signed [26:0] sub_module;

assign sub_module = {{{6{zero_out[20]}}, zero_out} - pole_out };//零点的输出与极点的输出进行相减
assign pole_res = {sub_module[26:10]} ;//移位除法除以 1024
assign pole_in = pole_res[9:0];
endmodule

```

其中，极点模块的输入应等于零点模块的输出减去极点模块的输出并处以 $y(n)$ 这个极点前的系数。全部代码编写完成后，可以编译得到如图 7 所示的 RTL 视图。

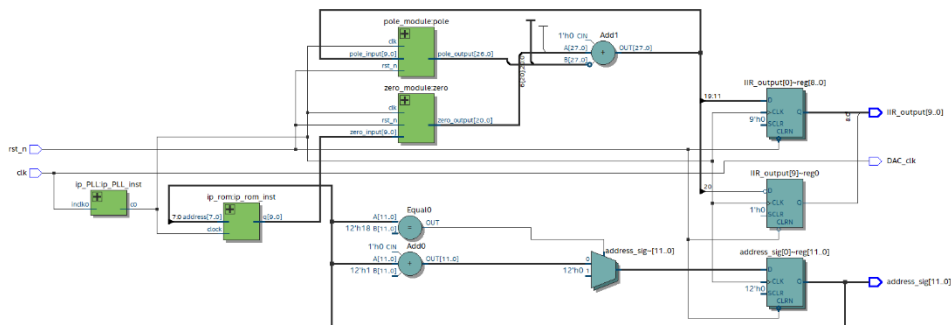


图 7 RTL 视图

最后，编写 testbench 代码，将 Matlab 生成的叠加信号作为输入，为了便于仿真结果的验证，将 `clk_5K` 的频率设置为和 `clk` 相同的 50Mhz，即 IIR 滤波器输出信号的频率为 50Mhz，进行仿真后输出如图 8 所示，可以清楚的看到在经过 IIR 滤波器后原本叠加在 20Hz 信号上的 2000Hz 的分量被低通滤波器滤除。

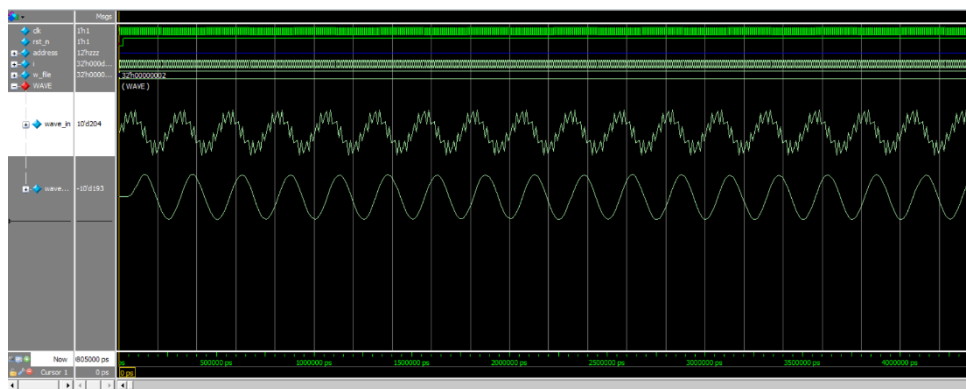


图 8 Modelsim 仿真波形图

7.5 FPGA 硬件调试

在编写完成全部代码并通过仿真后，将程序下载到 FPGA 开发板内，将 DAC900 模块的 10 位输入、VCC5V、GND 和 CLK 引脚连接到开发板上。用示波器可以看到如图 9 所示的波形，输出信号的频率满足滤波后的 200Hz，但波峰和波谷处出现了波形翻转的问题。

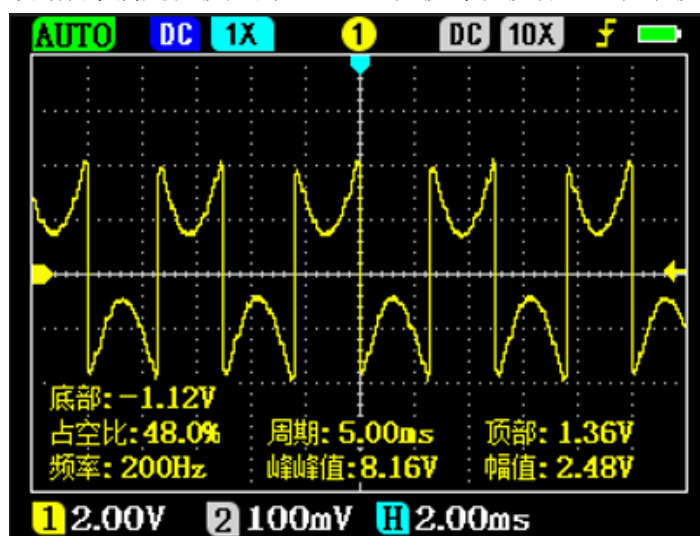


图 9 改正前的示波器波形图

在对代码进行检查后，发现是由于开发板输出和 DAC 输入数据格式不匹配造成的。IIR_out 输出为 10 位补码，而 DAC 无法识别补码只能使用二进制原码进行输入，所以需要更改输出的数据格式。首先将输出数据的最高位取反，再保持低位不变，这样处理过后的数据相当于把原来翻转的波峰向下平移，把原来翻转的波谷向上平移，修改后 DAC 输出的波形如图 10 所示，为 200Hz 正弦波。

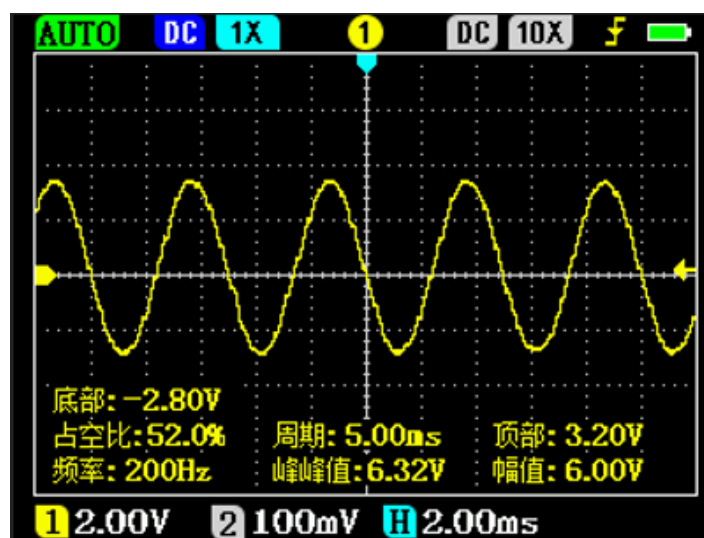


图 10 改正后的示波器波形图

7.6 参考文献

<https://blog.csdn.net/FPGADesigner/article/details/80652480>

8. 基于 FPGA 的 LZW 压缩算法优化实现

8.1 算法原理

LZW 压缩算法是一种新颖的压缩方法，由 Lempel-Ziv-Welch 三人共同创造，用他们的名字命名。它采用了一种先进的串表压缩，将每个第一次出现的串放在一个串表中，用一个数字来表示串，压缩文件只存贮数字，则不存贮串，从而使图象文件的压缩效率得到较大的提高。奇妙的是，不管是在压缩还是在解压缩的过程中都能正确的建立这个串表，压缩或解压缩完成后，这个串表又被丢弃，具体编码过程大致如下图所示。

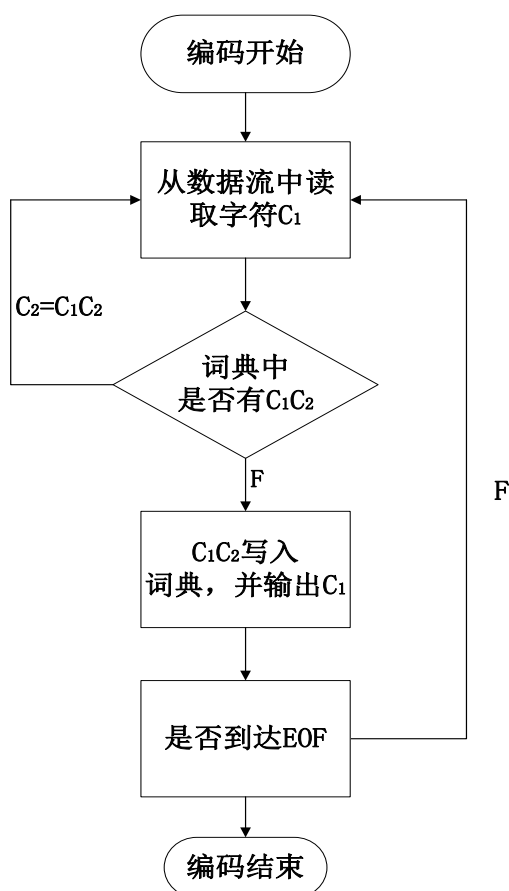


图 1 LZW 算法编码流程图

LZW 算法中，首先建立一个字符串表，把每一个第一次出现的字符串放入串表中，并用一个数字来表示，这个数字与此字符串在串表中的位置有关，并将这个数字存入压缩文件中，如果这个字符串再次出现时，即可用表示它的数字来代替，并将这个数字存入文件中。压缩完成后将串表丢弃。如“print”字符串，如果在压缩时用 266 表示，只要再次出现，均用 266 表示，并将“print”字符串存入串表中，在图像解码时遇到数字 266，即可从串表中查出 266 所代表的字符串“print”，在解压缩时，串表可以根据压缩数据重新生成。

下面通过一个简单的样例，来说明一下 LZW 算法的大致编码过程。

首先，假设我们要编码的对象数据流为 255,24,54,255,24,255,255,24,5,123,

45,255,24,5,24,54……因为数据流的位宽为 8bit，故我们的清除标志位 CLEAR 取 $255+1=256$ ，结束标志位 END 取 $255+2=257$ ，初始化标号集为 0,1, ……，255, CLEAR, END，接下来就依次读取数据流中每个元素。

首先读入的数据是 255，已存在于标号集中，不做处理；

然后读入第二个数据，并与前一个数据相组合，形成前后缀关系的 Entry

(255,24)，该 Entry 未在标号集中找到，故将该 Entry 记录到标号集对应索引号为 258 的位置，然后输出前缀 255，并将后缀 24 移至前缀的位置；

③取下一个数据 54，与前缀 24 形成新的 Entry (24,54)，该 Entry 未存在于标号集中，故放于索引号为 259 的位置，并输出 24，再将 54 移至前缀位置；

④以此类推，直至处理到最后一个数据（若文件读取，则读取至 EOF）。

将上述步骤简单归纳，如下表所示

LZW 编码过程表

步骤	前缀	后缀	Entry	是否存在	输出	标号
1	/	255	(,255)	/	/	/
2	255	24	(255,24)	N	255	258
3	24	54	(24,54)	N	24	259
4	54	255	(54,255)	N	54	260
5	255	24	(255,24)	Y	/	/

8.2 C++验证代码

代码总共可分为三部分：一个头文件 Compress.h、两个 cpp 文件（Compress.cpp 和 main.cpp），具体如下：

8.2.1 Compress.h

```
#include <iostream>
#include <fstream>
using namespace std;

#define UINT unsigned int
#define UCHAR unsigned char
#define EMPTY 0xFFFFFFFF
#define BufferSize 300
#define NEW_TABLE 256
#define START 257
#define END 258
typedef struct TABLE //定义字典
{
    UINT precode; //前缀码
    UINT code; //当前字符
    bool used; //是否使用
} TABLE;
class compress
{
public:
    compress(string _infile,string _outfile);
    ~compress();
    void Intial(); //初始化
    UCHAR ReadChar();
```

```

bool CheckEOF();
UINT CheckTable(UINT pre_code, UINT code);
bool WriteChar(UINT code);
void WriteEnd();
bool CheckFile();

private:
    UCHAR *InBuffer;           //读入缓冲区 256
    UINT InBuffer_dex;        //缓冲区索引
    UINT InBuffer_Maxdex;      //缓冲区最大值
    UINT tempcode;
    UCHAR *OutBuffer;          //写入文件缓冲区 256
    UINT OutBuffer_dex;        //缓冲区索引
    ifstream infile;
    ofstream outfile;
    TABLE *table;            //字典
    UINT table_dex;
    UINT Count;
    UINT _con;
};

```

8.2.2 Compress.cpp:

```

#include "Compress.h"

compress::compress(string _infile, string _outfile)
{
    infile.open(_infile, ifstream::binary);
    outfile.open(_outfile, ofstream::binary);
    InBuffer_dex = InBuffer_Maxdex = BufferSize;
    InBuffer = (UCHAR *) (new char[BufferSize]); //读文件缓冲
    OutBuffer = (UCHAR *) (new char[BufferSize]); //写文件缓冲
    tempcode = EMPTY;
    OutBuffer_dex = 0;
    Count = 0;
    _con = 0;
    table = new TABLE[4096]; //字典初始化
}

compress::~~compress()
{
    infile.close();
    outfile.close();
    delete[] InBuffer;
    delete[] OutBuffer;
    delete[] table;
}

bool compress::CheckFile()
{

```

```

    if (!infile.is_open() || !outfile.is_open())
    {
        cout << "Files can't be opened!!" << endl;
        return false;
    }
    return true;
}

void compress::Intial()//重置字典数据
{
    for (UINT i = 0; i < 4096; i++)
    {
        if (i <= END)
        {
            table[i].code = i;
            table[i].used = true;
        }
        else
        {
            table[i].precode = EMPTY;
            table[i].code = EMPTY;
            table[i].used = false;
        }
    }
}

UINT compress::CheckTable(UINT pre_code,UINT code)//查字典
{
    for (UINT i = END+1; i < 4096; i++)
    {
        if (table[i].used == true)
        {
            if (table[i].precode == pre_code && table[i].code == code)
                return i;
        }
        else //加入到字典中
        {
            if (i >= 4095)
            {
                _con++;
                return NEW_TABLE;
            }
            else
            {
                table[i].precode = pre_code;

```

```

        table[i].code = code;
        table[i].used = true;
        return EMPTY;
    }
}
}
return EMPTY;
}
UCHAR compress::ReadChar() //读取 8 位数据
{
    Count++;
    if (Count % 10000 == 0)
    {
        cout << '.';
        if (Count % 100000 == 0)
        {
            cout << endl;
            Count = 0;
        }
    }
    if (InBuffer_dex >= BufferSize || InBuffer_dex >= InBuffer_Maxdex)
    {
        infile.read((char *)InBuffer, BufferSize);
        InBuffer_Maxdex = (unsigned int)infile.gcount();
        InBuffer_dex = 0;
    }
    return InBuffer[InBuffer_dex++];
}
bool compress::WriteChar(UINT code)
{
    if (tempcode == EMPTY)
    {
        OutBuffer[OutBuffer_dex++] = code & 0xff; //写入低 8 位
        tempcode = (code >> 4) & 0xF0;
    }
    else
    {
        OutBuffer[OutBuffer_dex++] = code & 0xff;
        OutBuffer[OutBuffer_dex++] = (tempcode | ((code >> 8) & 0xf)) & 0xff;
        tempcode = EMPTY;
    }
    if (OutBuffer_dex >= BufferSize)
    {
        outfile.write((char *)OutBuffer, OutBuffer_dex );
    }
}

```

```

        OutBuffer_dex = 0;
    }
    return true;
}
bool compress::CheckEOF()    //检测文件结束
{
    if ( infile.eof() && InBuffer_dex >= InBuffer_Maxdex)
        return true;
    else
        return false;
}
void compress::WriteEnd()    //写最后的若干字节
{
    if (tempcode != EMPTY)
    {
        WriteChar(END);
    }
    else
    {
        WriteChar(END);
        WriteChar(END);
    }
    if (OutBuffer_dex != 0)
        outfile.write((char *)OutBuffer, OutBuffer_dex );
    cout << "Compressed file has been created !" << endl;
    cout << "The number of table is  " << _con << endl;
}

```

8.2.3 main.cpp:

```

#include <iostream>
#include <fstream>
#include "Compress.h"
using namespace std;
#define IN   "C:\\Users\\WTse_\\Desktop\\test.txt"
#define OUT "C:\\Users\\WTse_\\Desktop\\case.txt"
int main()
{
    compress COM(IN, OUT);
    if (!COM.CheckFile())
        return 0;
    COM.Intial();    //压缩初始化
    COM.WriteChar(START); //写入开始标志
    UINT pre_code = COM.ReadChar(); //读取前缀
    UINT Count = 0;
    while (!COM.CheckEOF())

```

```

{
    UINT code = COM.ReadChar(); //读入新字符
    UINT temp = COM.CheckTable(pre_code, code); //查字典
    if (temp == EMPTY)
    {
        COM.WriteChar(pre_code); //写入前缀
        pre_code = code;         //更新前缀
    }
    else if (temp == NEW_TABLE) //字典已满，重新初始化字典
    {
        COM.WriteChar(pre_code);
        COM.WriteChar(NEW_TABLE);
        COM.Intial();
        pre_code = code;
    }
    else
    {
        pre_code = temp;
    }
}
COM.WriteChar(pre_code); //文件结束，输出前缀
COM.WriteEnd();
return 0;
}

```

8.3 FPGA 实现架构

本设计中，LZW 优化算法在 FPGA 上实现时，其内部功能主要分为以下几大模块：数据输入缓存模块、数据处理模块、时钟与控制模块、数据输出模块。具体架构关系如下图所示。

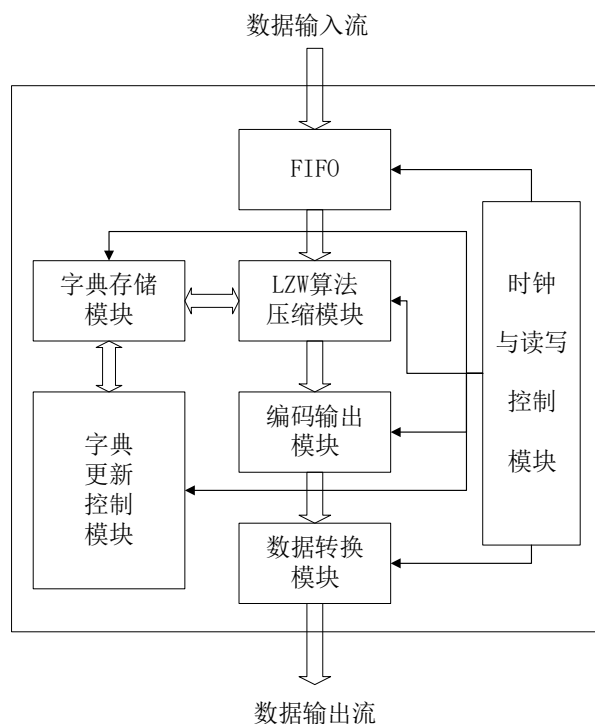


图 2 FPGA 实现架构框图

下面简单介绍一下核心模块：

LZW 算法数据处理部分：该部分主要包括核心算法模块、字典存储模块、字典更新控制模块。其中字典存储模块用来存储初始化和数据输入压缩过程中的所有字符串；字典更新模块主要完成在字典写满以后对字典内容和编码写入地址的更新；而核心算法模块主要完成字符串查找，比较字典相应内容是否匹配，同时将匹配到的字符串进行编码输出。在查找过程中采用并行查找方式，通过组合逻辑来实现，所有比较在一个时钟周期内完成。

8.4 FPGA 实现代码

由于篇幅所限，此处仅给出顶层 HDL 代码：

```
module Compression_LZW
(
    input clk,
    input rest,
    input data_end,

    output reg [7:0]compression_data,
    output reg compression_ok,
    output reg [8:0]pre_data,
    output reg [10:0]hash_index,
    output reg hash_end,
    output [9:0]code_value_ram_data,
    output reg [9:0]code_value_num,
    output reg com_finsh,
    output reg [3:0]compression_num,
    output reg start,
```

```

        output reg ram_frist,

        input data_start,
        input [7:0]data

    );

    //第一个 always 的变量
    reg [20:0]ss;
    reg [9:0]data_now;
    reg read_start,compression_start;

    //第二个 always 的变量 -- 哈希表的变量
    reg [8:0]prefix_code[20083:0];
    reg [8:0]append_code[20083:0];
    reg [12:0]hash_offset;
    reg hash_judge,hash_read,hash_judge_exist;
    reg hash_index_judge;//hash_end,

    //第三个 always 的变量--查找字典
    reg code_value_data_read;
    reg com_ok;
    reg code_end;
    reg com_start;
    reg com_wait;
    reg com_delay;

    //第四个 always 的变量 -- 编码输出

    reg [9:0]compression_data_pre;
    reg compression_frist;
    reg compression_end;

    //ram 的 always 的变量
    wire [9:0]code_value_ram_data1;
    wire [9:0]code_value_ram_data2;
    reg [9:0]LZW_write_data1;
    reg [9:0]LZW_write_data2;
    reg code_value_ram_rden1,code_value_ram_wren1;
    reg code_value_ram_wren2,code_value_ram_rden2;//
    reg [14:0]code_value_ram_address1;
    reg [12:0]code_value_ram_address2;

    assign code_value_ram_data = ram_frist ? code_value_ram_data2:code_value_ram_data1;

```

```

always @(posedge clk or negedge rest)
begin
    if(!rest)
    begin
        data_now<=10'd0;
        read_start<=0;
        compression_start<=0;
    end
    else if(data_start && !read_start )
    begin
        read_start<=1;
    end
    else if(read_start && data_start && start&& !data_end)
    begin
        data_now <= data;
        compression_start<=1;
    end
    else if(com_finsh)
    begin
        read_start<=0;
    end
    else compression_start<=0;
end
//哈希查找表
reg [13:0]hash_index_mid;
reg hash_wait;
reg [1:0]delay;
reg judge;
always @(posedge clk or negedge rest)
begin
    if(!rest)
    begin
        hash_judge<=0;
        hash_index_judge<=0;
        hash_read<=0;
        hash_index<=11'd0;
        hash_offset<=13'd0;
        hash_index_mid = 14'd0;
        hash_end<=0;
        delay=2'd0;
        judge<=0;
        hash_wait<=0;
    end
end

```

```

else if(compression_start)
begin
    hash_index<=(data_now<<2)^pre_data;
    hash_judge<=1;
end
else if(hash_judge)
begin
    hash_read<=1;
    hash_judge<=0;
    if(hash_index)
    begin
        hash_offset<=5021-hash_index;
    end
    else begin    hash_offset<=1; end
end
else if(hash_read)
begin
    if(delay<2)
        delay<=delay+1;
    else
    begin
        hash_read <= 0;
        delay<=2'd0;
        if(code_value_ram_data[9])
        begin
            hash_end<=1;
        end
        else if((prefix_code[hash_index] == pre_data) && (append_code[hash_index]
== data_now))
        begin
            hash_end<=1;
        end
        else
        begin
            hash_index_mid = hash_index - hash_offset;
            hash_index_judge<=1;
        end
    end
end
else if(hash_index_judge)
begin
    if(hash_index_mid & 14'h2000)
        hash_index<=5021-(14'h3fff-hash_index_mid+1);
        hash_index_judge<=0;

```

```

        hash_wait<=1;
    end
    else if(hash_wait)
    begin
        hash_wait<=0;hash_read<=1;
        judge<=0;
    end
    else if(hash_end) begin if(delay<2)delay <= delay+1;else begin judge <=0;delay <=
2'd0;hash_end<=0;end end
    end

always @(posedge clk or negedge rest)
begin
    if(!rest)
    begin
        code_value_data_read<=0;
        code_value_num<=9'd258;
        ram_frist<=0;
        com_ok <=0;
        code_end<=0;
        pre_data<=9'd0;
        com_finsh<=0;
        com_start<=0;
        com_delay<=0;
        com_wait<=0;
    end
    else if(data_start && !read_start)
    begin
        pre_data<=9'd256;
        com_ok<=1;
        com_start<=1;
    end
    else if(data_end)
    begin
        pre_data<=9'd257;
        com_ok<=1;
        com_start<=1;
    end
    else if(hash_end)
    begin
        code_value_data_read<=1;
    end
    else if(code_value_data_read && !hash_end)
    begin

```

```

    if(code_value_ram_data[9])
    begin
        if(code_value_num<9'h1ff)
        begin
            code_value_num<=code_value_num+1;
            prefix_code[hash_index] <= pre_data ;
            append_code[hash_index] <= data_now ;
        end
        else
        begin
            prefix_code[hash_index] <= pre_data ;
            append_code[hash_index] <= data_now ;
            code_value_num<=258;
            ram_frist <= ~ram_frist;
            code_end<=1;
        end
        com_ok <=1;
    end
    else pre_data <= code_value_ram_data;
    code_value_data_read<=0;
end
else if(com_ok &&!code_end )
begin
    if(!com_finsh)
    pre_data<=data_now;
    else com_finsh<=0;
    com_ok<=0;
end
else if(code_end)
begin
    pre_data<=data_now;com_finsh<=1;code_end<=0;
end
else if(com_start)
begin
    pre_data<=data;
    com_start<=0;
end
end

reg com_finsh_pre,com_finsh_now;
wire LZW_clear_start = com_finsh_pre && !com_finsh_now;
always @(posedge clk or negedge rest)
begin
    if(!rest)

```

```

begin
    com_finsh_now<=0;
    com_finsh_pre<=0;
end
else
begin
    com_finsh_now<=com_finsh;
    com_finsh_pre<=com_finsh_now;
end
end

//ram 读写
reg LZW_frist_clear;
reg LZW_ram_clear;
reg [12:0]address_clear_num;
always @(posedge clk or negedge rest)
begin
    if(!rest)
    begin
        code_value_ram_address1<=0;
        code_value_ram_rden1<=0;
        code_value_ram_wren1<=0;
        LZW_write_data1<=0;
        LZW_ram_clear<=1;
        address_clear_num<=13'd0;
        start<=0;
    end
    else if(!ram_frist && !LZW_ram_clear)
    begin
        if(hash_judge || hash_wait || hash_end)
        begin
            code_value_ram_address1<=hash_index;code_value_ram_rden1<=1;
        end
        else if(code_value_data_read && !LZW_ram_clear )
        begin
            if(code_value_ram_data[9])
            begin
                code_value_ram_address1<=hash_index;
                code_value_ram_wren1<=1;
                if(code_value_num<9'h1ff)
                begin
                    LZW_write_data1[8:0]<= code_value_num ;
                    LZW_write_data1[9]<= 0 ;
                end
            end
        end
    end
end

```

```

        else
        begin
            LZW_write_data1[8:0]<= code_value_num ;
            LZW_write_data1[9]<= 0 ;
        end
    end
end
else begin code_value_ram_rden1<=0;code_value_ram_wren1<=0; end
end
else if(LZW_clear_start && ram_frist)
begin
    LZW_ram_clear<=1;
end
else if(LZW_ram_clear)
begin
    code_value_ram_address1<=address_clear_num;
    code_value_ram_wren1<=1;
    LZW_write_data1<=10'h200;
    if(address_clear_num >5020)
    begin
        address_clear_num<=13'd0;
        LZW_ram_clear<=0;
        code_value_ram_wren1<=0;
        start<=1;
    end
    else address_clear_num<=address_clear_num+1;
end
end

reg LZW_frist_clear1;
reg LZW_ram_clear1;
reg [12:0]address_clear_num1;
always @(posedge clk or negedge rest)
begin
    if(!rest)
    begin
        code_value_ram_address2<=0;
        code_value_ram_rden2<=0;
        code_value_ram_wren2<=0;
        LZW_write_data2<=0;
        LZW_ram_clear1<=1;
        address_clear_num1<=13'd0;
    end
    else if(ram_frist)

```



```

begin
    if(hash_judge || hash_wait || hash_end)
    begin
        code_value_ram_address2<=hash_index;code_value_ram_rden2<=1;
    end
    else if(code_value_data_read )
    begin
        if(code_value_ram_data[9])
        begin
            code_value_ram_address2<=hash_index;
            code_value_ram_wren2<=1;
            if(code_value_num<9'h1ff)
            begin
                LZW_write_data2[8:0]<= code_value_num ;
                LZW_write_data2[9]<= 0 ;
            end
            else
            begin
                LZW_write_data2[8:0]<= code_value_num ;
                LZW_write_data2[9]<= 0 ;
            end
        end
    end
    else begin code_value_ram_rden2<=0;code_value_ram_wren2<=0; end
end
else if(LZW_clear_start && !ram_frist)
begin
    LZW_ram_clear1<=1;
end
else if(LZW_ram_clear1)
begin
    code_value_ram_address2<=address_clear_num1;
    code_value_ram_wren2<=1;
    LZW_write_data2<=10'h200;
    if(address_clear_num1>5020)
    begin
        address_clear_num1<=13'd0;
        LZW_ram_clear1<=0;
        code_value_ram_wren2<=0;
    end
    else address_clear_num1<=address_clear_num1+1;
end
end
end

```

```

reg [8:0]compression_data_pre_pre;
reg compression_one;
reg end_frist;
always @(posedge clk or negedge rest)
begin
    if(!rest)
    begin
        compression_data<=8'd0;
        compression_num<=4'd0;
        compression_data_pre<=10'd0;
        compression_data_pre_pre<=9'd0;
        compression_ok<=0;
        compression_frist<=0;
        compression_end<=0;
        compression_one<=0;
        end_frist<=0;
    end
    else if(com_ok && !compression_frist)
    begin
        compression_data  <=  ((pre_data  &  10'h1ff)>>(compression_num+1))  |
(compression_data_pre <<(8-compression_num));
        if(compression_num==7)
        begin
            compression_num<=0;
            compression_frist<=1;
        end
        else compression_num<=compression_num+1;
        compression_data_pre <= pre_data;
        compression_ok<=1;
    end
    else if(!compression_num && compression_frist)
    begin
        compression_data<=compression_data_pre;
        compression_frist<=0;
        compression_data_pre_pre <= pre_data;
    end
    else if(compression_ok)
    begin
        compression_ok<=0;
        compression_one<=0;
        if(compression_end)
        begin
            compression_end<=0;
            compression_data_pre <= pre_data;

```

```

        end
    end
    else if(data_end && !end_frist)
    begin
        compression_ok<=1;
        compression_data<=compression_data_pre;
        end_frist<=1;
    end
end

code_value u0_code_value
(
    .clock (clk),
    .data(LZW_write_data1),
    .wren(code_value_ram_wren1),
    .address(code_value_ram_address1),
    .rden(code_value_ram_rden1),

    .q(code_value_ram_data1)
);

code_value_ram_sec u1_code_value_ram_sec
(
    .clock (clk),
    .data(LZW_write_data2),
    .wren(code_value_ram_wren2),
    .address(code_value_ram_address2),
    .rden(code_value_ram_rden2),

    .q(code_value_ram_data2)
);
endmodule

```

Modelsim 仿真局部波形图及 FPGA 硬件资源利用率分别如下图所示。



图 3 Modelsim 仿真局部波形图

Flow Status	Successful - Thu Dec 31 15:27:02 2020
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Standard Edition
Revision Name	Compression_LZW
Top-level Entity Name	Compression_LZW
Family	Cyclone IV E
Device	EP4CE15E22C8
Timing Models	Final
Total logic elements	434 / 15,408 (3 %)
Total registers	263
Total pins	69 / 82 (84 %)
Total virtual pins	0
Total memory bits	135,256 / 516,096 (26 %)
Embedded Multiplier 9-bit elements	0 / 112 (0 %)
Total PLLs	0 / 4 (0 %)

图 4 FPGA 硬件资源利用率

8.6 参考文献

[1] 刘成明, 李新娥, 张艳兵. 基于 FPGA 的数据采集与压缩系统[J]. 仪表技术与传感器, 2012(01):36-39.

[2] 鄢海舟, 胥布工, 石东江, 等. 无损压缩算法 LZW 前缀编码优化及应用[J]. 计算机工程, 2017, 43(03):299-303.

[3] Aruna Malik, Geeta Sikka, Harsh K. Verma. A high capacity text steganography scheme based on LZW compression and color coding[J]. Engineering Science and Technology, an International Journal, 2017, 20(1):72-79.

[4] Shunji Funasaka, Koji Nakano, Yasuaki Ito. Adaptive loss-less data compression method optimized for GPU decompression[J]. Concurrency and Computation: Practice and Experience, 2017, 29(24):4283.

[5] 宋洪良, 唐小明, 张涛, 等. 用于雷达数据实时传输的 LZW 算法优化[J]. 雷达科学与技术, 2016, 14(04):397-402.

[6]陈世海,裴东兴,张琦.FPGA 实现滑动平均滤波算法和 LZW 压缩算法.电子设计工程,2010,18(2):67-69.

9. 基于 FPGA 的永磁同步电机伺服控制

9.1 算法原理

目前主流的 PMSM (永磁同步电机) 伺服控制方案为 DTC 控制(直接扭矩控制)和 FOC 控制(磁场矢量) 以及上述算法的优化改进方案, 此实验主要实现了传统的有感 FOC 控制。该算法主要包含了 PI 控制器设计、Cordic 算法设计、Park 变换及反变换算法设计、Clark 变换及反变换算法设计以及 SVPWM 波调制方案设计。

FOC 算法的核心思想为将复杂的三相电流关系解耦为控制系统无功功率的 I_d 和有功功率的 I_q , 实现以直流电机的控制方式来控制交流电机, 实现流程图如图 1 所示:

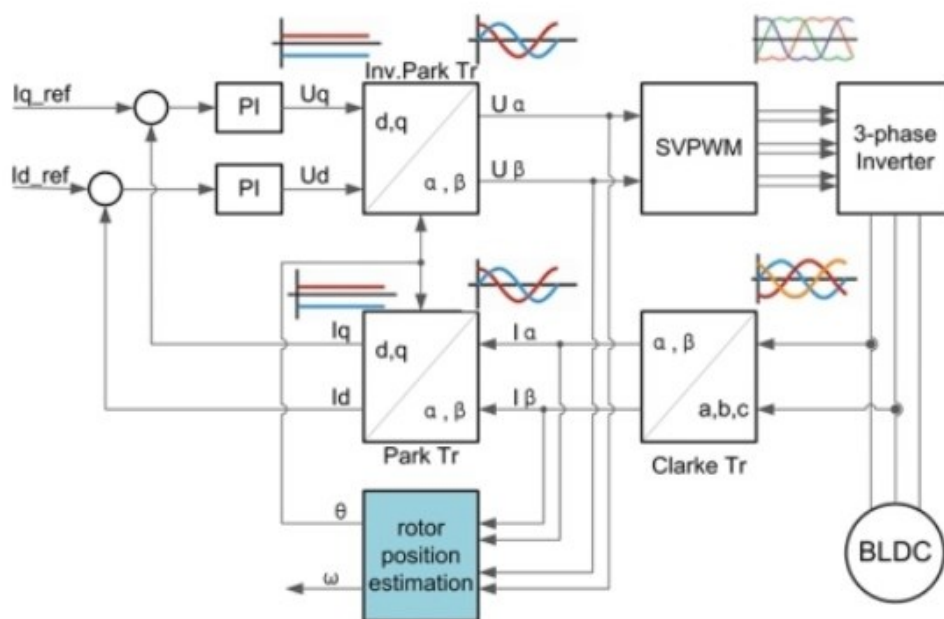


图 1 FOC 核心算法示意图

Clark 变换 (3S/2S)

Clark 变换实现三相电流到两相电流的坐标转换, 将复杂的电流关系表示到相对简单的直角坐标系, 实现电流关系的化简, 化简关系如图 2 所示。

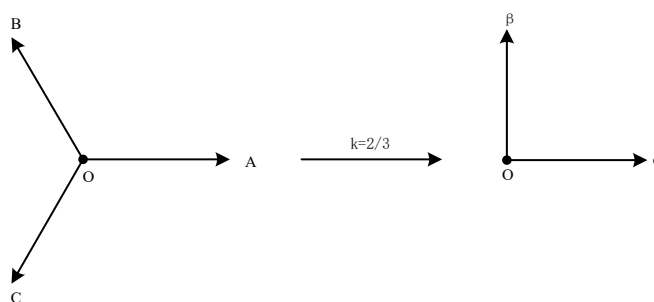


图 1 Clark 变换映射

$$\begin{cases} I_{\alpha} = k(I_A + I_B \cos \frac{2\pi}{3} + I_C \cos \frac{2\pi}{3}) \\ I_{\beta} = k(I_A \cos \frac{\pi}{2} + I_B \cos \frac{\pi}{6} + I_C \cos \frac{5\pi}{6}) \end{cases}$$

该处采用理想电机模型作为参照，故可采用基尔霍夫电流定理进行计算简化，化简结果如下所示：

$$\begin{cases} I_{\alpha} = I_A \\ I_{\beta} = \frac{1}{\sqrt{3}}I_A + \frac{2}{\sqrt{3}}I_B \end{cases}$$

Park 变换 (2S/2R)

Park 变换实现了电流关系的进一步化简，将 α β 坐标系关系转化为 DQ 坐标关系，其中 I_d (直轴电流) 用于控制转子的励磁大小， I_q (交轴电流) 用于控制转子的力矩大小，其化简关系如图 3 所示。

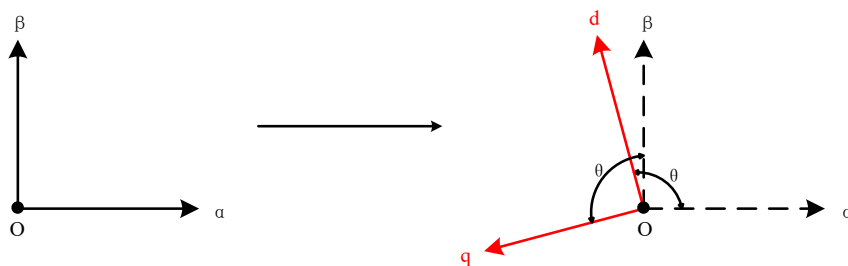


图 2 Park 变换映射

$$\begin{cases} I_d = I_{\alpha} \cos \theta + I_{\beta} \sin \theta \\ I_q = -I_{\alpha} \sin \theta + I_{\beta} \cos \theta \end{cases} \rightarrow \begin{bmatrix} I_d \\ I_q \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} I_{\alpha} \\ I_{\beta} \end{bmatrix}$$

观察上式可以发现，变换过程中需要获得相应电角度的三角函数关系，故需要提前计算进行 Cordic 变换得到 Sin 和 Cos 数值，本算法不是本次作业重点故不做展开介绍。

Clark 反变换及 SVPWM 波调制

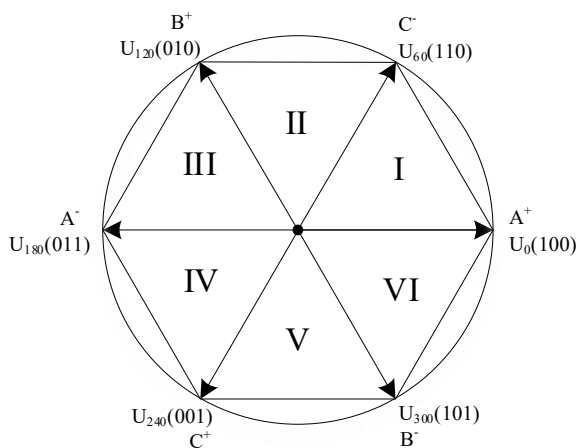


图 3 6 个基本矢量展示

由图 4 可看出，可以直接通过 UVW 三相的电平高低合成的是这 6 个基础向量。而在这 6 个基础向量之外的方向，都需要取上述这 6 个基础向量的其中 2 个来进行合成。这种合成的方式就是通过 PWM 的方式，改变两个基础向量在一段信号中的占比，从而达到对两个矢量方向大小的控制。但仅仅通过两个不同的基础向量是不能做到控制合成矢量的大小的，只能控制其合成的方向。因此还需要引入两个零向量 $U(000)$ 和 $U(111)$ ，在一个 SVPWM 周期中，通过调节 2 个基础向量和两个零向量的时间占比，可以达到对合成矢量方向和大小的控制。

在此过程中需要采用 Clark 反变换来进行三相电流的反解，利用求解得到的 UVW 三相

电流进行扇区估计并输出相应的 PWM 波形。其中 Clark 反变换如下式所示

$$\begin{cases} U_1 = U_\beta \\ U_2 = \frac{\sqrt{3}U_\alpha - U_\beta}{2} \\ U_3 = -\frac{\sqrt{3}U_\alpha + U_\beta}{2} \end{cases}$$

$$\begin{cases} \{A = 1 | U_1 > 0\} \\ \{B = 1 | U_2 > 0\} \\ \{C = 1 | U_3 > 0\} \end{cases}$$

N 值计算及扇区确定如下式所示：

$$N = A + 2B + 4C$$

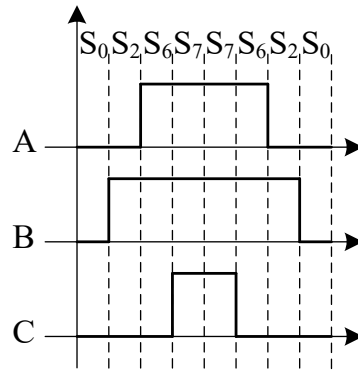
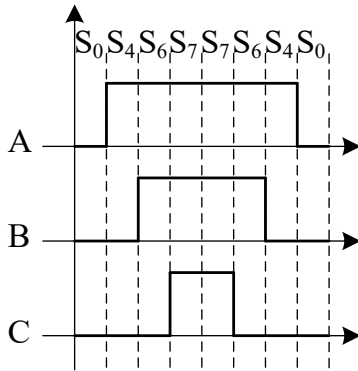
N 值	3	1	5	4	6	2
扇区	I	II	III	IV	V	VI

具体调制方式采用七段式调制方案，其为对称式波形输出，具有较小的谐波分量。

扇区 I ($\theta \in [0^\circ, 60^\circ]$) $S_x=S_4$ $S_y=S_6$

扇区 II ($\theta \in [60^\circ, 120^\circ]$) $S_x=S_2$

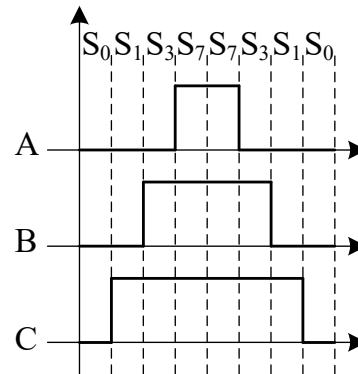
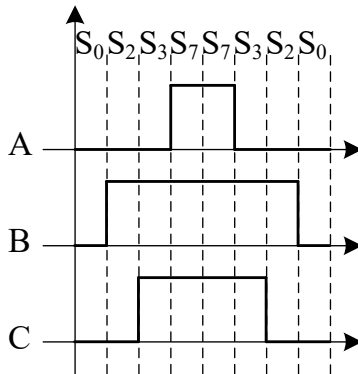
$S_y=S_6$



扇区 III ($\theta \in [120^\circ, 180^\circ]$) $S_x=S_2$ $S_y=S_3$

扇区 IV ($\theta \in [180^\circ, 240^\circ]$) $S_x=S_1$

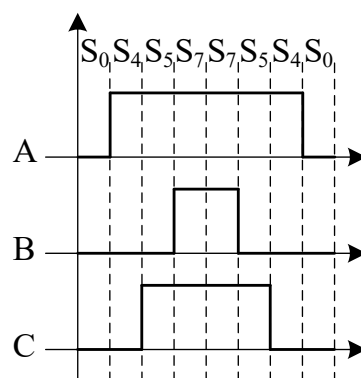
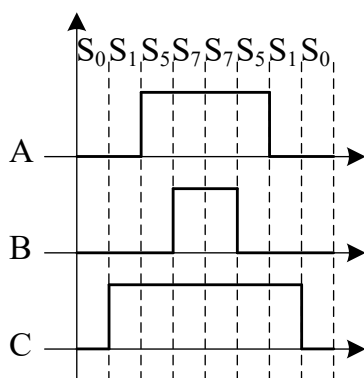
$S_y=S_3$



扇区 V ($\theta \in [240^\circ, 300^\circ]$) $S_x=S_1$ $S_y=S_5$

扇区 VI ($\theta \in [300^\circ, 360^\circ]$) $S_x=S_4$

$S_y = S_5$



Park 反变换

Park 反变换即为 Park 变换的逆过程可得 $\begin{bmatrix} I_\alpha \\ I_\beta \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}^{-1} \begin{bmatrix} I_d \\ I_q \end{bmatrix}$, 最终可化简为

$$\begin{bmatrix} I_\alpha \\ I_\beta \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} I_d \\ I_q \end{bmatrix}.$$

9.2 Matlab 验证代码

```
% i_A, i_B, i_C 为三相对称正弦电流，相位差 120°
% vec_i_A, vec_i_B, vec_i_C 为输出三相对称正弦电流的向量表示方法
% vec_i_S 为向量合成的结果，其结果为单一相电流模长的 1.5 倍，得到后续的映射到 a, b 轴上的系数 2/3
% a 轴与 A 轴重合，将 B 轴和 C 轴正交分解到 a 轴和 b 轴上，得到最终结果
function [i_a, i_b] = Clark_Transformation(i_A, i_B, i_C)

%   phase = 2i*pi/3;
%   vec_i_A = i_A;
%   vec_i_B = i_B * exp(phase);
%   vec_i_C = i_C * exp(-phase);
%   vec_i_S = vec_i_A + vec_i_B + vec_i_C;
%   % 以上步骤可以得到三相合成的矢量模长为单一相的矢量模长的 1.5 倍

i_a = (2/3)*(i_A - i_B * cos(pi/3) - i_C * cos(pi/3));
i_b = (2/3)*(i_B * cos(pi/6) - i_C * cos(pi/6));
%   % 以上步骤可以得到 Clark 变化后 a, b 轴上的电流映射关系
%   % 根据基尔霍夫电流定律 (i_A + i_B + i_C = 0) 可继续化简，最终得到下式
%   i_a = i_A;
%   i_b = (1/sqrt(3))*i_A + (2/sqrt(3))*i_B;
end
```

```

% i_a, i_b 为 clark 变换后得到的结果
% ang 为 d 轴与 a 轴的夹角, 也为 q 轴与 b 轴的夹角, 由编码器数据获得
% d 轴平行于磁场, 称为直轴; q 轴垂直于磁场, 称为交轴
function [i_d, i_q] = Park_Transformation(i_a, i_b, ang)

%   i_d_a = i_a .* cos(ang);
%   i_d_b = i_b .* sin(ang);
%   i_q_a = i_a .* sin(ang);
%   i_q_b = i_b .* cos(ang);
%   %将 a, b 轴上的矢量分别分解到 d, q 轴上

%   i_d = i_d_a + i_d_b;
%   i_q = -i_q_a + i_q_b;
%   %重新合成 d, q 上的矢量

%   %以下式子为最终化简结果
i_d = i_a.*cos(ang) + i_b.*sin(ang);
i_q = -i_a.*sin(ang) + i_b.*cos(ang);
end

```

```

function [i_A_i, i_B_i, i_C_i] = Clark_Inverse_Transformation(i_a_i,
i_b_i)
    i_A_i = i_a_i;
%   i_B_i = -cos(2*pi/3)*i_a_i + sin(pi/3)*i_b_i;
%   i_C_i = -cos(2*pi/3)*i_a_i - sin(pi/3)*i_b_i;

    i_B_i = (1/2)*(sqrt(3)*i_b_i - i_a_i);
    i_C_i = -i_A_i - i_B_i;
end

```

```

function [i_a_i, i_b_i] = Park_Inverse_Transformation(i_d, i_q, ang)
%   i_a_d = i_d.*cos(ang);
%   i_a_q = i_q.*sin(ang);
%   i_b_d = i_d.*sin(ang);
%   i_b_q = i_q.*cos(ang);
%   %将 d, q 轴上的矢量分别分解到 a, b 轴上
%
%   i_a_i = i_a_d - i_a_q;
%   i_b_i = i_b_d + i_b_q;
%   %重新合成 a, b 上的矢量

%   %以下式子为最终化简结果
i_a_i = i_d.*cos(ang) - i_q.*sin(ang);
i_b_i = i_d.*sin(ang) + i_q.*cos(ang);

```

```
end
```

```
clear
close all
clc

Im = 5;
w = pi;
T = 2*pi/w;
p = 2*pi/3;
t = -2:0.1:2;

i_A = Im * cos(w*t);
i_B = Im * cos(w*t-p);
i_C = Im * cos(w*t+p);

[i_a, i_b] = Clark_Transformation(i_A, i_B, i_C);
[i_d, i_q] = Park_Transformation(i_a, i_b, w*t);

figure(1);
subplot(3, 1, 1);
plot(t, i_A, 'r', t, i_B, 'g', t, i_C, 'b')
title('Origin')
subplot(3, 1, 2);
plot(t, i_a, 'r', t, i_b, 'g')
title('Clark')
subplot(3, 1, 3)
plot(t, i_d, 'r', t, i_q, 'g')
title('Park')

[i_a_i, i_b_i] = Park_Inverse_Transformation(i_d, i_q, w*t);
[i_A_i, i_B_i, i_C_i] = Clark_Inverse_Transformation(i_a_i, i_b_i);

figure(2);
subplot(3, 1, 1);
plot(t, i_d, 'r', t, i_q, 'g')
title('Origin')
subplot(3, 1, 2);
plot(t, i_a_i, 'r', t, i_b_i, 'g')
title('InversedPark')
subplot(3, 1, 3)
plot(t, i_A_i, 'r', t, i_B_i, 'g', t, i_C_i, 'b')
title('InversedClark')
```

```

U_dc = 9;
U_a = i_a_i;
U_b = i_b_i;

area = Get_Control_Area(U_a, U_b);
Wave(U_a, U_b);
[T_x, T_y, T_z] = Get_Control_Time(U_dc, U_a, U_b, T, area);
Modulate(T_x, T_y, T_z, T, area);

```

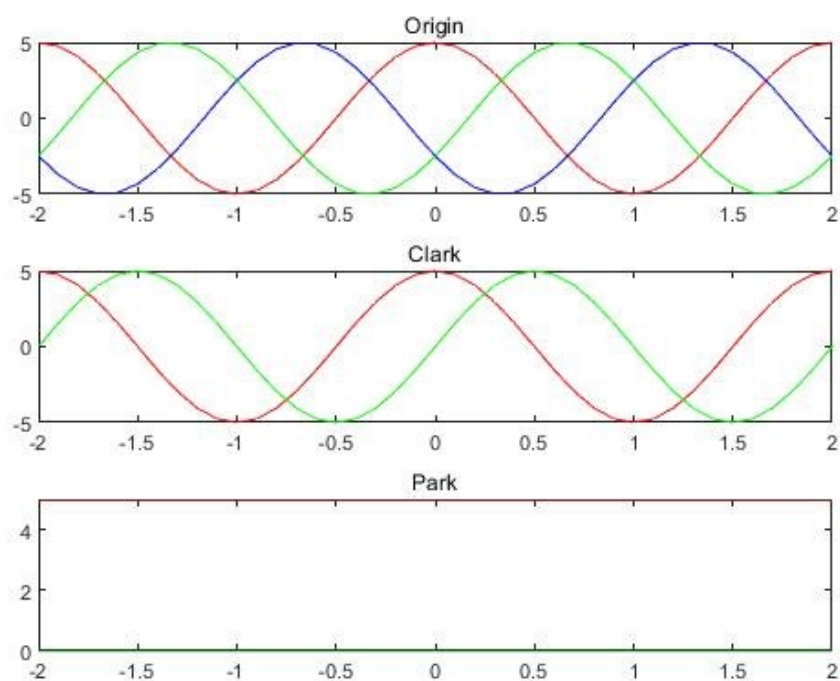


图 4 Matlab 仿真情况(单向)

9.3 C++验证代码

Clark 变换:

```

void Clark_Transformation(int* Ix, int* Iy, int* Ia, int* Ib){
    int Ix_temp, Iy_temp;
    Ix_temp = (*Ia_temp);
    Iy_temp = (((*Ia_temp) + (*Ib_temp)<<1)*2365)>>12;

    *Ix = Ix_temp;

    if(Iy_temp > MAX){
        *Iy = MAX;
    }
}

```

```

    }
    else if(Iy_temp < -MAX){
        *Iy = -MAX;
    }
    else{
        *Iy = Iy_temp;
    }
}

```

Park 变换:

```

void Park_Transformation(int* Id, int* Iq, int* Ix, int* iy, int*
Sin, int* Cos){
    int Id_temp, Iq_temp;
    Id_temp = ((*Sin)*(*Iy) + (*Cos)*(*Ix))>>11;
    Iq_temp = ((*Cos)*(*Iy) - (*Sin)*(*Ix))>>11;

    if(Id_temp > MAX){
        *Id = MAX;
    }
    else if(Id_temp < -MAX){
        *Id = -MAX;
    }
    else{
        *Id = Id_temp;
    }

    if(Iq_temp > MAX){
        *Iq = MAX;
    }
    else if(Iq_temp < -MAX){
        *Iq = -MAX;
    }
    else{
        *Iq = Iq_temp;
    }
}

```

Clark 反变换:

```

void InvClark_Transformation(int* U1, int* U2, int* U3, int* Ux,
int* Uy){
    int U1_temp, U2_temp, U3_temp;
    U1_temp = *Uy;
    U2_temp = ((*Ux)*28378 - (*Uy)*16384)>>15;
    U3_temp = (-(*Ux)*28378 - (*Uy)*16384)>>15;
}

```

```

    *U1 = U1_temp;

    if(U2_temp > MAX){
        *U2 = MAX;
    }
    else if(U2_temp < -MAX){
        *U2 = -MAX;
    }
    else{
        *U2 = U2_temp;
    }

    if(U3_temp > MAX){
        *U3 = MAX;
    }
    else if(U3_temp < -MAX){
        *U3 = -MAX;
    }
    else{
        *U3 = U3_temp;
    }
}

```

Park 反变换:

```

void InvPark_Transformation(int* Ux, int* Uy, int* Ud, int* Uq, int*
Sin, int* Cos){
    int Ux_temp, Uy_temp;
    Ux_temp = ((*Cos)*(*Ud) - (*Sin)*(*Uq))>>15;
    Uy_temp = ((*Sin)*(*Ud) + (*Cos)*(*Uq))>>15;

    if(Ux_temp > MAX){
        *Ux = MAX;
    }
    else if(Ux_temp < -MAX){
        *Ux = -MAX;
    }
    else{
        *Ux = Ux_temp;
    }

    if(Uy_temp > MAX){
        *Uy = MAX;
    }
}

```



```

    oDir,
    oTx,
    //ADC121 通讯信号(母线)

    //ADC124 通讯信号(电流/电压)
    iADC124_MISO,
    oADC124_CS_n,
    oADC124_SCLK,
    oADC124_MOSI,
    //控制信号
    oPWM_u,
    oPWM_v,
    oPWM_w,
    oSD_u_n,
    oSD_v_n,
    oSD_w_n
);
input wire iClk;
input wire iRst_n;
//编码器通讯信号
input wire iRx;
output wire oDir;
output wire oTx;
//ADC121 通讯信号(母线)

//ADC124 通讯信号(电流/电压)
input wire iADC124_MISO;
output wire oADC124_CS_n;
output wire oADC124_SCLK;
output wire oADC124_MOSI;
//控制信号
output wire oPWM_u,oPWM_v,oPWM_w;
output wire oSD_u_n,oSD_v_n,oSD_w_n;

localparam S0 = 1'd0;
localparam S1 = 1'd1;

reg nstate;
reg ncl_en;
wire nmodulate_done;
wire [15:0] nsin,ncos;
wire [12:0] nspeed;
wire [11:0] niq;
wire ncdt_warning;

```



```

wire nclk_100m;
wire np1l_locked;

assign oSD_u_n = 1'b1;
assign oSD_v_n = 1'b1;
assign oSD_w_n = 1'b1;

always @(posedge nclk_100m or negedge iRst_n) begin
    if(!iRst_n) begin
        ncl_en <= 1'b0;
        nstate <= S0;
    end
    else begin
        case (nstate)
            S0: begin
                if(np1l_locked) begin
                    ncl_en <= 1'b1;
                    nstate <= S1;
                end
                else begin
                    ncl_en <= ncl_en;
                    nstate <= nstate;
                end
            end
            S1: begin
                ncl_en <= 1'b0;
                nstate <= nstate;
            end
            default: nstate <= S0;
        endcase
    end
end

PLL_100M pll_100m(
    .inclk0(iClk),
    .c0(nclk_100m),
    .locked(np1l_locked)
);

Corder_DataTreat u_coder_datatreat(
    .iClk(nclk_100m),
    .iRst_n(iRst_n),
    .iEn_Rd(nmodulate_done),
    .iRx(iRx),

```

```

        .oDir(oDir),
        .oTx(oTx),
        .oSin(nsin),
        .oCos(ncos),
        .oSpeed(nspeed),
        // .oWarning(ncdt_warning),
        // .oDone_Rd(ncdt_done)
    );

    Current_Loop u_current_loop(
        .iClk(nclk_100m),
        .iRst_n(iRst_n),
        .iCL_en(ncl_en),
        .iSin(nsin),
        .iCos(ncos),
        .iId_set(12'd0),
        .iIq_set(niq),
        .iKp_d(16'd17653),
        .iKi_d(16'd697),
        .iKp_q(16'd17653),
        .iKi_q(16'd697),
        .iADC124_MISO(iADC124_MISO),
        .oADC124_CS_n(oADC124_CS_n),
        .oADC124_SCLK(oADC124_SCLK),
        .oADC124_MOSI(oADC124_MOSI),
        .oPWM_u(oPWM_u),
        .oPWM_v(oPWM_v),
        .oPWM_w(oPWM_w),
        .oModulate_done(nmodulate_done)
    );

    Speed_Loop u_speed_loop(
        .iClk(nclk_100m),
        .iRst_n(iRst_n),
        .iSL_en(nmodulate_done),
        .iSpd_set(13'd500),
        .iSpd_coder(nspeed),
        .iKp(16'd1206),
        .iKi(16'd12),
        .oIq(niq),
        // .oSL_done()
    );

endmodule

```

电流环模块:

```
module Current_Loop(
    iClk,
    iRst_n,
    iCL_en,
    iSin,
    iCos,
    iId_set,
    iIq_set,
    iKp_d,
    iKi_d,
    iKp_q,
    iKi_q,
    iADC124_MISO,
    oADC124_CS_n,
    oADC124_SCLK,
    oADC124_MOSI,
    oPWM_u,
    oPWM_v,
    oPWM_w,
    oModulate_done
);
    input wire iClk;
    input wire iRst_n;
    input wire iCL_en;
    input wire [15:0] iSin,iCos;
    input wire [11:0] iId_set,iIq_set;
    input wire [15:0] iKp_d,iKi_d,iKp_q,iKi_q;
    input wire iADC124_MISO;
    output wire oADC124_CS_n;
    output wire oADC124_SCLK;
    output wire oADC124_MOSI;
    output wire oPWM_u,oPWM_v,oPWM_w;
    output wire oModulate_done;

    wire ncl_en;
    wire signed [15:0] nvd,nvq;
    wire signed [15:0] nvalpha,nvbeta;
    wire signed [11:0] nid_current,niq_current;
    wire nip_done,nadc_treat_done,npical_done;

    assign ncl_en = iCL_en | oModulate_done;
    ADC_DataTreat adc_treat(
        .iClk(iClk),
```

```

        .iRst_n(iRst_n),
        .iEn(nc1_en),
        .iSin(iSin),
        .iCos(iCos),
        .iADC124_MISO(iADC124_MISO),
        .oADC124_CS_n(oADC124_CS_n),
        .oADC124_SCLK(oADC124_SCLK),
        .oADC124_MOSI(oADC124_MOSI),
        .oId_current(nid_current),
        .oIq_current(niq_current),
        .oDone(nadc_treat_done)
    );

```

```

Current_Loop_PI pi(
    .iClk(iClk),
    .iRst_n(iRst_n),
    .iTarget_d(iId_set),
    .iCurrent_d(nid_current),
    .iKp_d(iKp_d),
    .iKi_d(iKi_d),
    .iTarget_q(iIq_set),
    .iCurrent_q(niq_current),
    .iKp_q(iKp_q),
    .iKi_q(iKi_q),
    .iCal_en(nadc_treat_done),
    .oCal_d(nvd),
    .oCal_q(nvq),
    .oCal_done(npical_done)
);

```

```

Inv_Park inv_park(
    .iClk(iClk),
    .iRst_n(iRst_n),
    .iIP_en(npical_done),
    .iSin(iSin),
    .iCos(iCos),
    .iVd(nvd),
    .iVq(nvq),
    .oIP_done(nip_done),
    .oValpha(nvalpha),
    .oVbeta(nvbeta)
);

```

```

SVPWM svpwm(

```

```

        .iClk(iClk),
        .iRst_n(iRst_n),
        .iModulate_en(nip_done),
        .iValpha(nvalpha),
        .iVbeta(nvbeta),
        .oPWM_u(oPWM_u),
        .oPWM_v(oPWM_v),
        .oPWM_w(oPWM_w),
        .oModulate_done(oModulate_done)
    );

```

endmodule

Clark 反变换:

```

module Inv_Clark(
    iClk,
    iRst_n,
    iIC_en,
    iValpha,
    iVbeta,
    oV1,
    oV2,
    oV3,
    oIC_done
);
    input wire iClk;
    input wire iRst_n;
    input wire iIC_en;
    input wire signed [15:0] iValpha,iVbeta;
    output reg signed [15:0] oV1,oV2,oV3;
    output reg oIC_done;

    localparam num_sqrt3_2 = 10'd886;    //sqrt(3)/2 * (2^10-1)

    localparam S0 = 2'd0;
    localparam S1 = 2'd1;
    localparam S2 = 2'd2;

    reg nic_en_pre_state;
    reg [1:0] nstate;
    reg signed [26:0] ncalout_1,ncalout_2;

    always @(posedge iClk or negedge iRst_n) begin
        if(!iRst_n) begin
            nic_en_pre_state <= 1'b0;

```

```

        end
        else begin
            nic_en_pre_state <= iIC_en;
        end
    end

    always @(posedge iClk or negedge iRst_n) begin
        if(!iRst_n) begin
            ncalout_1 <= 27'd0;
            ncalout_2 <= 27'd0;
            nstate <= S0;
            oIC_done <= 1'b0;
            oV1 <= 16'd0;
            oV2 <= 16'd0;
            oV3 <= 16'd0;
        end
        else begin
            case (nstate)
                S0: begin
                    if((!nic_en_pre_state) & iIC_en) begin
                        ncalout_1 <= iValpha *
$signed({{1'b0},num_sqrt3_2});
                        ncalout_2 <= iVbeta>>>1;
                        nstate <= S1;
                    end
                    else begin
                        nstate <=nstate;
                        oIC_done <= 1'b0;
                    end
                end
                S1: begin
                    nstate <= S0;
                    oV1 <= iVbeta;
                    oV2 <= $signed(ncalout_1[25:10]) -
$signed(ncalout_2[15:0]);
                    oV3 <= -$signed(ncalout_1[25:10]) -
$signed(ncalout_2[15:0]);
                    oIC_done <= 1'b1;
                end
                default: nstate <=S0;
            endcase
        end
    end
end

```

```
endmodule
```

Park 反变换:

```
module Inv_Park(
    iClk,
    iRst_n,
    iIP_en,
    iSin,
    iCos,
    iVd,
    iVq,
    oIP_done,
    oValpha,
    oVbeta
);
    input wire iClk;
    input wire iRst_n;
    input wire iIP_en;
    input wire signed [15:0] iSin,iCos;
    input wire signed [15:0] iVd,iVq;
    output reg oIP_done;
    output reg signed [15:0] oValpha,oVbeta;//有符号类型

    localparam S0 = 2'd0;
    localparam S1 = 2'd1;
    localparam S2 = 2'd2;

    reg nip_en_pre_state;
    reg [1:0] nstate;
    reg signed [31:0] ntemp_dc,ntemp_qs,ntemp_ds,ntemp_qc;

    always @(posedge iClk or negedge iRst_n) begin
        if(!iRst_n) begin
            nip_en_pre_state <= 1'b0;
        end
        else begin
            nip_en_pre_state <= iIP_en;
        end
    end

    always @(posedge iClk or negedge iRst_n) begin
        if(!iRst_n) begin
            ntemp_dc <= 32'd0;
            ntemp_ds <= 32'd0;
            ntemp_qc <= 32'd0;
        end
    end
end
```

```

        ntemp_qs <= 32'd0;
        nstate <= S0;
        oValpha <= 16'd0;
        oVbeta <= 16'd0;
        oIP_done <= 1'b0;
    end
    else begin
        case (nstate)
            S0: begin
                if(!nip_en_pre_state) & iIP_en) begin
                    ntemp_dc <= iVd * iCos;
                    ntemp_ds <= iVd * iSin;
                    ntemp_qc <= iVq * iCos;
                    ntemp_qs <= iVq * iSin;
                    nstate <= S1;
                end
            end
            else begin
                oIP_done <= 1'b0;
                nstate <= nstate;
            end
        end
    end
    S1: begin
        nstate <= S0;
        oValpha <= $signed(ntemp_dc[30:15]) -
$signed(ntemp_qs[30:15]);
        oVbeta <= $signed(ntemp_ds[30:15]) +
$signed(ntemp_qc[30:15]);
        oIP_done <= 1'b1;
    end
    default: nstate <= S0;
endcase
end
end

endmodule

```

Clark 变换:

```

module Clark(
    iClk,
    iRst_n,
    iC_en,
    iIu,
    iIv,
    oIalpha,
    oIbeta,

```



```

        oC_done
    );
    input wire iClk;
    input wire iRst_n;
    input wire iC_en;
    input wire signed [11:0] iIu,iIv;
    output reg signed [11:0] oIalpha,oIbeta;
    output reg oC_done;

    localparam num_1_sqrt3 = 10'd591;    //1/sqrt(3) * (2^10-1)

    localparam S0 = 2'd0;
    localparam S1 = 2'd1;
    localparam S2 = 2'd2;

    reg nc_en_pre_state;
    reg [1:0] nstate;
    reg signed [22:0] niu_temp,niv_temp;

    always @(posedge iClk or negedge iRst_n) begin
        if(!iRst_n) begin
            nc_en_pre_state <= 1'b0;
        end
        else begin
            nc_en_pre_state <= iC_en;
        end
    end

    always @(posedge iClk or negedge iRst_n) begin
        if(!iRst_n) begin
            niu_temp <= 23'd0;
            niv_temp <= 23'd0;
            nstate <= S0;
            oIalpha <= 12'd0;
            oIbeta <= 12'd0;
            oC_done <= 1'b0;
        end
        else begin
            case (nstate)
                S0: begin
                    if((!nc_en_pre_state) & iC_en) begin
                        niu_temp <= iIu *
$signed({1'b0,num_1_sqrt3});

```

```

                                niv_temp <= iIv *
$signed({1'b0,num_1_sqrt3});
                                nstate <= S1;
                                end
                                else begin
                                    nstate <= nstate;
                                    oC_done <= 1'b0;
                                end
                                end
                                S1: begin
                                    nstate <= S0;
                                    oIalpha <= iIu;
                                    oIbeta <= $signed(niu_temp[21:10]) +
$signed(niv_temp[20:9]);
                                    oC_done <= 1'b1;
                                end
                                default: nstate <= S0;
                            endcase
                        end
                    end
                end

endmodule

```

Park 变换:

```

module Park(
    iClk,
    iRst_n,
    iP_en,
    iSin,
    iCos,
    iIalpha,
    iIbeta,
    oId,
    oIq,
    oP_done
);
    input wire iClk;
    input wire iRst_n;
    input wire iP_en;
    input wire signed [15:0] iSin,iCos;
    input wire signed [11:0] iIalpha,iIbeta;
    output reg signed [11:0] oId,oIq;
    output reg oP_done;

    localparam S0 = 2'd0;

```

```

localparam S1 = 2'd1;
localparam S2 = 2'd2;

reg np_en_pre_state;
reg [1:0] nstate;
reg signed [27:0] ntemp_ac, ntemp_as, ntemp_bc, ntemp_bs;

always @(posedge iClk or negedge iRst_n) begin
    if(!iRst_n) begin
        np_en_pre_state <= 1'b0;
    end
    else begin
        np_en_pre_state <= iP_en;
    end
end

always @(posedge iClk or negedge iRst_n) begin
    if(!iRst_n) begin
        ntemp_ac <= 28'd0;
        ntemp_as <= 28'd0;
        ntemp_bc <= 28'd0;
        ntemp_bs <= 28'd0;
        nstate <= S0;
        oId <= 12'd0;
        oIq <= 12'd0;
        oP_done <= 1'b0;
    end
    else begin
        case (nstate)
            S0: begin
                if((!np_en_pre_state) & iP_en) begin
                    ntemp_ac <= iIalpha * iCos;
                    ntemp_bc <= iIbeta * iCos;
                    ntemp_as <= iIalpha * iSin;
                    ntemp_bs <= iIbeta * iSin;
                    nstate <= S1;
                end
            end
            else begin
                nstate <= nstate;
                oP_done <= 1'b0;
            end
        end
        S1: begin
            nstate <= S0;
        end
    end
end

```

```

        oId <= $signed(ntemp_ac[26:15]) +
$signed(ntemp_bs[26:15]);
        oIq <= $signed(ntemp_bc[26:15]) -
$signed(ntemp_as[26:15]);
        oP_done <= 1'b1;
    end
    default: nstate <= S0;
endcase
end
end
endmodule

```

SVPWM 调制:

```

module SVPWM(
    iClk,
    iRst_n,
    iModulate_en,
    iValpha,
    iVbeta,
    oPWM_u,
    oPWM_v,
    oPWM_w,
    oModulate_done
);
    input wire iClk;
    input wire iRst_n;
    input wire iModulate_en;
    input wire [15:0] iValpha,iVbeta;
    output wire oPWM_u,oPWM_v,oPWM_w;
    output wire oModulate_done;

    wire [2:0] nsector;
    wire [15:0] nv1,nv2,nv3;
    wire nic_done,nas_done,ncw_done;
    wire [11:0] nccr_a,nccr_b,nccr_c;

    Inv_Clark inv_clark(
        .iClk(iClk),
        .iRst_n(iRst_n),
        .iIC_en(iModulate_en),
        .iValpha(iValpha),
        .iVbeta(iVbeta),
        .oV1(nv1),
        .oV2(nv2),

```

```
.oV3(nv3),
.oIC_done(nic_done)
);

SVPWM_Analyse_Sector svpwm_analyse_sector(
.iClk(iClk),
.iRst_n(iRst_n),
.iAS_en(nic_done),
.iA(!nv1[15]),
.iB(!nv2[15]),
.iC(!nv3[15]),
.oSector(nsector),
.oAS_done(nas_done)
);

SVPWM_Calculate_Workingtime svpwm_calculate_workingtime(
.iClk(iClk),
.iRst_n(iRst_n),
.iCW_en(nas_done),
.iSector(nsector),
.iV1(nv1),
.iV2(nv2),
.iV3(nv3),
.oCCR_a(nccr_a),
.oCCR_b(nccr_b),
.oCCR_c(nccr_c),
.oCW_done(ncw_done)
);

SVPWM_Modulate_PWM svpwm_modulate_pwm(
.iClk(iClk),
.iRst_n(iRst_n),
.iMP_en(ncw_done),
.iCCR_a(nccr_a),
.iCCR_b(nccr_b),
.iCCR_c(nccr_c),
.oPWM_u(oPWM_u),
.oPWM_v(oPWM_v),
.oPWM_w(oPWM_w),
.oMP_done(oModulate_done)
);

endmodule
```

```

module SVPWM_Analyse_Sector(
    iClk,
    iRst_n,
    iAS_en,
    iA,
    iB,
    iC,
    oSector,
    oAS_done
);
    input wire iClk;
    input wire iRst_n;
    input wire iAS_en;
    input wire iA,iB,iC;
    output reg [2:0] oSector;
    output reg oAS_done;

    localparam Sector_1 = 3'd1;
    localparam Sector_2 = 3'd2;
    localparam Sector_3 = 3'd3;
    localparam Sector_4 = 3'd4;
    localparam Sector_5 = 3'd5;
    localparam Sector_6 = 3'd6;

    localparam S0 = 2'b00;
    localparam S1 = 2'b01;
    localparam S2 = 2'b11;
    localparam S3 = 2'b10;

    reg [1:0] state;
    reg nas_en_pre_state;
    reg [2:0] ntemp_a,ntemp_b,ntemp_c,nN;

    always @(posedge iClk or negedge iRst_n) begin
        if(!iRst_n) begin
            nas_en_pre_state <= 1'b0;
        end
        else begin
            nas_en_pre_state <= iAS_en;
        end
    end

    always @(posedge iClk or negedge iRst_n) begin
        if(!iRst_n) begin

```

```

        state <= S0;
        ntemp_a <= 3'd0;
        ntemp_b <= 3'd0;
        ntemp_c <= 3'd0;
        nN <= 3'd0;
        oAS_done <= 1'b0;
        oSector <= 3'd0;
    end
    else begin
        case (state)
            S0: begin
                if((!nas_en_pre_state) & iAS_en) begin
                    ntemp_a <= iA;
                    ntemp_b <= iB;
                    ntemp_c <= iC;
                    state <= S1;
                end
            end
            else begin
                ntemp_a <= 3'd0;
                ntemp_b <= 3'd0;
                ntemp_c <= 3'd0;
                oAS_done <= 1'b0;
                state <= state;
            end
        end
    end
    S1: begin
        nN <= ntemp_a + (ntemp_b<<1) + (ntemp_c<<2);
        state <= S2;
    end
    S2: begin
        case (nN)
            3'd1: oSector <= Sector_2;
            3'd2: oSector <= Sector_6;
            3'd3: oSector <= Sector_1;
            3'd4: oSector <= Sector_4;
            3'd5: oSector <= Sector_3;
            3'd6: oSector <= Sector_5;
            default: ;
        endcase
        oAS_done <= 1'b1;
        state <= S0;
    end
    default: state <= S0;
endcase

```

```

        end
    end

endmodule

module SVPWM_Calculate_Workingtime(
    iClk,
    iRst_n,
    iCW_en,
    iSector,
    iV1,
    iV2,
    iV3,
    oCCR_a,
    oCCR_b,
    oCCR_c,
    oCW_done
);
    input wire iClk;
    input wire iRst_n;
    input wire iCW_en;
    input wire [2:0] iSector;
    input wire signed [15:0] iV1,iV2,iV3;
    output reg [11:0] oCCR_a,oCCR_b,oCCR_c;
    output reg oCW_done;

    localparam T = 13'd4999;

    localparam S0 = 2'b00;
    localparam S1 = 2'b01;
    localparam S2 = 2'b11;
    localparam S3 = 2'b10;

    localparam Sector_1 = 3'd1;
    localparam Sector_2 = 3'd2;
    localparam Sector_3 = 3'd3;
    localparam Sector_4 = 3'd4;
    localparam Sector_5 = 3'd5;
    localparam Sector_6 = 3'd6;

    reg ncw_en_pre_state;
    reg [1:0] state;
    reg signed [15:0] nux,nuy;
    reg [28:0] ntx,nty;

```



```

reg [13:0] nccr_a,nccr_b,nccr_c;

always @(posedge iClk or negedge iRst_n) begin
    if(!iRst_n) begin
        ncw_en_pre_state <= 1'b0;
    end
    else begin
        ncw_en_pre_state <= iCW_en;
    end
end

always @(posedge iClk or negedge iRst_n) begin
    if(!iRst_n) begin
        nux <= 16'd0;
        nuy <= 16'd0;
        ntx <= 29'd0;
        nty <= 29'd0;
        oCCR_a <= 12'd0;
        oCCR_b <= 12'd0;
        oCCR_c <= 12'd0;
        oCW_done <= 1'b0;
        state <= S0;
    end
    else begin
        case (state)
            S0: begin
                if((!ncw_en_pre_state) & iCW_en) begin
                    case (iSector)
                        Sector_1: begin
                            nux <= iV2;
                            nuy <= iV1;
                        end
                        Sector_2: begin
                            nux <= -iV2;
                            nuy <= -iV3;
                        end
                        Sector_3: begin
                            nux <= iV1;
                            nuy <= iV3;
                        end
                        Sector_4: begin
                            nux <= -iV1;
                            nuy <= -iV2;
                        end
                    end
                end
            end
        end
    end
end

```

```

        Sector_5: begin
            nux <= iV3;
            nuy <= iV2;
        end
        Sector_6: begin
            nux <= -iV3;
            nuy <= -iV1;
        end
        default: begin
            nux <= nux;
            nuy <= nuy;
        end
    endcase
    state <= S1;
end
else begin
    oCW_done <= 1'b0;
    state <= state;
end
end
S1: begin
    ntx <= (nux * T)>>15;
    nty <= (nuy * T)>>15;
    state <= S2;
end
S2: begin
    nccr_a <= (T - ntx[12:0] - nty[12:0])>>2;
    nccr_b <= (T + ntx[12:0] - nty[12:0])>>2;
    nccr_c <= (T + ntx[12:0] + nty[12:0])>>2;
    state <= S3;
end
S3: begin
    case (iSector)
        Sector_1: begin
            oCCR_a <= nccr_a[11:0];
            oCCR_b <= nccr_b[11:0];
            oCCR_c <= nccr_c[11:0];
        end
        Sector_2: begin
            oCCR_a <= nccr_b[11:0];
            oCCR_b <= nccr_a[11:0];
            oCCR_c <= nccr_c[11:0];
        end
        Sector_3: begin

```

```

        oCCR_a <= nccr_c[11:0];
        oCCR_b <= nccr_a[11:0];
        oCCR_c <= nccr_b[11:0];
    end
    Sector_4: begin
        oCCR_a <= nccr_c[11:0];
        oCCR_b <= nccr_b[11:0];
        oCCR_c <= nccr_a[11:0];
    end
    Sector_5: begin
        oCCR_a <= nccr_b[11:0];
        oCCR_b <= nccr_c[11:0];
        oCCR_c <= nccr_a[11:0];
    end
    Sector_6: begin
        oCCR_a <= nccr_a[11:0];
        oCCR_b <= nccr_c[11:0];
        oCCR_c <= nccr_b[11:0];
    end
    default: begin
        oCCR_a <= oCCR_a;
        oCCR_b <= oCCR_b;
        oCCR_c <= oCCR_c;
    end
endcase
oCW_done <= 1'b1;
state <= S0;
end
default: state <= S0;
endcase
end
end

endmodule

module SVPWM_Modulate_PWM(
    iClk,
    iRst_n,
    iMP_en,
    iCCR_a,
    iCCR_b,
    iCCR_c,
    oPWM_u,
    oPWM_v,

```

```

    oPWM_w,
    oMP_done
);
input wire iClk;
input wire iRst_n;
input wire iMP_en;
input wire [11:0] iCCR_a,iCCR_b,iCCR_c;
output wire oPWM_u,oPWM_v,oPWM_w;
output reg oMP_done;

localparam Sector_1 = 3'd1;
localparam Sector_2 = 3'd2;
localparam Sector_3 = 3'd3;
localparam Sector_4 = 3'd4;
localparam Sector_5 = 3'd5;
localparam Sector_6 = 3'd6;

localparam T = 13'd4999;

localparam S0 = 2'b00;
localparam S1 = 2'b01;
localparam S2 = 2'b11;
localparam S3 = 2'b10;

reg nmp_en_pre_state;
reg [11:0] ncount;
reg [1:0] state;
reg [11:0]
nccr_a_autoreload,nccr_b_autoreload,nccr_c_autoreload;
reg nflag_autoreload;

always @(posedge iClk or negedge iRst_n) begin
    if(!iRst_n) begin
        nmp_en_pre_state <= 1'b0;
    end
    else begin
        nmp_en_pre_state <= iMP_en;
    end
end

always @(posedge iClk or negedge iRst_n) begin
    if(!iRst_n) begin
        nccr_a_autoreload <= 12'd0;
        nccr_b_autoreload <= 12'd0;
    end
end

```

```

        nccr_c_autoreload <= 12'd0;
    end
    else begin
        if((!nmp_en_pre_state) & iMP_en) begin
            nccr_a_autoreload <= iCCR_a;
            nccr_b_autoreload <= iCCR_b;
            nccr_c_autoreload <= iCCR_c;
        end
        else if(nflag_autoreload) begin
            nccr_a_autoreload <= iCCR_a;
            nccr_b_autoreload <= iCCR_b;
            nccr_c_autoreload <= iCCR_c;
        end
        else begin
            nccr_a_autoreload <= nccr_a_autoreload;
            nccr_b_autoreload <= nccr_b_autoreload;
            nccr_c_autoreload <= nccr_c_autoreload;
        end
    end
end

always @(posedge iClk or negedge iRst_n) begin
    if(!iRst_n)begin
        nflag_autoreload <= 1'b0;
        ncount <= 11'd0;
        state <= S0;
        oMP_done <= 1'b0;
    end
    else begin
        case (state)
            S0: begin
                if((!nmp_en_pre_state) & iMP_en) begin
                    state <= S1;
                end
            end
            else begin
                nflag_autoreload <= 1'b0;
                ncount <= 11'd0;
                oMP_done <= 1'b0;
                state <= S0;
            end
        end
        S1: begin
            if(ncount == (T>>1)) begin
                nflag_autoreload <= 1'b1;
            end
        end
    end
end

```

```

        state <= S2;
    end
    else begin
        nflag_autoreload <= 1'b0;
        ncount <= ncount + 1'd1;
        oMP_done <= 1'b0;
    end
end
S2: begin
    if(ncount == 10'd0) begin
        nflag_autoreload <= 1'b1;
        oMP_done <= 1'b1;
        state <= S1;
    end
    else begin
        nflag_autoreload <= 1'b0;
        ncount <= ncount - 1'd1;
        oMP_done <= 1'b0;
    end
end
default: state <= S0;
endcase
end
end
assign oPWM_u = (ncount >= nccr_a_autoreload) ? 1'b1:1'b0;
assign oPWM_v = (ncount >= nccr_b_autoreload) ? 1'b1:1'b0;
assign oPWM_w = (ncount >= nccr_c_autoreload) ? 1'b1:1'b0;

endmodule

```

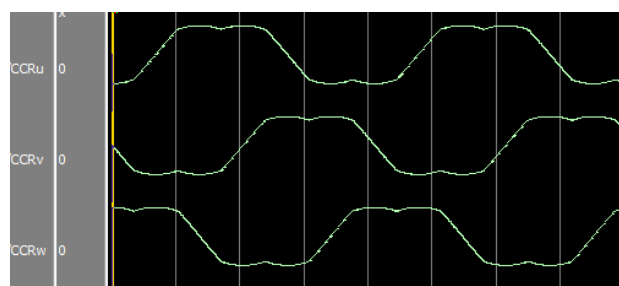


图 7 驼峰波输出(仿真)效果

9.6 FPGA 硬件调试



图 8 实物静态连接示意图

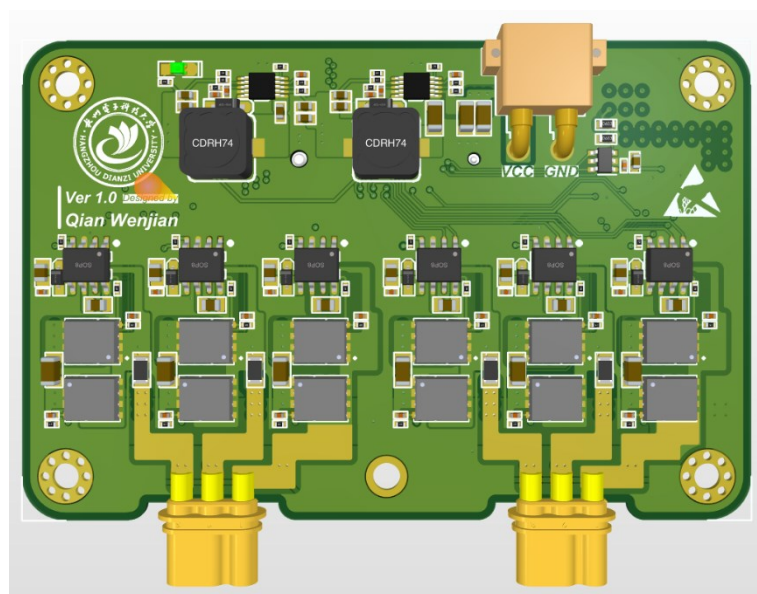


图 9 驱动电路板设计

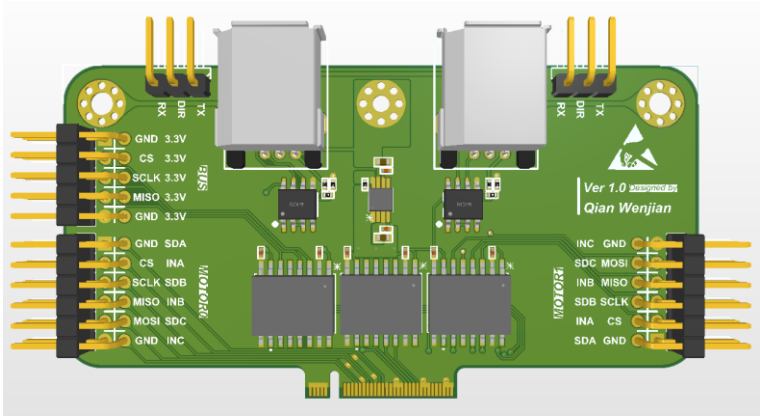


图 10 接口板设计

FOC 算法实现的直观表现为速度值或者位置值的表现，该实验以速度指标作为验证参照，采用 Signal Tap Logic Analyzer 进行编码器和模数转换器数值获取和处理，可以方便观察算法实现情况，下图所示为两种不同设定值情况下的算法表现：

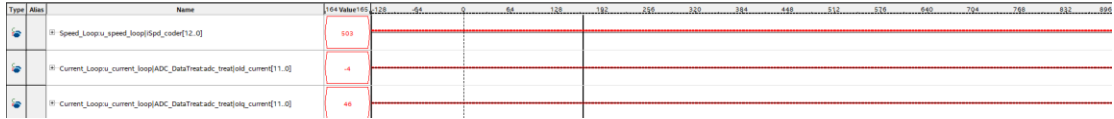


图 11 速度设定值为 500

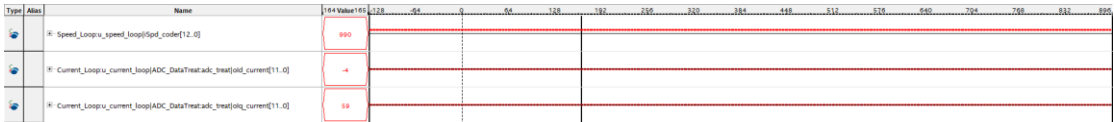


图 12 速度值设定为 1000

9.7 参考文献

https://blog.csdn.net/weixin_45263626/article/details/105358762

10. 基于 FPGA 的 FAST 角点检测算法 HLS 实现

10.1 算法原理

FAST 角点检测算法定义为在像素点的周围邻域内有足够多的像素点与该点处于不同的区域。应用到灰度图像中，即有足够多的像素点的灰度值大于该点的灰度值或小于该点的灰度值。

在图像中选择一个要被识别是否为兴趣点的像素 p 点，假设 p 点像素值为 I ，以半径为 3 个像素点选择 16 个像素点组成圆环，如图 1 所示。同时选取一个合适的阈值 t ，如果在这个大小为 16 个像素点的圆上有 n 个连续的像素点，并且他们的像素值都比 $I_p + t$ 大或者都比 $I_p - t$ 小，那么他就是一个角点。这里 n 的值可以设置为 12 或者 9。

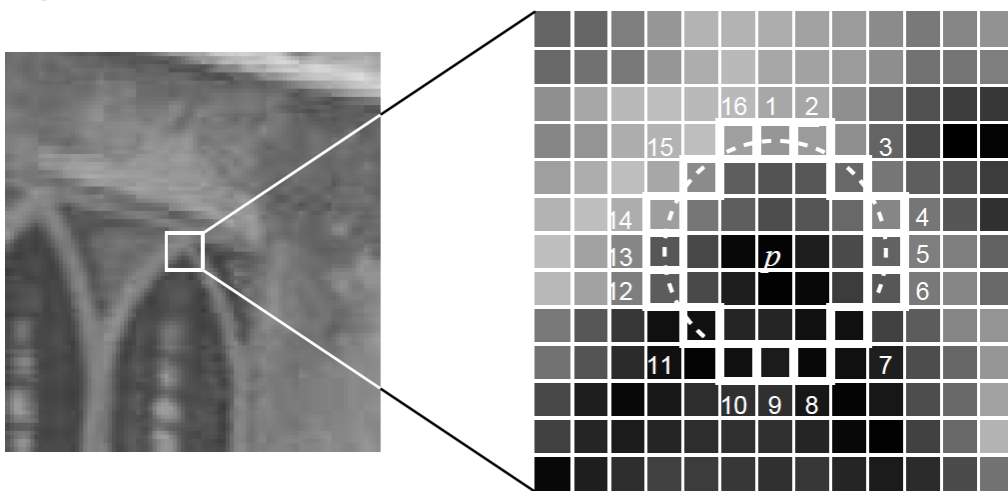


图 1.1 Brenham 圆

FAST 特征点检测是公认的比较快速的特征点检测方法，只利用周围像素比较的信息就可以得到特征点，简单，有效。FAST 特征检测算法来源于 corner 的定义，这个定义基于特征点周围的图像灰度值，检测候选特征点周围一圈的像素值，如果候选点周围领域内有足够多的像素点与该候选点的灰度值差别够大，则认为该候选点为一个特征点。公式如下：

$$N = \sum_{x \in \text{circle}(p)} |I(x) - I(p)| > \epsilon_t$$

其中 $I(p)$ 为检测点像素值， $I(x)$ 为周围像素点的像素值， ϵ_t 为选取的阈值。

FAST 角点具体检测步骤如下：

1) 以图像内任意一个像素作为候选特征点，定义一个以该像素为中心，半径为 3 的离散化 Brenham 圆作为检测模板，如图 1 所示，圆环上分布着 16 个像素点，记作 $P_1, P_2, \dots, P_{16}$ 。

2) 计算圆心像素 P 与 P_1, P_9 的像素灰度差，若其绝对值小于设定阈值，则将点 P 从候选特征点中剔除，否则进行下一步。

3) 计算圆心像素 P 与 P_1, P_5, P_9, P_{13} 的像素灰度差，若其中至少有 3 个绝对值大于设定阈值，则将点 P 保留，否则剔除。

4) 计算圆心像素 P 与全部 16 个的像素灰度差，若其中至少有 n 个绝对值大于设定阈值，则将 P 点认定为特征点。经验证， $n < 9$ 时，边缘点易被误认为是角点；不小于 9 的情况下， $n = 9$ 时，检测性能最佳，称作 FAST-9。

5) 非极大值抑制，计算步骤 4) 筛选出的特征点的 FAST 得分，即 16 个圆环像素与特征点 P 的灰度差的绝对值总和，若 P 点是周围 3×3 邻域内得分值最大的点，则保留，否则剔除。

10.2 C++验证代码

opencv 中默认采用 Fast-9-16（还包括 Fast-5-8, Fast-7-12）。即在周围取 16 个像素点，若超过连续 9 个点与中心点差值大于阈值即成为候选角点。角点强度计算方法采用最小的差值作为其角点强度值。

```
#include <stdio.h>
#include <iostream>
#include "cv.h"
#include "opencv2/highgui/highgui.hpp"
#include "opencv2/core/core.hpp"
#include "opencv2/features2d/features2d.hpp"
using namespace std;
using namespace cv;
int main( int argc, char** argv )
{
    Mat img_1 = imread( "F:\\Picture\\hotel.jpg" );
    if( !img_1.data )
    { return -1; }
    FastFeatureDetector detector(50,true); //第一个参数为阈值，第二个采用非最大值抑制
    std::vector<KeyPoint> keypoints_1;
    detector.detect( img_1, keypoints_1 );//调用 FAST_t 函数检测，见下面详细解析
    drawKeypoints(img_1,keypoints_1,img_1,Scalar::all(255));
    imshow("HOTEL",img_1);
    waitKey(0);
    return 0;
}

void FAST_t(InputArray _img, std::vector<KeyPoint>& keypoints, int threshold, bool
nonmax_suppression)
{
    Mat img = _img.getMat();
    const int K = patternSize/2, N = patternSize + K + 1;
    int i, j, k, pixel[25];
    makeOffsets(pixel, (int)img.step, patternSize);
    keypoints.clear();
    threshold = std::min(std::max(threshold, 0), 255);//保证阈值在 0-255 之间。
    uchar threshold_tab[512];
    for( i = -255; i <= 255; i++ )
        threshold_tab[i+255] = (uchar)(i < -threshold ? 1 : i > threshold ? 2 : 0);
    AutoBuffer<uchar> _buf((img.cols+16)*3*(sizeof(int) + sizeof(uchar)) + 128);
    uchar* buf[3];
    buf[0] = _buf; buf[1] = buf[0] + img.cols; buf[2] = buf[1] + img.cols;
    int* cpbuf[3];
    cpbuf[0] = (int*)alignPtr(buf[2] + img.cols, sizeof(int)) + 1;
    cpbuf[1] = cpbuf[0] + img.cols + 1;
```

```

        cpbuf[2] = cpbuf[1] + img.cols + 1;
        memset(buf[0], 0, img.cols*3);
        for(i = 3; i < img.rows-2; i++)
        {
            const uchar* ptr = img.ptr<uchar>(i) + 3;
            uchar* curr = buf[(i - 3)%3];
            int* cornerpos = cpbuf[(i - 3)%3];
            memset(curr, 0, img.cols);
            int ncorners = 0;
            if( i < img.rows - 3 ){
                j = 3;
                for( ; j < img.cols - 3; j++, ptr++){
                    int v = ptr[0];
                    const uchar* tab = &threshold_tab[0] - v + 255;
                    int d = tab[ptr[pixel[0]]] | tab[ptr[pixel[8]]];
                    if( d == 0 )
                        continue;
                    d &= tab[ptr[pixel[2]]] | tab[ptr[pixel[10]]];
                    d &= tab[ptr[pixel[4]]] | tab[ptr[pixel[12]]];
                    d &= tab[ptr[pixel[6]]] | tab[ptr[pixel[14]]];
                    if( d == 0 )
                        continue;
                    d &= tab[ptr[pixel[1]]] | tab[ptr[pixel[9]]];
                    d &= tab[ptr[pixel[3]]] | tab[ptr[pixel[11]]];
                    d &= tab[ptr[pixel[5]]] | tab[ptr[pixel[13]]];
                    d &= tab[ptr[pixel[7]]] | tab[ptr[pixel[15]]];
                    if( d & 1 ){
                        int vt = v - threshold, count = 0;
                        for( k = 0; k < N; k++){
                            int x = ptr[pixel[k]];
                            if(x < vt){
                                if( ++count > K ){
                                    cornerpos[ncorners++] = j;
                                    if(nonmax_suppression)//非最大值抑制
                                        curr[j]=(uchar)cornerScore<patternSize>(ptr,pixel,threshold);
                                    break;
                                }
                            }
                        }
                    }
                    else
                        count = 0;
                }
            }
            if( d & 2 ){
                int vt = v + threshold, count = 0;

```

```

        for( k = 0; k < N; k++ ){
            int x = ptr[pixel[k]];
            if(x > vt){
                if( ++count > K ){
                    cornerpos[ncorners++] = j;
                    if(nonmax_suppression)
                        curr[j]=(uchar)cornerScore<patternSize>(ptr,pixel,threshold);
                    break;
                }
            }
            else
                count = 0;
        }
    }
}

cornerpos[-1] = ncorners;//存储第 i 行上的角点总数量
if( i == 3 )
    continue;
/*与邻域的 8 个角点响应值做比较，非角点的响应值为 0*/
const uchar* prev = buf[(i - 4 + 3)%3]; //相邻的两行
const uchar* pprev = buf[(i - 5 + 3)%3];//
cornerpos = cdbuf[(i - 4 + 3)%3];//存储角点的列位置
ncorners = cornerpos[-1]; //存储第 i 行上的角点总数量
for( k = 0; k < ncorners; k++ ){
    j = cornerpos[k];
    int score = prev[j];
    if(!nonmax_suppression||((score>prev[j+1]&&score>prev[j-1]&&score>pprev[j-1]&&score>pprev[j]&&score>pprev[j+1]&&score>curr[j-1]&&score>curr[j]&&score>curr[j+1]))){
        keypoints.push_back(KeyPoint((float)j, (float)(i-1), 7.f, -1, (float)score));
    }
}
}
}

角点的强度计算方法：若采用 Fast-9-16,计算连续的 9 个位置与中心位置的差值的绝对值，取最小的一个差值作为其强度值。
int cornerScore<16>(const uchar* ptr,const int pixel[],int threshold)//角点强度{
const int K = 8, N = K*3 + 1;
int k, v = ptr[0];
short d[N];
for( k = 0; k < N; k++ )    //计算与周围 16 个像素点的差值,保存在 d[k]中
    d[k] = (short)(v - ptr[pixel[k]]);
int a0 = threshold;

```

10.3 FPGA 实现架构



10.4 FPGA 实现代码

FPGA 的算法采用高层次综合（HLS）的方式来实现，其中使用到了 vivado HLS 提供的视频库函数。fast_corner 为本次设计实现的角点检测函数，该函数内使用了 HLS 的库函数，其中用到的主要函数为 AXIvideo2Mat（可综合的视频接口转换函数）、hls::CvtColor 将 RGB 图像转换为灰度图像、hls::FASTX（HLS 提供的可综合角点检测函数）、hls::Dilate 用于扩展图像、hls::Mat2AXIvideo 将 MAT 类图像转换为 AXI 接口。

```
fast_corner 代码
#include "hls_video.h"
// maximum image size
#define MAX_WIDTH 1920
#define MAX_HEIGHT 1080
// I/O Image Settings
#define INPUT_IMAGE "test.bmp"
#define OUTPUT_IMAGE "result.bmp"
#define OUTPUT_IMAGE_GOLDEN "result_golden.bmp"
// typedef video library core structures
typedef hls::stream<ap_axi<32,1,1,1>> AXI_STREAM;
typedef hls::Scalar<3, unsigned char> RGB_PIXEL;
typedef hls::Mat<MAX_HEIGHT, MAX_WIDTH, HLS_8UC3> RGB_IMAGE;
typedef hls::Mat<MAX_HEIGHT, MAX_WIDTH, HLS_8UC1> GRAY_IMAGE;
// top level function for HW synthesis
void fast_corner(AXI_STREAM& INPUT_STREAM, AXI_STREAM&
OUTPUT_STREAM, int rows, int cols, int threshold)
{
#pragma HLS INTERFACE axis port=INPUT_STREAM
#pragma HLS INTERFACE axis port=OUTPUT_STREAM
#pragma HLS RESOURCE core=AXI_SLAVE variable=rows metadata="-bus_bundle
CONTROL_BUS"
#pragma HLS RESOURCE core=AXI_SLAVE variable=cols metadata="-bus_bundle
CONTROL_BUS"
#pragma HLS RESOURCE core=AXI_SLAVE variable=threshold metadata="-bus_bundle
CONTROL_BUS"
#pragma HLS RESOURCE core=AXI_SLAVE variable=return metadata="-bus_bundle
CONTROL_BUS"
#pragma HLS INTERFACE ap_stable port=rows
#pragma HLS INTERFACE ap_stable port=cols
#pragma HLS INTERFACE ap_stable port=threshold
RGB_IMAGE _src(rows, cols);
RGB_IMAGE _dst(rows, cols);
RGB_IMAGE src0(rows, cols);
RGB_IMAGE src1(rows, cols);
GRAY_IMAGE mask(rows, cols);
GRAY_IMAGE dmask(rows, cols);
```

```

GRAY_IMAGE      gray(rows, cols);
#pragma HLS dataflow
#pragma HLS stream depth=20000 variable=src1.data_stream
hls::AXIvideo2Mat(INPUT_STREAM, _src);
hls::Scalar<3,unsigned char> color(255,0,0);
hls::Duplicate(_src,src0,src1);
hls::CvtColor<HLS_BGR2GRAY>{(src0,gray);
hls::FASTX(gray,mask,threshold,true);
hls::Dilate(mask,dmask);
hls::PaintMask(src1,dmask,_dst,color);
hls::Mat2AXIvideo(_dst, OUTPUT_STREAM);
}

```

Testbench 代码通过读取一张图片，并将图片通过 AXI_STREAM 接口函数传送给目标函数接口。通过 IplImage2AXIvideo 将图片从 MAT 格式的图片转换为 AXI 的接口传送给 fast_corner 角点检测函数，fast_corner 检点检测函数处理完成后通过 AXIvideo2IplImage 将 AXI 格式的图片转换为 MAT 类的格式送到 cvShowImage 进行显示。

Testbench 代码

```

int main (int argc, char** argv)
{
    IplImage* src = cvLoadImage(INPUT_IMAGE);
    IplImage* dst = cvCreateImage(cvGetSize(src), src->depth, src->nChannels);
    cvShowImage("hls_src", src);
    AXI_STREAM  src_axi, dst_axi;
    IplImage2AXIvideo(src, src_axi);
    fast_corner(src_axi, dst_axi, src->height, src->width, 20);
    AXIvideo2IplImage(dst_axi, dst);
    cvShowImage("hls_dst", dst);
    cvSaveImage(OUTPUT_IMAGE,dst);
    waitKey(0);
    return 0;
}

```

10.5 X86 仿真结果



图3 原始图片

FPGA 仿真结果:

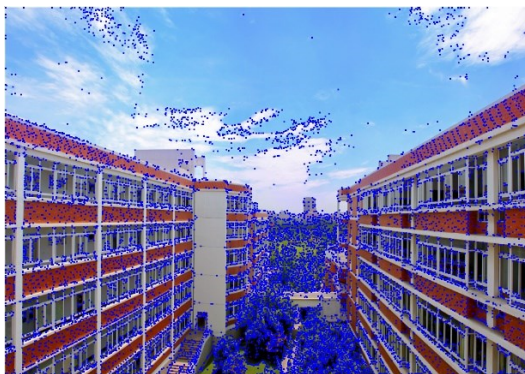


图 2 阈值设为 10

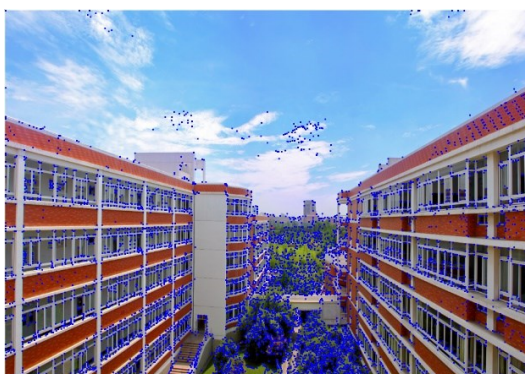


图 3 阈值设为 20



图 4 阈值设为 30

图 5 阈值设为 40

10.6 参考文献

- [1] Trajkovic M, Hedley M. Fast corner detection. Image and Vision Computing, 1988, 16(2): 75 - 87.
- [2] Rosten E, Drummond T. Machine learning for high-speed corner detection. Proceedings of the 9th European Conference on Computer Vision. Graz, Austria. 2006. 430 - 443.
- [3] Park S, Kim G, Park J, et al. A 1. 5nJ/pixel super-resolution enhanced FAST corner detection processor for high accuracy AR. Proceedings of the ESSCIRC 2014 —40th European Solid State Circuits Conference. Venice Lido, Italy. 2014. 191 - 194.
- [4] 燕鹏, 安如. 基于 FAST 改进的快速角点探测算法 [J]. 红外与激光工程, 2009, 38 (6) : 1104-1108, 1115. YAN P, AN R. Improved fast corner detection algorithm

based on FAST [J]. Infrared and Laser Engineering, 2009, 38(6) : 1104-1108, 1115.

[5] Wang LF, Zhao YN, Qin PL, et al. Fast Corner detection based on ORB and GroupSAC in complex scenes video image. Science Technology and Engineering, 2017, 17(2): 88 - 94.

[6] Jin YT, Wang WL, Zhao YW, et al. Fast corner detector based on chord-to-point distance accumulation. Computer Science, 2014, 41(4): 306 - 308, 313.